Tutorial de MkDocs

Bem-vindo! Este tutorial vai te ensinar, passo a passo e em português, a criar e personalizar um site de documentação com o **MkDocs** — exatamente como foi feito neste projeto.

Ao longo das próximas páginas você vai aprender a:

- **Instalar os pacotes necessários** (`mkdocs`, `mkdocs-material`, `mkdocstrings[python]` e `mkdocstrings-typescript`) usando `uv`.
- **Configurar o arquivo** `mkdocs.yml`, entendendo para que servem as seções `nav`, `theme`, `plugins` e `markdown_extensions`.
- **Personalizar o visual do site**, ajustando fontes, paleta de cores, logo e favicon do tema Material.
- **Integrar com GitHub**, usando o GitHub CLI (`gh`) para autenticar, trabalhar com repositórios e pull requests e acompanhar o GitHub Actions.
- **Gerar documentação a partir de código-fonte** usando o plugin `mkdocstrings`, comparando um bloco de código estático com a documentação extraída automaticamente da docstring de uma função Python real do projeto.
- **Seguir um roteiro prático completo**, equivalente ao notebook `mkdocks.ipynb`, cobrindo desde a instalação dos pacotes até a visualização do site no navegador.

Não é necessário nenhum conhecimento prévio de MkDocs: cada página explica os conceitos do zero, com comandos reais que você pode copiar e executar no seu próprio terminal.

O que é o MkDocs e por que usá-lo?

O MkDocs é uma ferramenta que transforma arquivos Markdown simples (como este que você está lendo) em um site de documentação estático, pronto para ser publicado. Em vez de escrever HTML e CSS manualmente, você escreve apenas texto em Markdown dentro da pasta `docs/`, e o MkDocs se encarrega de gerar as páginas, o menu de navegação, a busca e o visual do site.

Algumas razões para usar o MkDocs:

- **Simplicidade**: basta escrever Markdown; não é preciso saber desenvolvimento web.
- **Tema profissional pronto para uso**: com o tema Material for MkDocs, o site já nasce com um visual moderno, responsivo e com modo claro/escuro.
- **Documentação de código automática**: plugins como o `mkdocstrings` leem as docstrings do seu código-fonte (Python, TypeScript, etc.) e geram a documentação de funções e classes automaticamente, sem duplicar esforço.

- **Configuração centralizada**: todo o comportamento do site — navegação, tema, plugins e extensões — é definido em um único arquivo, o `mkdocs.yml`.
- **Fluxo de trabalho simples**: com dois comandos (`mkdocs serve` para visualizar localmente com recarregamento automático, e `mkdocs build` para gerar o site final) você já tem um fluxo de documentação completo.

Agora que você já sabe o que é o MkDocs e por que ele é útil, siga as páginas abaixo na ordem sugerida para colocar tudo em prática:

1. Instalação
2. Arquivos Markdown e navegação
3. Configuração do mkdocs.yml
4. Identidade visual
5. Exemplos de código
6. Exemplos do notebook
7. Integração com GitHub
8. GitHub CLI (`gh`)

2026-11-12 2026-11-11

Instalação

1. Introdução: o que vamos instalar e por quê

Antes de configurar ou escrever qualquer página de documentação, precisamos preparar o ambiente instalando as ferramentas que tornam tudo isso possível. Neste projeto, usamos quatro pacotes Python que trabalham juntos:

- **`mkdocs`**: é o gerador de site estático. Ele lê arquivos Markdown (como este que você está lendo agora) e os transforma em um site HTML completo, com navegação, busca e layout prontos.
- **`mkdocs-material`**: é o tema visual "Material for MkDocs". Sem ele, o MkDocs ainda funciona, mas usa um tema padrão bem simples. O Material adiciona um visual moderno, modo claro/escuro, ícones, busca aprimorada e diversas opções de personalização.
- **`mkdocstrings`** (com o extra `python`): é o plugin que gera documentação **automaticamente** a partir dos *docstrings* do código-fonte. Em vez de copiar e colar manualmente a explicação de uma função no Markdown, o `mkdocstrings` lê o docstring diretamente do código Python e o renderiza como parte da página.

Com esses três pacotes, já é possível ter um site de documentação completo, com tema bonito e documentação de código gerada automaticamente a partir do Python. Neste projeto também instalamos um quarto pacote, explicado a seguir.

2. Pré-requisitos

Antes de instalar os pacotes, confirme que seu ambiente atende aos seguintes requisitos:

- **Python 3.12 ou superior** instalado na máquina. Você pode verificar a versão com:

```
python --version
```

- **`uv` instalado**. O `uv` é o gerenciador de pacotes e ambientes Python usado neste projeto (ele substitui o uso combinado de `pip` + `venv` por uma ferramenta única e mais rápida). O tutorial passo a passo está em uv: gerenciar projetos e dependências Python. Para verificar se já está instalado:

```
uv --version
```

Se o comando não for reconhecido, siga as instruções de instalação do `uv` para o seu sistema operacional antes de continuar.

3. Comando de instalação do projeto

O projeto é gerenciado pelo `uv`, então a forma recomendada de instalar os pacotes é usando o comando `uv add`. Execute o comando abaixo na raiz do projeto:

```
uv add mkdocs mkdocs-material "mkdocstrings[python]" mkdocstrings-typescript
```

Vamos entender esse comando passo a passo:

- `uv add` diz ao `uv` para adicionar um ou mais pacotes como dependências do projeto, registrando-os no `pyproject.toml` e instalando-os no ambiente virtual gerenciado pelo `uv`.
- `mkdocs` e `mkdocs-material` são informados como nomes de pacote simples, sem nenhuma sintaxe especial.
- `"mkdocstrings[python]"` está entre aspas e usa a sintaxe de **extra** (`[python]`). Isso é necessário porque:
- O pacote `mkdocstrings` é um núcleo genérico, que por si só não sabe documentar nenhuma linguagem específica. Ele precisa de um *handler* (manipulador) para cada linguagem que ele deve suportar.
- O extra `[python]` instala automaticamente o handler Python do `mkdocstrings` (o pacote `mkdocstrings-python`) junto com o núcleo, habilitando a documentação de código Python a partir de docstrings.
- As aspas são necessárias porque os colchetes (`[` e `]`) têm significado especial em muitos shells (por exemplo, são usados para expansão de glob). Colocar o pacote entre aspas garante que o shell trate `mkdocstrings[python]` como um texto literal e o passe exatamente assim para o `uv`, em vez de tentar interpretá-lo.
- `mkdocstrings-typescript` é instalado como um pacote separado, e não como um extra. Esse pacote adiciona um **handler para TypeScript** ao `mkdocstrings`, permitindo documentar código `.ts` da mesma forma que o handler Python documenta código `.py`. Ele é útil caso o projeto venha a incluir arquivos TypeScript no futuro: o mesmo mecanismo de diretivas do `mkdocstrings` poderá ser usado para gerar documentação a partir desse código, sem precisar de outra ferramenta.

Instalar Node.js, npm e TypeDoc para documentação TypeScript

O `uv add` e o `uv sync` instalam dependências Python; eles não instalam o Node.js, o npm ou o TypeDoc. Quando uma página usa o handler TypeScript do `mkdocstrings`, o build precisa executar o comando `typedoc`. Por isso, além dos pacotes Python, instale o Node.js (que inclui o npm) e o TypeDoc.

1. Instale a versão LTS do Node.js. O npm é instalado junto com ele.
2. Feche e abra novamente o terminal e confirme que os comandos estão disponíveis:

```
node --version
```

```
npm --version
```

1. Se Node.js estiver instalado no local padrão do Windows, mas `node` ou `npm` não forem reconhecidos no PowerShell, adicione temporariamente os diretórios ao `PATH` da janela atual:

```
$env:Path = "C:\Program Files\nodejs;$env:APPDATA\npm;$env:Path"
node --version
npm --version
```

Para manter a correção em novos terminais, abra **Editar as variáveis de ambiente do sistema → Variáveis de Ambiente →** `Path` (variáveis do usuário) e adicione `C:\Program Files\nodejs` e `%APPDATA%\npm`. Depois, abra um novo terminal. Se o Node.js estiver instalado em outro diretório, use o caminho real da instalação.

1. Instale o TypeDoc globalmente pelo npm e confirme que o comando está disponível:

```
npm install --global typedoc
typedoc --version
```

Se `node` ou `npm` não forem reconhecidos, o terminal não está encontrando a instalação do Node.js no `PATH`. Se o build falhar com `FileNotFoundError: [WinError 2]`, confira também se o TypeDoc foi instalado globalmente e se `%APPDATA%\npm` está no `PATH`. Veja os passos detalhados na página Exemplos de código.

4. O que cada pacote faz

Para deixar bem claro o papel de cada dependência, segue um resumo individual:

`mkdocs`

É o motor principal. Lê os arquivos Markdown da pasta `docs/`, aplica a configuração do `mkdocs.yml` (navegação, tema, plugins, extensões) e gera o site estático em HTML/CSS/JS. É ele que fornece os comandos `mkdocs build` (gerar o site) e `mkdocs serve` (visualizar localmente com recarregamento automático).

`mkdocs-material`

É um tema para o MkDocs, instalado como um pacote independente e ativado na chave `theme.name: material` do `mkdocs.yml`. Fornece a aparência visual do site (cores, tipografia, modo claro/escuro), componentes de interface (como blocos de admonição e abas) e recursos de navegação (busca, índice lateral, botão de topo).

`mkdocstrings[python]`

É o plugin que conecta o MkDocs ao código-fonte Python. Quando uma página Markdown contém uma diretiva como `teste.calcular_media`, o `mkdocstrings` (através do handler Python instalado pelo extra `[python]`) importa o módulo indicado, localiza a função ou classe, lê seu docstring e gera a seção de documentação diretamente no HTML final, formatando parâmetros, tipos de retorno e exceções.

`mkdocstrings-typescript`

É o handler adicional que ensina o `mkdocstrings` a entender código TypeScript. Ele segue a mesma ideia do handler Python, mas analisa arquivos `.ts` / `.tsx` em vez de `.py`. Mesmo que o projeto ainda não tenha código TypeScript, instalar esse handler desde já deixa a configuração do `mkdocstrings` pronta (seção `plugins.mkdocstrings.handlers.typescript` no `mkdocs.yml`) para o caso de o projeto crescer e passar a incluir esse tipo de arquivo.

5. Alternativa: instalação com `pip`

Se você não estiver usando `uv` (por exemplo, em um ambiente que só tem `pip` disponível), é possível instalar os mesmos quatro pacotes com:

```
pip install mkdocs mkdocs-material "mkdocstrings[python]" mkdocstrings-typescript
```

O significado de cada parte do comando é o mesmo explicado na seção 3: o extra `[python]` instala o handler Python do `mkdocstrings`, e as aspas evitam que o shell interprete os colchetes como caracteres especiais. A única diferença é que o `pip` instala os pacotes no ambiente Python ativo no momento (por exemplo, um `venv` ativado manualmente), em vez de gerenciar o ambiente e o `pyproject.toml` automaticamente como o `uv` faz.

6. Verificando a instalação

Depois de instalar os pacotes, confirme que tudo funcionou corretamente verificando a versão do MkDocs:

```
uv run mkdocs --version
```

Se você instalou com `pip`, o comando equivalente é:

```
mkdocs --version
```

Em ambos os casos, o comando deve exibir o número da versão instalada do `mkdocs` (por exemplo, `mkdocs, version 1.6.1 from ...`), sem nenhuma mensagem de erro. Se o comando falhar com "comando não encontrado", revise se o ambiente virtual correto está ativo (ou use `uv run`, que já ativa o ambiente do projeto automaticamente) e repita a instalação da seção 3.

7. Diagnóstico de imagens e arquivos JavaScript que não carregam

Se o navegador ou o exportador de PDF registrar `ERR_FILE_NOT_FOUND`, confira se o arquivo existe dentro de `docs/`. Os caminhos configurados em `theme.logo`, `theme.favicon`, `extra_css` e `extra_javascript` são relativos ao diretório de documentação, não à raiz do repositório.

Por exemplo, estas configurações:

```
theme:
  logo: assets/logo.png
  favicon: assets/favicon.svg

extra_javascript:
  - javascripts/mathjax.js
```

exigem que os arquivos estejam nestes locais:

```
docs/
├── assets/
│   ├── logo.png
│   └── favicon.svg
└── javascripts/
    └── mathjax.js
```

No PowerShell, confirme os arquivos antes de gerar o site:

```
Test-Path .\docs\assets\logo.png
Test-Path .\docs\assets\favicon.svg
Test-Path .\docs\javascripts\mathjax.js
```

Se `extra_javascript` aponta para um arquivo que não existe, crie-o no caminho esperado ou remova a referência do `mkdocs.yml` se ela não for necessária. O MkDocs copia arquivos existentes de `docs/` para `site/`; não cria esses arquivos por causa da configuração.

Confira também se a extensão corresponde ao formato real da imagem. Renomear um PNG para `.ico` não o converte em ícone ICO: use `.png` no arquivo e na configuração, ou converta-o para ICO de verdade. Isso evita que o servidor envie um tipo de conteúdo incompatível com os bytes da imagem.

Em imagens inseridas no Markdown, use o caminho relativo ao arquivo dentro de `docs/`. Por exemplo, em `docs/index.md`, para exibir `docs/assets/Imagem13.jpg`, escreva:

```
![Logotipo](assets/Imagem13.jpg){: width="300" align="right" }
```

Não use `/docs/assets/Imagem13.jpg`: `docs/` é o diretório-fonte do MkDocs e não faz parte da URL publicada. Um `/` inicial aponta para a raiz do domínio. Confirme tanto o arquivo-fonte quanto o arquivo copiado para o site:

```
Test-Path .\docs\assets\Imagem13.jpg
Test-Path .\site\assets\Imagem13.jpg
```

Depois de corrigir os caminhos, gere o site novamente e confirme que os recursos foram copiados:

```
uv run mkdocs build --clean
Test-Path .\site\assets\logo.png
Test-Path .\site\javascripts\mathjax.js
```

Se os arquivos estiverem presentes mesmo assim, abra as ferramentas de desenvolvedor do navegador e confira a aba **Network**: ela mostra a URL requisitada e o status HTTP que falhou. Verifique também letras maiúsculas/minúsculas nos nomes; isso importa quando o site é publicado no Linux pelo GitHub Actions.

2026-11-13 2026-11-11

Arquivos Markdown e navegação

Este capítulo mostra como organizar os arquivos de conteúdo de um projeto MkDocs, escrever páginas com títulos, listas e links e controlar onde cada página aparece nos menus do site.

Como organizar as pastas

O arquivo `mkdocs.yml` fica na raiz do repositório, e o conteúdo publicado fica dentro da pasta definida por `docs_dir`. Neste projeto, `docs_dir: docs`, então páginas, imagens e outros recursos do site devem ficar em `docs/` — não na raiz do repositório.

Atualmente, este projeto mantém os capítulos diretamente em `docs/` e separa os recursos estáticos em suas próprias pastas:

```
projeto/
├── mkdocs.yml
└── docs/
    ├── index.md
    ├── instalacao.md
    ├── configuracao-yaml.md
    ├── markdown-e-navegacao.md
    └── assets/
        ├── logo.png
        └── exemplo.png
```

Use `docs/index.md` como página inicial. Para uma documentação pequena ou média, manter os Markdown diretamente em `docs/` é simples e funciona bem. Em projetos maiores, agrupe capítulos relacionados em subpastas e crie subpastas dentro de `assets/` para separar imagens, diagramas e outros recursos.

Os nomes de arquivos devem ser curtos e descritivos. Prefira letras minúsculas, hífens entre palavras e evite espaços, acentos e nomes genéricos como `pagina1.md`.

Cada arquivo `.md` dentro de `docs/` pode se tornar uma página do site, mas a localização física e a posição nos menus são coisas diferentes: pastas organizam os arquivos; a configuração `nav` determina a hierarquia exibida ao leitor.

Se a documentação crescer, uma estrutura alternativa pode separar os capítulos por assunto:

```
docs/
├── index.md
└── guia/
    ├── index.md
    ├── instalacao.md
    └── configuracao.md
```

o caminho usado no `nav` para a página de instalação é `guia/instalacao.md`, relativo a `docs/`. Em links dentro de Markdown, o caminho é relativo ao arquivo atual: em `docs/guia/instalacao.md`, use `[Configuração](configuracao.md)` para apontar a `docs/guia/configuracao.md`. Para um recurso compartilhado em `docs/assets/imagens/exemplo.png`, o mesmo arquivo `instalacao.md` pode usar `![Exemplo](../assets/imagens/exemplo.png)`.

Como escrever uma página

Markdown usa caracteres simples para indicar a estrutura do conteúdo. Um arquivo de página normalmente começa com um título principal e segue com subtítulos:

```
# Instalação

Uma introdução breve sobre o objetivo desta página.

## Pré-requisitos

Confira os itens necessários antes de começar.

### Instalar as dependências

Siga as etapas na ordem indicada.
```

Use apenas um título de nível 1 (`#`) para o título principal. Organize os assuntos com `##` e detalhes relacionados com `###`; não pule níveis sem motivo. O MkDocs Material usa esses títulos para montar o sumário da página e, conforme o tamanho da tela e a configuração, exibi-lo na navegação lateral direita.

Tópicos e passos

Use hífen para listas de tópicos e números para instruções que devem ser seguidas em ordem:

```
## O que você vai precisar

- Python instalado
- A ferramenta `uv`

## Etapas

1. Abra o terminal na pasta do projeto.
2. Instale as dependências.
3. Inicie o servidor local.
```

Links, código e imagens

Links internos apontam para outros arquivos Markdown. Blocos de código devem indicar a linguagem para que o tema aplique o realce de sintaxe:

Consulte também Configuração.

```
uv run mkdocs serve
```

Imagens e outros arquivos estáticos devem ficar dentro de `docs/`, por exemplo em `docs/assets/`. A partir de um Markdown localizado diretamente em `docs/`, uma imagem pode ser referenciada assim:

```
![Descrição da imagem](assets/exemplo.png)
```

Escreva um texto alternativo (`alt`) que explique o conteúdo visual. Para exemplos de dimensionamento, legenda e ampliação de imagens, veja Imagens e recursos visuais.

Adicionar uma página à navegação

Criar um arquivo Markdown não garante que ele apareça no menu. Com um `nav` explícito, adicione o arquivo à lista no `mkdocs.yml`, usando um título para o menu e o caminho relativo a `docs/`:

```yaml
nav:
  - Início: index.md
  - Começar:
      - Instalação: instalacao.md
      - "Markdown e navegação": markdown-e-navegacao.md
      - Configuração: configuracao-yaml.md
      - Guia:
          - guia/index.md
          - Instalação detalhada: guia/instalacao.md
          - Configuração detalhada: guia/configuracao.md
```

A indentação define os níveis da hierarquia: `Começar` é uma seção e os itens indentados são suas páginas. Os títulos definem os rótulos que o leitor vê; eles não precisam ser iguais ao nome do arquivo. A ordem das entradas determina a ordem no menu.

Quando uma seção tem sua própria página de índice, como `guia/index.md`, colocá-la como primeiro item do grupo torna o título da seção um link para essa página. O recurso `navigation.indexes` já está habilitado neste projeto e permite esse comportamento. Cada arquivo listado precisa existir dentro de `docs/`.

Depois de editar o `nav`, confira se todos os caminhos existem e execute:

```
uv run mkdocs build --strict
```

Esse projeto mantém a navegação no `mkdocs.yml`. Se o `nav` explícito for removido, plugins como `awesome-pages` podem participar da descoberta e organização automática das páginas, mas essa é outra estratégia de configuração.

Menu superior e navegação lateral

Neste projeto, `navigation.tabs` e o `nav` trabalham juntos:

- **Menu superior:** os itens de primeiro nível (`Início`, `Começar`, `Conteúdo`, `Ferramentas` e `GitHub`) aparecem como abas. Por isso, capítulos novos devem ser colocados dentro de um grupo existente, como `Começar`, para não criar uma aba adicional no cabeçalho.
- **Menu lateral esquerdo:** ao selecionar uma aba, as páginas e subseções listadas dentro daquele grupo são exibidas na navegação lateral. `navigation.sections` organiza os itens em seções; os rótulos e a ordem vêm do `nav`.
- **Sumário lateral direito:** é montado automaticamente a partir dos títulos da página. A extensão `toc` já está habilitada neste projeto, e `permalink: true` adiciona links diretos aos títulos. Não é preciso cadastrar cada título no `nav`.

O recurso Material `toc.integrate` pode mover os itens do sumário da página para a navegação lateral esquerda. Para usá-lo, inclua-o em `theme.features` no `mkdocs.yml`:

```yaml
theme:
  features:
    - navigation.tabs
    - navigation.sections
    - toc.integrate
```

Sem `toc.integrate`, mantenha o sumário independente na lateral direita, como no comportamento padrão do Material em telas largas. Em telas menores, o tema adapta os painéis de navegação ao espaço disponível.

Resumo da configuração

`nav` controla quais páginas e grupos aparecem na navegação; os títulos Markdown (`#`, `##`, `###`) controlam o sumário da página. O cabeçalho superior exibe os grupos de primeiro nível quando `navigation.tabs` está ativo.

2026-11-12 2026-11-12

Configuração do `mkdocs.yml`

O arquivo `mkdocs.yml`, na raiz do projeto, é o **arquivo de configuração central** do MkDocs. Nele você define o título do site, qual tema usar, quais plugins habilitar, quais extensões de Markdown ativar e qual é a estrutura do menu de navegação. É a partir dele que os comandos `mkdocs build` e `mkdocs serve` sabem exatamente como montar o site.

Esta página explica, em detalhes e passo a passo, as quatro seções principais do `mkdocs.yml` deste projeto: `nav`, `theme`, `plugins` e `markdown_extensions`. Todos os trechos de YAML mostrados abaixo foram copiados diretamente do `mkdocs.yml` real do projeto.

`nav`: a estrutura de navegação do site

```yaml
nav:
  - Início: index.md
  - Começar:
      - Instalação: instalacao.md
      - "Markdown e navegação": markdown-e-navegacao.md
      - Configuração: configuracao-yaml.md
      - "Identidade visual": personalizacao.md
  - Conteúdo:
      - "Exemplos de código": exemplos-codigo.md
      - Notebook: exemplos-notebook.md
      - Imagens: imagens-recursos-visuais.md
      - Diagramas: diagramas-arquitetura.md
  - Ferramentas:
      - Preview: playwright-preview.md
      - "Docker e linguagens": dockerfile-outras-linguagens.md
      - Extensões: extensoes-pymdownx.md
      - Plugins: outros-plugins.md
      - "Pacotes utilizados": pacotes-utilizados.md
      - Referências: referencias.md
  - GitHub:
      - Integração: integracao-github.md
      - "CLI (`gh`)": github-cli.md
```

A seção `nav` define o **menu de navegação** do site gerado. Além de páginas individuais, ela permite agrupar páginas em seções aninhadas. Com `navigation.tabs` ativo, as seções de primeiro nível aparecem como abas no cabeçalho; os capítulos ficam dentro delas, evitando que cada página ocupe uma aba própria.

Uma página individual associa um **título exibido** no menu ao **caminho de um arquivo** `.md` dentro de `docs/`. Uma seção associa um título a uma lista de páginas e/ou subseções. Por exemplo:

```yaml
- Começar:
    - Configuração: configuracao-yaml.md
```

O grupo **Começar** reúne páginas relacionadas e aparece como uma única aba; a página **Configuração** abre `docs/configuracao-yaml.md`. Os caminhos são relativos à pasta configurada em `docs_dir` (neste projeto, `docs/`), nunca caminhos absolutos.

Alguns detalhes importantes sobre essa sintaxe:

- Quando o título não contém caracteres especiais (como `:` ou acentos problemáticos para o YAML), ele pode ser escrito sem aspas, como em `Início: index.md`.
- Quando o título contém um `:` no meio do texto, ele precisa ser colocado entre aspas, pois o `:` tem significado especial em YAML (separa chave e valor).
- A **ordem das entradas na lista define a ordem do menu**: o cabeçalho tem cinco abas — Início, Começar, Conteúdo, Ferramentas e GitHub — e cada seção agrupa seus capítulos.
- Os rótulos curtos mantêm o cabeçalho compacto; em telas largas, `docs/stylesheets/extra.css` reduz levemente o tamanho da fonte e o espaçamento horizontal das abas.
- Cada arquivo referenciado em `nav` precisa realmente existir dentro de `docs/`. Se um caminho estiver errado ou o arquivo não existir, o `mkdocs build` falha (ou gera um aviso, dependendo do modo `--strict`).

`theme`: a aparência visual do site

```yaml
theme:
  name: material
  logo: assets/logo.png
  favicon: assets/datarain-logo.ico
  font:
    text: Roboto
    code: Roboto Mono
  palette:
    - scheme: default
      primary: custom
      accent: custom
      toggle:
        icon: material/weather-sunny
        name: "Ativar modo escuro"
    - scheme: slate
      primary: custom
      accent: custom
      toggle:
        icon: material/weather-night
        name: "Ativar modo claro"
  features:
    - content.action.edit
    - content.action.view
    - navigation.instant
    - navigation.tabs
    - navigation.sections
    - navigation.top
    - navigation.footer
```

```
    - navigation.indexes
    - content.code.copy
    - content.code.annotate
    - content.tabs.link
    - search.suggest
    - search.highlight

extra_css:
  - stylesheets/extra.css
```

A seção `theme` define **qual tema visual** o MkDocs deve usar para renderizar o site e como esse tema deve se comportar. A chave `name: material` é a mais importante dela: diz ao MkDocs para usar o tema Material for MkDocs em vez do tema padrão, mais simples, que vem embutido no MkDocs.

Por que usar o tema `material`? Alguns motivos práticos:

- Ele entrega, sem esforço extra, um layout moderno e responsivo (funciona bem tanto no computador quanto no celular).
- Já vem com suporte a modo claro/escuro, busca integrada com destaque dos termos encontrados, navegação em abas/seções e muitos outros recursos visuais.
- É o tema mais usado e mais documentado do ecossistema MkDocs, o que facilita encontrar exemplos e resolver dúvidas.

As demais chaves dentro de `theme` neste arquivo — `font`, `palette` e `logo` / `favicon` — controlam a aparência do site. As cores institucionais são definidas em `docs/stylesheets/extra.css`, carregado pela opção `extra_css` do `mkdocs.yml`. Esses detalhes estão documentados em Identidade visual.

`plugins`: funcionalidades adicionais do MkDocs

```
plugins:
  - search
  - mkdocstrings:
      handlers:
        python:
          paths: [.]              # resolve `teste` a partir da raiz do projeto
          options:
            show_source: true
            show_root_heading: true
            docstring_style: google
        typescript:
          paths: [.]
```

A seção `plugins` é uma lista dos **plugins habilitados** no site. Cada plugin adiciona uma funcionalidade ao MkDocs que não existe no comportamento padrão da ferramenta. Este projeto habilita dois plugins: `search` e `mkdocstrings`.

O plugin `search`

```
plugins:
  - search
```

`search` é o plugin de **busca do site**. Ele é responsável por indexar o conteúdo de todas as páginas Markdown durante o build e gerar o índice de busca usado pela caixa de pesquisa que aparece no topo do site (fornecida pelo tema Material). É graças a esse plugin que um leitor consegue digitar um termo na busca e ser levado diretamente à página e à seção onde esse termo aparece, sem precisar navegar manualmente pelo menu.

O plugin `mkdocstrings`

```
- mkdocstrings:
    handlers:
      python:
        paths: [.]
        options:
          show_source: true
          show_root_heading: true
          docstring_style: google
      typescript:
        paths: [.]
```

`mkdocstrings` é o plugin responsável por **gerar documentação automaticamente a partir do código-fonte e das suas docstrings/comentários de documentação**, em vez de o autor precisar escrever manualmente, em Markdown, a assinatura e a descrição de cada função ou classe. Ele funciona através de *handlers*: cada handler sabe ler e documentar código de uma linguagem específica.

Este projeto configura dois handlers:

- **`python`**: lê código Python e extrai docstrings de funções, classes e módulos. É esse handler que permite, por exemplo, escrever `teste.calcular_media` em uma página Markdown (como é feito em Exemplos de código) e ter a documentação daquela função gerada automaticamente a partir do seu docstring real.
- **`typescript`**: faz o mesmo processo, mas para código TypeScript, documentando funções/tipos a partir de comentários de documentação em arquivos `.ts`.

Dentro do handler `python`, três opções merecem destaque:

- **`paths: [.]`** — diz ao `mkdocstrings` para procurar os módulos Python **a partir da raiz do projeto** (representada por `.`). Isso é necessário porque, por padrão, o handler Python resolve módulos a partir do diretório em que o comando `mkdocs` é executado; adicionar `paths: [.]` torna essa resolução explícita e robusta, garantindo que o módulo `teste` (o arquivo `teste.py` na raiz do projeto) seja encontrado mesmo que o build seja disparado de um diretório diferente ou por alguma ferramenta que altere o diretório de trabalho atual.

- **`show_source: true`** — faz com que o código-fonte da função/classe documentada também seja exibido junto com a documentação gerada, não apenas o texto do docstring.
- **`docstring_style: google`** — informa ao `mkdocstrings` que os docstrings do projeto seguem o formato do Google Python Style Guide (com seções como `Args:`, `Returns:` e `Raises:`), para que ele saiba interpretar e formatar corretamente cada seção do docstring.

O handler `typescript` recebe apenas `paths: [.]`, pelo mesmo motivo do handler Python: garantir que os arquivos TypeScript do projeto sejam localizados a partir da raiz, independentemente de onde o comando de build é executado.

`markdown_extensions`: recursos extras de renderização do Markdown

```
markdown_extensions:
  - admonition
  - pymdownx.highlight
  - pymdownx.superfences
  - pymdownx.inlinehilite
  - toc:
      permalink: true
```

A seção `markdown_extensions` lista **extensões do Python-Markdown** (a biblioteca que o MkDocs usa para converter `.md` em HTML) que adicionam uma sintaxe ou um comportamento de renderização além do Markdown padrão. Sem essas extensões, vários recursos usados nas páginas deste tutorial (como caixas de aviso e blocos de código coloridos) simplesmente não funcionariam. Este projeto habilita cinco extensões:

- **`admonition`** — adiciona a sintaxe de *admonitions*, ou seja, caixas de aviso/nota/dica destacadas visualmente do restante do texto. Com essa extensão, um bloco como:

```
!!! note "Observação"
    Este é um texto de observação.
```

é renderizado como uma caixa colorida e destacada, chamando a atenção do leitor para informações importantes, avisos ou dicas, em vez de aparecer apenas como um parágrafo comum.

- **`pymdownx.highlight`** — fornece o mecanismo de **destaque de sintaxe (syntax highlighting)** para blocos de código, colorindo palavras-chave, strings, comentários etc. de acordo com a linguagem do bloco (Python, YAML, Bash, etc.). Essa extensão não funciona isoladamente: ela é a base usada pela extensão `pymdownx.superfences` logo abaixo para efetivamente colorir o código exibido nas páginas.
- **`pymdownx.superfences`** — melhora os blocos de código delimitados por três crases (`` ```python ``, `` ```yaml ``, `` ```bash ``, etc.), permitindo que eles sejam renderizados com o destaque de sintaxe fornecido por `pymdownx.highlight` e também possibilitando blocos de código aninhados dentro de outros elementos (como dentro de uma *admonition*). Todos os blocos de código vistos nesta própria

página de documentação (os trechos YAML acima, por exemplo) dependem dessa extensão para aparecer formatados e coloridos corretamente.

- `pymdownx.inlinehilite` — estende o destaque de sintaxe para **trechos de código em linha**, ou seja, pequenos pedaços de código citados no meio de uma frase usando crase simples, como `` `mkdocs build` ``. Sem essa extensão, apenas os blocos de código de múltiplas linhas (com três crases) receberiam destaque; com ela, até referências curtas a comandos ou nomes de função dentro de um parágrafo normal ficam estilizadas de forma consistente.
- `toc` **com** `permalink: true` — habilita e configura o **sumário automático (table of contents)** da página, gerado a partir dos títulos (`#` , `##` , `###` , etc.) usados no Markdown. A opção `permalink: true` adiciona, ao lado de cada título renderizado, um pequeno link-âncora clicável que aponta diretamente para aquela seção, permitindo compartilhar ou favoritar o link de uma seção específica da página (por exemplo, um link direto para a seção "`plugins`: funcionalidades adicionais do MkDocs" desta própria página).

Resumo

Seção	Para que serve	Página relacionada
`nav`	Define o menu de navegação, mapeando título → arquivo `.md` em `docs/`	—
`theme`	Define o tema visual (Material) e seus detalhes (fonte, paleta, logo, favicon)	Identidade visual
`plugins`	Habilita busca (`search`) e documentação automática de código (`mkdocstrings` , handlers `python` e `typescript`)	Exemplos de código
`markdown_extensions`	Habilita recursos extras de Markdown: admonitions, destaque de sintaxe e sumário com links âncora	—

2026-11-12 2026-11-11

Identidade visual

Esta página documenta a identidade visual DataRain aplicada ao site com Material for MkDocs: paletas para os modos claro e escuro, alternador de tema, cores de interface, tipografia, logo e favicon. As cores detalhadas e os estilos de componentes estão em `docs/stylesheets/extra.css`; a configuração do tema e os recursos de imagem ficam no `mkdocs.yml` e em `docs/assets/`.

Todas as opções descritas nesta página já estão aplicadas no `mkdocs.yml` real deste projeto, reproduzido abaixo:

```yaml
theme:
  name: material
  logo: assets/logo.png
  favicon: assets/datarain-logo.ico
  font:
    text: Roboto
    code: Roboto Mono
  palette:
    - scheme: default
      primary: custom
      accent: custom
      toggle:
        icon: material/weather-sunny
        name: "Ativar modo escuro"
    - scheme: slate
      primary: custom
      accent: custom
      toggle:
        icon: material/weather-night
        name: "Ativar modo claro"
  features:
    - navigation.instant
    - navigation.tabs
    - navigation.sections
    - navigation.top
    - navigation.footer
    - navigation.indexes
    - content.code.copy
    - content.code.annotate
    - content.tabs.link
    - search.suggest
    - search.highlight

extra_css:
  - stylesheets/extra.css
```

Vamos entender cada parte, uma por uma.

1. Fontes (`theme.font`)

O Material for MkDocs usa o Google Fonts para carregar as fontes do site. A chave `theme.font` aceita duas subchaves:

- `text`: a fonte usada no corpo do texto (títulos, parágrafos, menu, etc.).
- `code`: a fonte monoespaçada usada nos blocos de código.

No `mkdocs.yml` deste projeto, a configuração é:

```yaml
theme:
  font:
    text: Roboto
    code: Roboto Mono
```

Isso significa que todo o texto normal das páginas é renderizado com a fonte **Roboto**, e todo trecho de código (inline ou em bloco) usa a fonte monoespaçada **Roboto Mono**.

Passo a passo para trocar a fonte

1. Escolha um nome de fonte disponível no Google Fonts. O Material for MkDocs busca a fonte diretamente pelo nome, sem necessidade de baixar arquivos manualmente. Alguns exemplos de fontes suportadas: `Roboto`, `Open Sans`, `Lato`, `Fira Sans`, `Noto Sans`, `Inter`, `Merriweather` (serifada) e, para código, `Roboto Mono`, `Fira Code`, `Source Code Pro`, `JetBrains Mono`.
2. Edite as chaves `theme.font.text` e/ou `theme.font.code` no `mkdocs.yml`, substituindo o valor pelo nome exato da fonte escolhida (respeitando maiúsculas/minúsculas e espaços, por exemplo `"Fira Sans"`).
3. Salve o arquivo e execute `uv run mkdocs serve` para visualizar a mudança: o MkDocs recarrega a página automaticamente e já exibe a nova fonte.

> **Desativar fontes do Google**
>
> Se o site precisar funcionar sem acesso à internet para carregar fontes externas (ex.: ambientes totalmente offline), é possível definir `theme.font: false` no `mkdocs.yml`. Nesse caso, o Material usa as fontes padrão do sistema do visitante.

2. Paleta de cores e identidade DataRain

A paleta usa azul-petróleo e azul-marinho como cores institucionais, laranja para chamadas e interações, e tons claros para fundos e textos. Os valores de `primary` e `accent` são `custom`; as cores reais são definidas em `docs/stylesheets/extra.css`.

Uso	Modo claro (`default`)	Modo escuro (`slate`)
Cor principal	`#3791A6`	`#132B40`
Destaque	`#F27507`	`#F27507`
Fundo	`#FFFFFF`	`#132B40`
Texto principal	`#132B40`	`#F2F2F2`
Fundo de código	`#F2F2F2`	`#0D1E2C`
Rodapé	`#132B40`	`#0D1E2C`

O CSS também configura botões, links, seleção de texto, abas e admonitions. Para alterar a identidade visual, edite as variáveis nos seletores `[data-md-color-scheme="default"]` e `[data-md-color-scheme="slate"]` e confira os dois modos no navegador.

3. Seletor de tema claro e escuro

O `theme.palette` define o esquema claro `default` e o escuro `slate`, além dos ícones e rótulos acessíveis do alternador:

```yaml
theme:
  palette:
    - scheme: default
      primary: custom
      accent: custom
      toggle:
        icon: material/weather-sunny
        name: Ativar modo escuro
    - scheme: slate
      primary: custom
      accent: custom
      toggle:
        icon: material/weather-night
        name: Ativar modo claro
```

O botão com o ícone de sol ativa o modo escuro; o botão com a lua ativa o modo claro. O tema Material persiste a escolha no armazenamento local do navegador. Como o esquema claro é o primeiro da lista, ele é usado quando ainda não existe uma preferência salva.

4. Logo (`theme.logo`)

A chave `theme.logo` define a imagem exibida no canto superior esquerdo do site, substituindo o ícone padrão do tema. No `mkdocs.yml` deste projeto:

```yaml
theme:
  logo: assets/logo.png
```

Um ponto importante: esse caminho é **relativo à pasta `docs/`**, e não à raiz do projeto. Ou seja, `assets/logo.png` aponta para o arquivo `docs/assets/logo.png` — o MkDocs sempre resolve os caminhos de recursos do tema a partir do `docs_dir` configurado (ver Configuração do mkdocs.yml).

Passo a passo para adicionar um logo

1. Use uma imagem de logo em formato `.png` ou `.svg`, preferencialmente com fundo transparente.
2. Salve-a em `docs/assets/` e defina `theme.logo` com o caminho relativo a `docs/`.
3. Neste projeto, o logo DataRain fornecido em ICO foi convertido para `docs/assets/logo.png` para uso no cabeçalho.
4. Execute `uv run mkdocs build` (ou `uv run mkdocs serve`) e verifique a imagem no cabeçalho.

5. Favicon (`theme.favicon`)

A chave `theme.favicon` segue exatamente a mesma lógica do logo, mas define o ícone exibido na aba do navegador. No `mkdocs.yml` deste projeto:

```yaml
theme:
  favicon: assets/datarain-logo.ico
```

Assim como `theme.logo`, o caminho de `theme.favicon` também é relativo à pasta `docs/`, portanto `assets/datarain-logo.ico` corresponde ao arquivo `docs/assets/datarain-logo.ico`.

Passo a passo para adicionar um favicon

1. Crie ou obtenha uma imagem pequena para a aba do navegador (formatos aceitos incluem `.png`, `.ico`, `.svg` e `.gif`).
2. Salve a imagem em `docs/assets/`.
3. No `mkdocs.yml`, defina `theme.favicon` apontando para esse arquivo a partir de `docs/`.
4. Execute `uv run mkdocs build` (ou `uv run mkdocs serve`) e confirme, na aba do navegador, que o ícone foi atualizado.

6. Convenção de pasta: `docs/assets/`

Tanto o logo quanto o favicon (e, de forma geral, qualquer imagem ou recurso estático usado nas páginas do tutorial) devem ficar guardados na subpasta `docs/assets/`. Neste projeto, `assets/logo.png` fornece o logo do cabeçalho e `assets/datarain-logo.ico` o favicon.

7. Folha de estilos personalizada

O `mkdocs.yml` carrega `docs/stylesheets/extra.css` por meio da opção `extra_css`. Esse arquivo centraliza as cores para os esquemas `default` e `slate`, além dos estilos dos botões, links, código, abas, seleção de texto e caixas informativas.

Os estilos das caixas seguem as categorias de admonition: note e success usam azul-petróleo; tip e important usam laranja; warning usa laranja-avermelhado. Os títulos das caixas recebem peso de fonte maior para facilitar a leitura.

2026-11-11 2026-11-11

Exemplos de código

Esta página ensina a documentar código Python e TypeScript no MkDocs. Em ambos os casos, o fluxo é: escrever a documentação no próprio código, configurar o handler correspondente e incluir uma diretiva `mkdocstrings` na página Markdown. O resultado aparece como documentação HTML gerada durante o build.

Python com docstrings no formato Google

1. Escreva a função e a docstring em `teste.py`

O formato Google organiza a explicação em seções como `Args`, `Returns` e `Raises`. A descrição do argumento fica indentada sob `Args`; inclua o tipo na anotação Python e descreva o significado no texto:

```python
def calcular_media(valores: list[float]) -> float:
    """
    Calcula a média de uma lista de valores.

    Args:
        valores: Lista de valores numéricos.

    Returns:
        Média aritmética dos valores.

    Raises:
        ValueError: Se a lista estiver vazia.
    """
    if not valores:
        raise ValueError("A lista não pode estar vazia.")

    return sum(valores) / len(valores)
```

2. Configure o handler Python no `mkdocs.yml`

Em `plugins.mkdocstrings.handlers.python`, defina o caminho onde o módulo pode ser encontrado e informe que o parser deve interpretar docstrings Google:

```yaml
plugins:
  - mkdocstrings:
      handlers:
        python:
          paths: [.]
          options:
            show_source: true
            show_root_heading: true
            docstring_style: google
```

```
        typescript: {}
```

`paths: [.]` permite localizar `teste.py` na raiz do projeto. `docstring_style: google` faz o handler reconhecer as seções no formato Google; não altera a escrita do docstring.

3. Inclua o objeto na página Markdown

Escreva a diretiva no arquivo Markdown sem cercá-la por crases:

```
::: teste.calcular_media
```

Saída renderizada pelo MkDocs: a diretiva abaixo é processada de verdade no build. O cabeçalho, a assinatura e as seções da documentação vêm de `teste.py`:

`teste.calcular_media(valores)`

Calcula a média de uma lista de valores.

Parameters:

Name	Type	Description	Default
`valores`	`list[float]`	Lista de valores numéricos.	*required*

Returns:

Type	Description
`float`	Média aritmética dos valores.

Raises:

Type	Description
`ValueError`	Se a lista estiver vazia.

Source code in `teste.py`

```python
def calcular_media(valores: list[float]) -> float:
    """
    Calcula a média de uma lista de valores.

    Args:
        valores: Lista de valores numéricos.

    Returns:
        Média aritmética dos valores.

    Raises:
        ValueError: Se a lista estiver vazia.
    """
    if not valores:
        raise ValueError("A lista não pode estar vazia.")

    return sum(valores) / len(valores)
```

Bloco de código estático

O trecho de Python mostrado acima é um **bloco de código estático**: um texto fixo, delimitado por crases (```` ``` ````) com a linguagem `python` indicada logo após as crases de abertura. Essa é a forma mais simples de exibir um exemplo de código em uma página MkDocs — você simplesmente copia e cola o conteúdo do arquivo-fonte (neste caso, `teste.py`) diretamente dentro do Markdown da página.

Não há nenhuma "mágica" por trás desse bloco: o MkDocs não lê `teste.py` para gerá-lo, não executa nada e não sabe que essa função realmente existe no projeto. O texto que aparece é exatamente o texto que foi digitado (ou colado) no arquivo `exemplos-codigo.md`. A única coisa que o MkDocs faz é aplicar o destaque de sintaxe (*syntax highlighting*) do Python ao conteúdo, graças às extensões `pymdownx.highlight` e `pymdownx.superfences` configuradas no `mkdocs.yml` — mas o texto em si é escrito manualmente por quem edita a página.

Como a documentação é gerada

Agora, em vez de colar o código novamente, vamos deixar o plugin `mkdocstrings` gerar a documentação da mesma função automaticamente a partir do código-fonte. A sintaxe usada para isso é a seguinte:

```
::: teste.calcular_media
```

Essa linha é chamada de **diretiva** do `mkdocstrings`. Ela não é um bloco de código: é uma instrução especial, reconhecida pelo plugin durante a geração do site, escrita diretamente no corpo da página Markdown (sem as crases de bloco de código). A diretiva inserida acima é processada pelo `mkdocstrings` e substituída pelo resultado gerado.

Passo a passo do que acontece quando o comando `mkdocs build` (ou `mkdocs serve`) processa essa diretiva:

1. **Leitura da página**: o MkDocs lê o arquivo `exemplos-codigo.md` e encontra a linha `teste.calcular_media`, que é reconhecida pelo plugin `mkdocstrings` (configurado na seção `plugins` do `mkdocs.yml`) como uma diretiva válida, e não como texto comum.
2. **Interpretação do identificador**: o `mkdocstrings` interpreta o texto após `:::` — `teste.calcular_media` — como um caminho no formato `módulo.função`. Isso indica ao handler Python do plugin que ele deve procurar um módulo chamado `teste` e, dentro dele, um objeto chamado `calcular_media`.
3. **Importação do módulo**: o handler Python importa o módulo `teste` (ou seja, executa `import teste` internamente), usando a lista `paths: [.]` configurada em `plugins.mkdocstrings.handlers.python` no `mkdocs.yml` para saber que deve procurar esse módulo a partir da raiz do projeto, onde o arquivo `teste.py` realmente está.
4. **Inspeção da função**: com o módulo importado, o `mkdocstrings` inspeciona o objeto `calcular_media` em tempo real — obtém sua assinatura (nome do parâmetro `valores`, anotação de tipo `list[float]`, tipo de retorno `float`) e lê a docstring completa da função, exatamente como ela está escrita em `teste.py` naquele momento.
5. **Geração do HTML**: a partir dessas informações (assinatura + docstring), o plugin gera automaticamente o HTML da documentação da função — incluindo um cabeçalho com o nome e a assinatura, e seções formatadas para `Args`, `Returns` e `Raises`, de acordo com o estilo de docstring configurado (`docstring_style: google`) — e insere esse HTML no lugar onde a diretiva estava escrita.

Em resumo: nada do conteúdo visível acima (assinatura, parâmetros, descrição, seções `Args` / `Returns` / `Raises`) foi digitado manualmente nesta página — tudo foi extraído e formatado automaticamente a partir do código real de `teste.py` no momento do build.

Bloco estático vs. mkdocstrings — qual usar?

A diferença fundamental entre as duas formas mostradas nesta página está em **de onde vem o conteúdo** e **quando ele é atualizado**:

- **Bloco de código estático**: o conteúdo é um texto fixo, digitado ou colado manualmente dentro do arquivo `exemplos-codigo.md`. Ele representa uma "fotografia" do código no momento em que foi copiado. Se, no futuro, alguém alterar a função `calcular_media` em `teste.py` — por exemplo, mudando um parâmetro, a lógica interna ou a docstring — o bloco estático **não muda automaticamente**. Ele continuará mostrando a versão antiga do código até que alguém edite manualmente a página e cole o trecho atualizado. Ou seja, esse bloco pode ficar **desatualizado (dessincronizado)** em relação ao código real, e a responsabilidade de mantê-lo sincronizado é inteiramente de quem edita a documentação.

- **Diretiva do mkdocstrings** (`teste.calcular_media`): o conteúdo **não é armazenado** na página — apenas a referência `teste.calcular_media` é armazenada. Todo o restante (assinatura, parâmetros, docstring) é **gerado dinamicamente a cada execução de `mkdocs build` ou `mkdocs serve`**, lendo o código-fonte de `teste.py` naquele exato momento. Isso significa que, se a função `calcular_media` for alterada, a próxima vez que o site for reconstruído, a documentação exibida mudará automaticamente, sem que ninguém precise editar `exemplos-codigo.md`. Por isso essa forma **nunca fica desatualizada** em relação ao código real.

Recomendação prática:

- Use um **bloco de código estático** quando quiser mostrar um trecho de código ilustrativo, hipotético, simplificado ou que não corresponde a uma função real do projeto — por exemplo, um exemplo didático genérico que não precisa (e não deve) ser acoplado a um arquivo específico do código-fonte.
- Use a **diretiva do mkdocstrings** sempre que quiser documentar uma função, classe ou módulo que realmente existe no projeto. Isso evita duplicação de esforço (não é preciso copiar e colar código na documentação) e garante que a documentação do projeto esteja sempre fiel ao comportamento real do código, mesmo depois de várias alterações futuras em `teste.py`.

Exemplo em TypeScript

As duas seções acima compararam um bloco de código estático com a documentação gerada automaticamente pelo `mkdocstrings`, usando a função Python `calcular_media`. A mesma comparação é repetida agora para um exemplo equivalente em TypeScript, a função `calcularMedia`, definida em `src/calcular-media/src/index.ts`. A diferença é que, para TypeScript, o `mkdocstrings` usa o handler `typescript`, que por sua vez depende do **TypeDoc** para extrair a documentação do código-fonte.

Bloco de código estático

Assim como no exemplo em Python, o trecho abaixo é uma cópia fixa do conteúdo de `src/calcular-media/src/index.ts`, colada diretamente nesta página:

```typescript
/**
 * Calcula a média de uma lista de valores.
 *
 * @param valores - Lista de valores numéricos.
 * @returns Média aritmética dos valores.
 * @throws {Error} Se a lista estiver vazia.
 */
export function calcularMedia(valores: number[]): number {
  if (valores.length === 0) {
    throw new Error("A lista não pode estar vazia.");
  }

  return valores.reduce((soma, valor) => soma + valor, 0) / valores.length;
}
```

Pré-requisito: instalar o TypeDoc

Diferente do handler Python do `mkdocstrings`, o handler `typescript` depende do **TypeDoc**, uma ferramenta Node.js que extrai a documentação dos arquivos TypeScript. Neste projeto, `src/calcular-media/typedoc.json` aponta diretamente para `src/index.ts`; não é necessário gerar um arquivo `.d.ts` antes do build.

Primeiro, instale a versão LTS do Node.js. O instalador do Node.js também instala o **npm**, que será usado para instalar o TypeDoc. Depois da instalação, feche e abra novamente o terminal e confirme que os dois comandos estão disponíveis:

```
node --version
npm --version
```

Se o Node.js já estiver instalado, mas os comandos não forem reconhecidos no PowerShell, inclua os diretórios padrão nesta janela do terminal:

```
$env:Path = "C:\Program Files\nodejs;$env:APPDATA\npm;$env:Path"
node --version
npm --version
```

Para manter a correção em novos terminais, adicione `C:\Program Files\nodejs` e `%APPDATA%\npm` à variável `Path` do usuário em **Variáveis de Ambiente** do Windows e abra um novo terminal. Se o Node.js estiver instalado em outro local, use o diretório real da instalação.

Em seguida, instale o TypeDoc globalmente:

```
npm install --global typedoc
```

O handler do `mkdocstrings` executa o comando `typedoc` diretamente. Confirme que ele está acessível no terminal antes de gerar a documentação:

```
typedoc --version
```

Se `typedoc` não for reconhecido, abra um novo terminal para atualizar o `PATH` ou confira se `%APPDATA%\npm` está incluído nele.

Configure o handler e o pacote TypeScript

O handler TypeScript é habilitado em `mkdocs.yml` com `typescript: {}`. Para cada pacote documentado, o TypeDoc também precisa localizar o projeto e seus arquivos-fonte. Neste exemplo, o pacote está em `src/calcular-media/`:

```
plugins:
  - mkdocstrings:
```

```
      handlers:
        typescript: {}
```

O `package.json` identifica o pacote pelo campo `name`. Esse mesmo nome será usado na diretiva:

```
{
  "name": "calcular-media",
  "private": true
}
```

O arquivo `src/calcular-media/typedoc.json` indica ao TypeDoc o ponto de entrada `.ts` e o `tsconfig.json` do pacote:

```
{
  "extends": ["../../typedoc.base.json"],
  "entryPointStrategy": "expand",
  "entryPoints": ["src/index.ts"],
  "tsconfig": "tsconfig.json"
}
```

Documente as funções no próprio TypeScript com comentários JSDoc (`/** ... */`) e tags como `@param`, `@returns` e `@throws`, como no exemplo de `calcularMedia` acima. O TypeDoc usa esse arquivo de configuração para extrair a documentação.

O identificador `calcular-media` não é um caminho de arquivo nem o nome de uma função TypeScript: é o valor de `name` em `package.json`. O handler padrão do projeto é o Python, por isso informe `handler: typescript` na diretiva.

Na página `.md`, escreva a diretiva sem cercá-la por crases:

```
::: calcular-media
    handler: typescript
```

Saída renderizada pelo MkDocs: a diretiva abaixo é processada no build. O handler localiza o pacote `calcular-media`, executa o TypeDoc e apresenta a documentação extraída do `src/index.ts`:

Functions

`calcularMedia`

Bloco estático vs. mkdocstrings (TypeScript)

A mesma diferença explicada para o exemplo em Python se aplica aqui: o bloco de código estático acima é uma cópia fixa de `index.ts`, que pode ficar desatualizada se a função `calcularMedia` for alterada e ninguém lembrar de atualizar esta página manualmente. Já a diretiva `calcular-media` gera sua documentação dinamicamente a cada build, usando o TypeDoc configurado para ler `src/index.ts`.

Há, porém, uma diferença prática importante em relação ao exemplo Python: a diretiva `teste.calcular_media` funciona a partir do arquivo `.py`, enquanto a diretiva `calcular-media` precisa do Node.js e do TypeDoc disponíveis no ambiente de build. Não é necessário executar `tsc` separadamente: o TypeDoc lê o arquivo-fonte `.ts` configurado no pacote.

> ⚠ **Toolchain do TypeDoc não disponível neste ambiente**
>
> Se Node.js, npm e TypeDoc não estiverem instalados — ou se o comando `typedoc` não estiver no `PATH` —, o handler `typescript` não consegue gerar a documentação e o build falha ao processar esta diretiva. No Windows, isso pode aparecer como `FileNotFoundError: [WinError 2]` ao tentar iniciar o processo `typedoc`.
>
> No GitHub Actions, o workflow instala Node.js e TypeDoc antes de executar `uv run mkdocs build --strict`. Em uma máquina local, siga os passos de instalação acima e depois rode `uv run mkdocs build --strict`.

Diagnóstico: incompatibilidade do decoder do TypeDoc

Além de instalar o Node.js e o TypeDoc, o build precisa de versões compatíveis do handler TypeScript e do decoder que lê o JSON produzido pelo TypeDoc. Uma incompatibilidade entre `griffe-typedoc` 0.1.1 e a saída atual do TypeDoc fazia o build terminar com um erro semelhante a:

```
Project.__init__() got an unexpected keyword argument 'schema_version'
```

Nesse caso, o TypeDoc já encontrou o ponto de entrada e produziu sua saída; a falha acontece depois, durante a desserialização do JSON pelo `griffe-typedoc`. Portanto, alterar o `tsconfig.json`, a diretiva desta página ou o ponto de entrada não corrige esse erro específico.

A correção foi implementada no commit upstream d0afb317, que adiciona suporte ao campo `schema_version` no modelo `Project`. Até que essa correção esteja disponível em uma versão publicada no PyPI, este projeto fixa `griffe-typedoc` nesse commit em `pyproject.toml` e `uv.lock`. Mantenha os dois arquivos sincronizados ao atualizar dependências:

```
uv lock --check
uv sync
uv run mkdocs build --strict
```

Após essa correção, o build e o deploy pelo GitHub Actions foram executados com sucesso. Para acompanhar uma execução futura, consulte **Actions → Build e publica a documentação** ou use os comandos `gh run list` e `gh run watch` descritos em GitHub CLI.

2026-11-13 2026-11-11

Exemplos do notebook

O arquivo `mkdocks.ipynb`, na raiz do projeto, funciona como um **roteiro de anotações**: um notebook Jupyter com os títulos das etapas que foram seguidas para colocar este site MkDocs no ar, na ordem em que aconteceram. Ele não contém explicações completas nem o detalhe de cada comando, apenas lembretes curtos como "Instalando Mkdocks", "fazendo o build" ou "Executando o mkdocks para visualizar no browser".

Esta página pega esse roteiro e o transforma em um passo a passo completo, em prosa e com os comandos reais de terminal, para que você consiga reproduzir o mesmo fluxo do início ao fim. Em vez de copiar literalmente as anotações do notebook, cada etapa abaixo é explicada com suas próprias palavras e aponta para a página do tutorial onde o assunto é tratado em profundidade.

1. Instalação dos pacotes

A primeira anotação do notebook é "Instalando Mkdocks". Antes de qualquer configuração ou conteúdo, é preciso ter os pacotes do MkDocs disponíveis no ambiente do projeto. Isso inclui o próprio MkDocs, o tema Material e os handlers do `mkdocstrings` usados para gerar documentação a partir do código-fonte.

```
uv add mkdocs mkdocs-material "mkdocstrings[python]" mkdocstrings-typescript
```

Esse é o mesmo comando detalhado, pacote por pacote, na página Instalação. Se você ainda não leu essa página, vale a pena revisá-la antes de seguir, pois ela explica por que o `mkdocstrings[python]` precisa de aspas e para que serve o handler de TypeScript.

2. Inicialização do projeto

A segunda anotação, "Inicializando um novo projeto", corresponde ao comando que cria a estrutura mínima de um site MkDocs:

```
uv run mkdocs new .
```

Esse comando pede ao MkDocs para inicializar um projeto na pasta atual (o `.` indica "aqui mesmo"). Quando executado em uma pasta vazia, ele cria dois elementos:

- um arquivo `mkdocs.yml` mínimo, apenas com um `site_name` padrão;
- uma pasta `docs/` contendo um único arquivo `index.md` de exemplo, com um texto de boas-vindas genérico.

Neste projeto específico, porém, esse ponto de partida automático não é mais o que você vai encontrar ao rodar o comando: tanto o `mkdocs.yml` quanto o conteúdo de `docs/` já foram criados e customizados manualmente nas etapas anteriores deste tutorial (navegação com as seis páginas, tema Material configurado, plugins de `mkdocstrings`, etc.). Ou seja, `uv run mkdocs new .` é o comando que *daria o pontapé inicial* em um projeto novo do zero; aqui ele serve apenas como referência histórica do que originou a estrutura do projeto, e não precisa ser executado novamente.

3. Build do site

A anotação "fazendo o build" se refere à geração do site estático a partir da configuração e do conteúdo já existentes:

```
uv run mkdocs build
```

Esse comando lê o `mkdocs.yml` da raiz do projeto, processa cada arquivo Markdown listado em `nav` dentro da pasta `docs/` (aplicando o tema, os plugins e as extensões configuradas) e grava o resultado como um conjunto de arquivos HTML, CSS e JavaScript prontos na pasta `site/`. É esse conteúdo de `site/` que, de fato, constitui o site de documentação publicável: nada nessa pasta precisa ser editado à mão, pois ela é recriada do zero a cada execução do `build`.

4. Visualização local

A anotação "executando o mkdocs para visualizar a documentação html" descreve o comando usado para conferir o resultado enquanto o conteúdo ainda está sendo escrito:

```
uv run mkdocs serve
```

Diferente do `build`, o `serve` não apenas gera os arquivos: ele também sobe um servidor web local, normalmente disponível em `http://127.0.0.1:8000`, e fica em execução observando os arquivos do projeto. Qualquer alteração salva em `mkdocs.yml` ou em algum arquivo dentro de `docs/` é detectada automaticamente, o site é reconstruído em segundo plano e a página aberta no navegador é recarregada sozinha. Esse é o comando mais usado durante a escrita da documentação, já que permite ver o resultado de cada mudança quase em tempo real, sem precisar rodar `build` e abrir o arquivo manualmente a cada edição.

5. Configuração do `mkdocs.yml`

A anotação "Configurando o Yaml" corresponde ao trabalho de definir, dentro do `mkdocs.yml`, tudo o que controla o comportamento do site: a lista de páginas em `nav`, o tema em `theme`, os plugins em `plugins` (incluindo os handlers do `mkdocstrings` para Python e TypeScript) e as extensões de Markdown em `markdown_extensions`.

Esse trabalho de configuração é explicado seção por seção, com os trechos de YAML reais usados neste projeto, na página Configuração do mkdocs.yml.

6. Personalizações visuais

A anotação "Personalizações" corresponde aos ajustes feitos sobre o tema Material depois que a configuração básica já estava funcionando: escolha de fontes (`theme.font`), paleta de cores com variantes clara e escura (`theme.palette`), logo (`theme.logo`) e favicon (`theme.favicon`).

Esses ajustes, incluindo onde guardar os arquivos de imagem usados por eles, são detalhados na página Identidade visual.

7. Visualizando no navegador

A última anotação do notebook, "Executando o mkdocks para visualizar no browser", fecha o roteiro: depois de configurar e personalizar o site, falta apenas abri-lo em um navegador para conferir o resultado final. Existem duas formas de fazer isso:

Opção recomendada — usando `mkdocs serve` novamente:

```
uv run mkdocs serve
```

Com o servidor local em execução, basta abrir `http://127.0.0.1:8000` no navegador. Essa é a forma mais simples e também a mais indicada quando você ainda pode estar fazendo ajustes no conteúdo, já que o recarregamento automático mostra cada mudança na hora.

Alternativa — servindo a pasta `site/` já gerada:

Se o site já foi gerado com `uv run mkdocs build` e você quer apenas visualizar o resultado final (sem um servidor de desenvolvimento), é possível servir a pasta `site/` com um servidor HTTP simples, como o módulo embutido do Python:

```
python -m http.server --directory site
```

Esse comando inicia um servidor básico (por padrão em `http://localhost:8000`) apontando diretamente para os arquivos estáticos já construídos.

Em ambos os casos, evite abrir o arquivo `site/index.html` diretamente no navegador através do protocolo `file://` (por exemplo, clicando duas vezes no arquivo). O tema Material depende de requisições feitas por um servidor HTTP para carregar corretamente alguns recursos, em especial o índice de busca; ao abrir via `file://` , essas requisições costumam falhar silenciosamente e a busca do site simplesmente não funciona, mesmo que o restante da página pareça normal.

2026-11-11 2026-11-11

Imagens e recursos visuais

Além de configurar o logo e o favicon do tema em Identidade visual, também é possível inserir imagens diretamente no conteúdo das páginas Markdown. O Material for MkDocs e seus plugins oferecem recursos para ampliar, dimensionar e legendar essas imagens.

Esta página cobre três formas de enriquecer imagens no Documentation_Site:

1. **Lightbox** com o plugin `mkdocs-glightbox`: zoom em tela cheia ao clicar em qualquer imagem.
2. **Legendas e alinhamento** com as extensões `attr_list` e `pymdownx.blocks.caption`.
3. **Otimização automática** de imagens com o recurso nativo `optimize` do tema Material.

1. Lightbox com `mkdocs-glightbox`

Instalação

O plugin de lightbox não vem embutido no MkDocs nem no Material; é um pacote Python separado, instalado com:

```
uv add mkdocs-glightbox
```

Configuração

Depois de instalado, basta registrá-lo na seção `plugins` do `mkdocs.yml`:

```yaml
plugins:
  - glightbox
```

Este projeto já tem exatamente essa entrada configurada (ver Configuração do mkdocs.yml).

O que o plugin faz

O `glightbox` atua no momento do `mkdocs build`: ele percorre o HTML gerado de **cada página** e envolve **toda imagem inserida via sintaxe Markdown** (`![alt](caminho)`) em um comportamento de lightbox — ou seja, ao clicar na imagem, ela se expande em uma camada de zoom sobre o restante da página, com suporte a navegação entre as imagens da mesma página e fechamento ao clicar fora ou pressionar `Esc`.

O ponto mais importante é que **isso acontece automaticamente para qualquer imagem Markdown**, sem exigir nenhuma marcação especial por imagem. Não é necessário envolver a imagem em um link, adicionar

uma classe CSS ou qualquer atributo extra: basta que a imagem exista como `![alt](caminho)` em qualquer página do site, e o `glightbox` já cuida do resto durante o build.

Exemplo: ampliar uma imagem

A imagem abaixo usa o arquivo DataRain `dataRain_Color-removebg-preview.png`, salvo em `docs/assets/`. Como ela está escrita com a sintaxe Markdown comum, o `glightbox` permite clicar nela para abrir a visualização ampliada:



Logo DataRain exibido como imagem de conteúdo. Clique na imagem para ampliá-la.

Como o lightbox funciona nas demais imagens

O mesmo comportamento se aplica automaticamente às demais imagens Markdown do site. Não é necessário envolver cada imagem em um link nem adicionar configurações especiais do `glightbox`.

2. Legendas e alinhamento com `attr_list` e `pymdownx.blocks.caption`

Duas extensões de Markdown já habilitadas neste projeto (ver `markdown_extensions` em Configuração do mkdocs.yml) permitem controlar o tamanho, classes de posicionamento e a legenda de uma imagem diretamente no Markdown.

`attr_list`: dimensionar e adicionar classes a uma imagem

A extensão `attr_list` permite anexar atributos HTML — como `width`, `height` ou classes CSS — a praticamente qualquer elemento Markdown, escrevendo-os entre chaves `{: ... }` imediatamente após o elemento. Aplicada a uma imagem, a sintaxe é:

```
![Logotipo DataRain](assets/dataRain_Color-removebg-preview.png){: width="300"
align="right" }
```

Aqui:

- `![Logotipo DataRain](assets/dataRain_Color-removebg-preview.png)` é a sintaxe Markdown comum de imagem: o texto entre `[` e `]` é o texto alternativo (`alt`), e o texto entre `(` e `)` é o caminho do arquivo.
- `{: width="300" align="right" }` é o bloco de atributos do `attr_list`. Neste exemplo, `width="300"` define a largura e `align="right"` usa o alinhamento legado de imagens do HTML.

O atributo HTML `align` é mantido por compatibilidade, mas é legado. Para novas páginas, prefira as classes CSS `.image-left`, `.image-right` e `.image-center` documentadas a seguir; elas permitem controlar margens e comportamento responsivo.

`pymdownx.blocks.caption`: adicionar uma legenda

A extensão `pymdownx.blocks.caption` adiciona uma legenda visível abaixo de uma imagem (ou de outro bloco), usando a sintaxe de blocos `///` do pymdown-extensions:

```
![Logotipo DataRain](assets/dataRain_Color-removebg-preview.png)
/// caption
Figura 1 – Logo DataRain inserido no conteúdo da página.
///
```

A leitura dessa sintaxe é:

- A primeira linha é a imagem comum, exatamente como antes.
- O bloco que começa em `/// caption` e termina no `///` de fechamento é um **bloco pymdownx** do tipo `caption`. Na linha de abertura, o texto logo depois de `///` (`caption`) identifica o tipo de bloco; todo o texto entre essa linha e o `///` final é o conteúdo da legenda.
- No HTML gerado, o pymdownx.blocks.caption associa automaticamente essa legenda à imagem (ou bloco) imediatamente anterior, renderizando-a como um elemento `<figcaption>` — o equivalente semântico de uma legenda de figura em HTML.

As duas extensões podem ser combinadas na mesma imagem: primeiro o `attr_list` para controlar tamanho e classe, depois o bloco `/// caption ... ///` para a legenda.

3. Grades e posicionamento de imagens

O projeto carrega `docs/stylesheets/extra.css` por meio de `extra_css` em `mkdocs.yml`. As classes desta seção são incluídas nesse arquivo e podem ser reutilizadas em outras páginas. Coloque os arquivos de imagem dentro de `docs/` e use caminhos relativos, como `assets/logo.png`.

Criar uma grade responsiva

Uma grade organiza várias imagens lado a lado e ajusta o número de colunas conforme a largura disponível. O HTML abaixo usa `markdown="1"` para que as imagens e as legendas continuem sendo escritas em Markdown; a extensão `md_in_html` já está habilitada neste projeto.

Código para copiar para uma página Markdown:

```
<div class="image-grid" markdown="1">

![Logo compacto](assets/logo.png)
/// caption
Versão compacta do logotipo.
///

![Logo horizontal](assets/dataRain_Color-removebg-preview.png)
/// caption
Versão horizontal do logotipo.
///

</div>
```

Resultado na página:



Versão compacta do logotipo.



Versão horizontal do logotipo.

O CSS correspondente, já adicionado a `docs/stylesheets/extra.css`, usa `auto-fit` para criar quantas colunas couberem. O valor `14rem` define a largura mínima aproximada de cada cartão; aumente-o para exibir menos colunas ou reduza-o para exibir mais:

```
.md-typeset .image-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 14rem), 1fr));
  gap: 1rem;
  align-items: start;
  margin: 1.5rem 0;
}

.md-typeset .image-grid figure {
  min-width: 0;
```

```
  margin: 0;
}

.md-typeset .image-grid img {
  display: block;
  width: 100%;
  height: auto;
}

.md-typeset .image-grid figcaption {
  margin-top: 0.4rem;
  text-align: center;
}
```

Se precisar de exatamente três colunas em telas grandes, substitua a declaração de `grid-template-columns` por `repeat(3, minmax(0, 1fr))` e acrescente regras de media query para reduzir a quantidade em telas menores:

```
.md-typeset .image-grid {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}

@media screen and (max-width: 50em) {
  .md-typeset .image-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}

@media screen and (max-width: 35em) {
  .md-typeset .image-grid {
    grid-template-columns: 1fr;
  }
}
```

Centralizar ou alinhar a imagem à esquerda ou à direita

Para centralizar uma imagem, aplique a classe `.image-center` e defina uma largura. A imagem permanece dentro da largura da página:

```
![Logotipo centralizado](assets/logo.png){: width="300" .image-center}
```

Prévia:



Para deixar o texto fluir ao lado da imagem, use `.image-left` ou `.image-right`. Depois do trecho que deve contornar a imagem, insira o elemento de limpeza para que o conteúdo seguinte volte a ocupar a largura inteira:

Imagem à esquerda:

```
![Logotipo à esquerda](assets/logo.png){: width="220" .image-left}
```

Imagem à direita, seguida de uma limpeza para o próximo conteúdo:

```
![Logotipo à direita](assets/logo.png){: width="220" .image-right}

Este parágrafo flui ao lado da imagem. O alinhamento à direita funciona bem para
ilustrações pequenas acompanhadas por um texto curto. Para uma galeria de várias imagens,
prefira a grade responsiva em vez de floats.

<div class="image-clear"></div>

Este parágrafo começa abaixo da imagem, sem continuar ao lado dela.
```

Prévia do alinhamento à direita:

Este texto ocupa o espaço ao lado da imagem. O exemplo é curto para deixar visível o efeito do alinhamento.



As classes de posicionamento são implementadas assim:

```
.md-typeset img.image-center {
  display: block;
  float: none;
  max-width: 100%;
  height: auto;
  margin: 1rem auto;
}

.md-typeset img.image-left,
.md-typeset img.image-right {
  max-width: 45%;
  height: auto;
}

.md-typeset img.image-left {
  float: left;
  margin: 0 1rem 0.75rem 0;
}

.md-typeset img.image-right {
  float: right;
  margin: 0 0 0.75rem 1rem;
```

```
}

.md-typeset .image-clear {
  clear: both;
}

@media screen and (max-width: 40em) {
  .md-typeset img.image-left,
  .md-typeset img.image-right {
    display: block;
    float: none;
    max-width: 100%;
    margin: 1rem auto;
  }
}
```

As mesmas regras já estão em `extra.css`; não é necessário copiá-las novamente nesse projeto. Se levar os exemplos para outro projeto MkDocs, copie o CSS para a folha de estilos do tema e confirme que ela está referenciada em `extra_css`.

4. Otimização automática de imagens com `optimize`

Além do lightbox e das legendas, o Material for MkDocs oferece um plugin nativo chamado `optimize`, que **não precisa ser instalado separadamente** (já vem embutido no pacote `mkdocs-material`, já declarado no `pyproject.toml` deste projeto — ver Instalação). Ele é habilitado apenas adicionando uma entrada em `plugins`:

```
plugins:
  - optimize
```

Quando habilitado, o `optimize` processa automaticamente, durante o `mkdocs build`, todas as imagens referenciadas dentro de `docs/`: ele comprime o arquivo original e pode gerar variantes otimizadas (por exemplo, em formato `.webp`), reduzindo o tamanho final dos arquivos de imagem servidos pelo Documentation_Site — sem que o autor do conteúdo precise rodar nenhuma ferramenta externa de compressão de imagens antes de salvar o arquivo em `docs/assets/`.

Este projeto não habilita `optimize` no `mkdocs.yml` atualmente. O recurso é opcional e deve ser ativado quando a otimização automática das imagens for necessária; para isso, acrescente `optimize` à lista de `plugins` e valide o resultado do build antes de publicar.

2026-11-13 2026-11-11

Diagramas e arquitetura

1. Por que documentar com diagramas

Uma descrição textual de um fluxo de CI/CD, da arquitetura de um sistema ou da sequência de chamadas entre serviços costuma exigir várias frases para transmitir algo que um diagrama comunica em um único olhar. Diagramas são particularmente úteis para:

- **Fluxos de processo**: a ordem em que etapas acontecem (por exemplo, o caminho entre um `git push` e a publicação final do site).
- **Arquitetura de sistemas**: como componentes se conectam e dependem uns dos outros.
- **Sequências de interação**: a troca de mensagens entre partes (cliente, servidor, banco de dados) ao longo do tempo.

O problema histórico de manter diagramas em uma documentação é que ferramentas de diagramação tradicionais (desenhadas em um editor gráfico separado, exportadas como imagem) tendem a se desatualizar: ninguém reabre o editor original só para mover uma caixa quando o fluxo muda, e a imagem exportada fica desconectada do texto que a cerca.

O Mermaid resolve esse problema definindo diagramas como **texto simples**, em uma sintaxe parecida com Markdown, que é interpretada e desenhada no navegador. Como o diagrama é texto, ele vive dentro do próprio arquivo `.md`, junto do conteúdo que ele ilustra, pode ser revisado em um `diff` de Pull Request como qualquer outra mudança de texto, e não exige nenhuma ferramenta externa de edição gráfica.

O Material for MkDocs já inclui a biblioteca JavaScript do Mermaid como parte do próprio tema — não é necessário instalar nenhum pacote Python adicional (como aconteceria com um plugin dedicado); a única coisa necessária é uma pequena configuração do `pymdownx.superfences`, descrita a seguir.

2. A configuração `custom_fences`

O trecho relevante do `mkdocs.yml` deste projeto é:

```yaml
markdown_extensions:
  - pymdownx.superfences:
      custom_fences:
        - name: mermaid
          class: mermaid
          format: !!python/name:pymdownx.superfences.fence_code_format
```

Por padrão, a extensão `pymdownx.superfences` trata todo bloco de código cercado por crases (```) da mesma forma: aplica destaque de sintaxe (*syntax highlighting*) de acordo com a linguagem declarada após

as crases de abertura, como visto nos blocos `python`, `yaml` ou `dockerfile` usados em outras páginas deste tutorial.

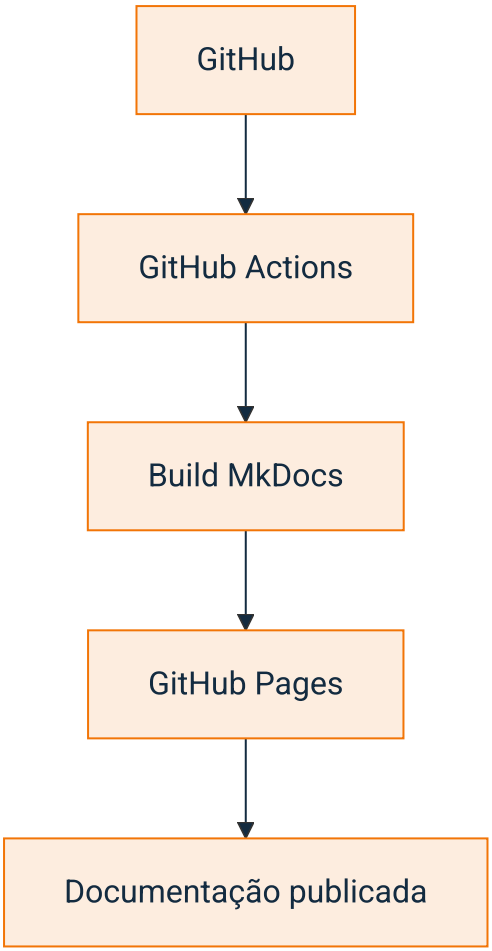
A opção `custom_fences` ensina o `pymdownx.superfences` a abrir uma exceção para um formato específico. Cada item de `custom_fences` tem três chaves:

- `name: mermaid` — o nome da cerca que recebe tratamento especial; é o texto que aparece depois das três crases de abertura, ou seja, ```` ```mermaid ````.
- `class: mermaid` — a classe CSS aplicada ao elemento HTML gerado. É essa classe (`mermaid`) que a biblioteca JavaScript do Mermaid, incluída pelo tema Material, procura automaticamente na página para saber quais elementos ela deve processar e renderizar.
- `format: !!python/name:pymdownx.superfences.fence_code_format` — em vez do caminho normal (destacar a sintaxe como código), usa a função `fence_code_format` fornecida pelo próprio `pymdownx.superfences` para envolver o conteúdo bruto do bloco em uma `<div class="mermaid">...</div>`, sem aplicar nenhum destaque de sintaxe a ele.

Juntando as três peças: quando o `mkdocs build` encontra um bloco ```` ```mermaid ````, ele não tenta colorir a sintaxe como se fosse uma linguagem de programação — ele gera uma `<div class="mermaid">` contendo o texto exato do diagrama. Essa `<div>` chega ao navegador como HTML comum; a biblioteca JavaScript do Mermaid, carregada pelo tema Material em toda página, varre o documento à procura de elementos com a classe `mermaid`, interpreta o texto dentro de cada um deles como uma definição de diagrama e o substitui por um SVG desenhado dinamicamente. Toda essa renderização acontece no navegador do leitor, no momento em que a página é carregada — não durante o `mkdocs build`, que apenas prepara o HTML intermediário.

3. Exemplo real: fluxo de CI/CD da documentação

O bloco abaixo não é uma ilustração estática: é um diagrama Mermaid real, processado pela configuração `custom_fences` descrita acima e renderizado como um `flowchart` pelo Documentation_Site no momento em que esta página é carregada no navegador.



Esse diagrama representa o fluxo de CI/CD de documentação mais comum para um projeto MkDocs hospedado no GitHub:

1. **GitHub**: o repositório onde o conteúdo em Markdown e o `mkdocs.yml` são versionados. Um `git push` para a branch principal é o gatilho de todo o fluxo.
2. **GitHub Actions**: a plataforma de automação do próprio GitHub detecta o push e inicia um workflow configurado no repositório.
3. **Build MkDocs**: dentro do workflow, um passo executa `mkdocs build` (ou `uv run mkdocs build`, no caso deste projeto), convertendo os arquivos `.md` e o `mkdocs.yml` em um site estático (HTML, CSS, JavaScript) na pasta `site/`.
4. **GitHub Pages**: o conteúdo gerado em `site/` é publicado no serviço de hospedagem estática do GitHub, associado ao repositório.
5. **Documentação publicada**: o resultado final, acessível por uma URL pública, é a versão mais recente da documentação, sempre que o fluxo completo é executado com sucesso.

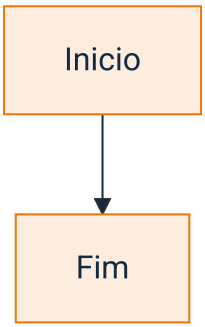
Este diagrama é conceitual: o projeto ainda não possui um workflow de GitHub Actions real configurado (veja a observação sobre o `Repository_Placeholder_URL` na página Integração com GitHub), mas o fluxo acima descreve a prática padrão para quem deseja publicar este mesmo tipo de projeto automaticamente.

4. Construir um fluxograma passo a passo

Um fluxograma Mermaid é composto por três peças: uma declaração do tipo e da direção, nós (as caixas) e conexões (as linhas ou setas). Comece descrevendo o fluxo em poucas etapas e acrescente os detalhes depois.

4.1 Declarar o tipo e a direção

A primeira linha indica o tipo do diagrama. Para fluxogramas, use `flowchart`:



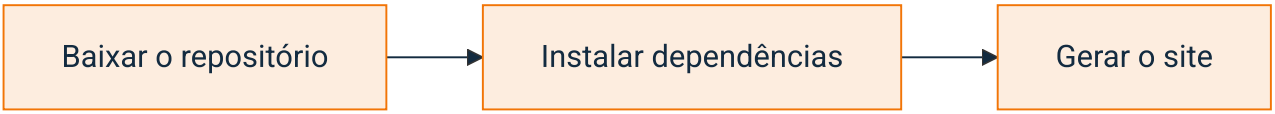
O identificador após `flowchart` define a direção:

Código	Direção	Quando usar
`TD` ou `TB`	de cima para baixo	fluxos curtos e etapas em sequência
`LR`	da esquerda para a direita	pipelines e sequências horizontais
`BT`	de baixo para cima	fluxos em que a base representa o início
`RL`	da direita para a esquerda	leitura da direita para a esquerda

O Mermaid organiza os nós automaticamente. A direção é uma sugestão de layout, não uma posição fixa para cada nó.

4.2 Criar nós com identificador e rótulo

Cada nó tem um identificador curto, usado nas conexões, e um rótulo que aparece no desenho. Mantenha os identificadores sem espaços e coloque o texto legível entre os delimitadores:



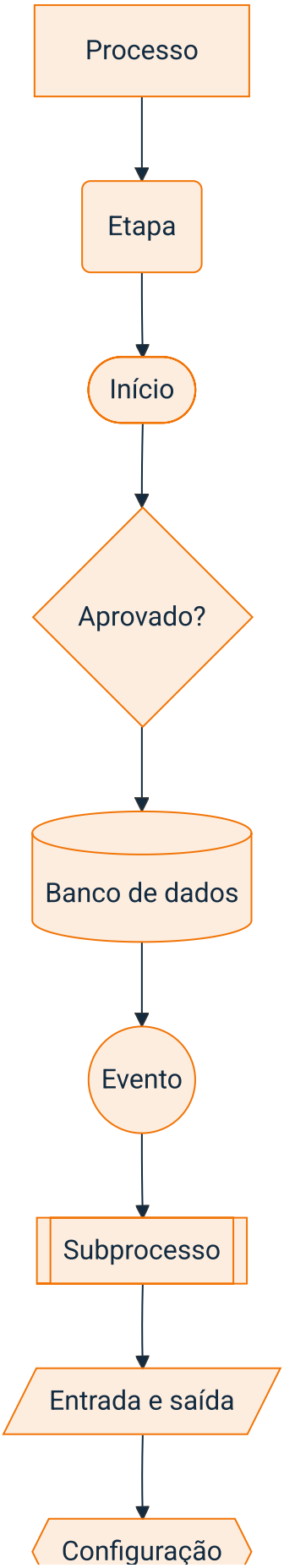
aws PARTNER Advanced Tier Services
Immersion Day
Solution Provider
DevOps Services Competency
Migration Services Competency
Nonprofit Services Competency
Public Sector Solution Provider
Well-Architected Partner Program
Solution Spark Public Sector Partners
Amazon EC2 for Windows Server Delivery
dataRain FAR BEYOND THE CLOUD

O identificador (`checkout`, `instalar` ou `build`) pode ser reutilizado para ligar o nó a outras etapas. O rótulo entre colchetes pode ter espaços e acentos. Para rótulos com pontuação ambígua, coloque o texto entre aspas, como em `A["Validar pedido (v2)?"]`.

Os delimitadores mais usados mudam o formato do nó:

Sintaxe	Forma comum	Uso sugerido
`A["Etapa"]`	retângulo	processo ou atividade
`A(Etapa)`	retângulo arredondado	etapa com destaque
`A([Início])`	cápsula	início ou fim
`A{Aprovado?}`	losango	decisão ou pergunta
`A[(Banco de dados)]`	cilindro	banco ou armazenamento
`A((Evento))`	círculo	evento ou ponto de conexão
`A[[Subprocesso]]`	processo predefinido	rotina detalhada em outro lugar
`A[/Entrada e saída/]`	paralelogramo	dados recebidos ou apresentados
`A{{ Configuração }}`	hexágono	preparação ou configuração

Veja as formas juntas em um exemplo:

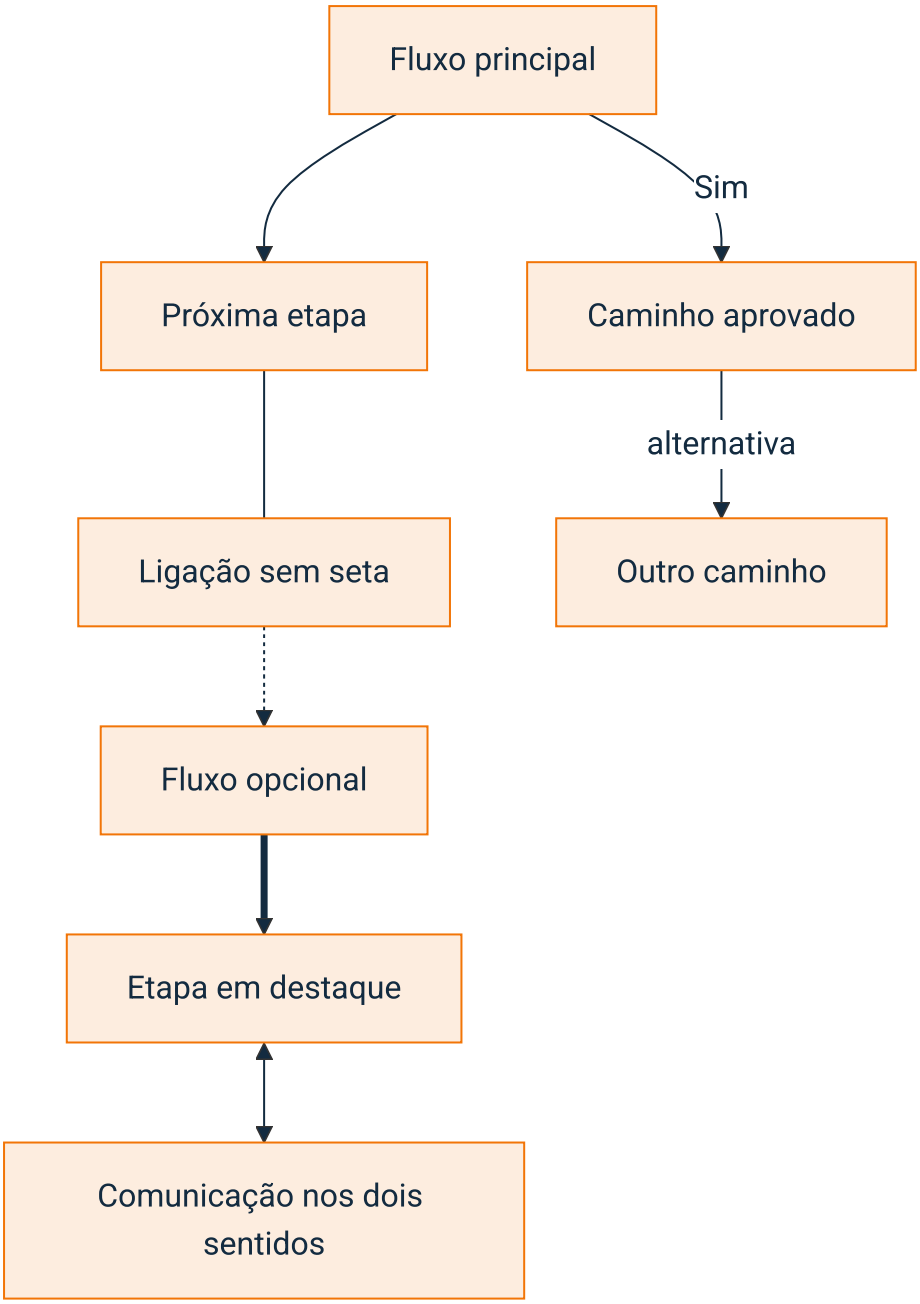

Processo
Etapa
Início
Aprovado?
Banco de dados
Evento
Subprocesso
Entrada e saída
Configuração

4.3 Conectar os nós

Escreva uma conexão entre os identificadores. As formas mais comuns são:

Sintaxe	Aparência/uso
`A --> B`	seta indicando direção
`A --- B`	linha sem ponta de seta
`A -.-> B`	seta pontilhada, útil para indicar fluxo opcional
`A ==> B`	seta mais espessa, para dar ênfase
`A <--> B`	conexão com setas nas duas direções
`A -->|Sim| B`	seta com rótulo
`A -- texto --> B`	outra forma de escrever uma conexão rotulada

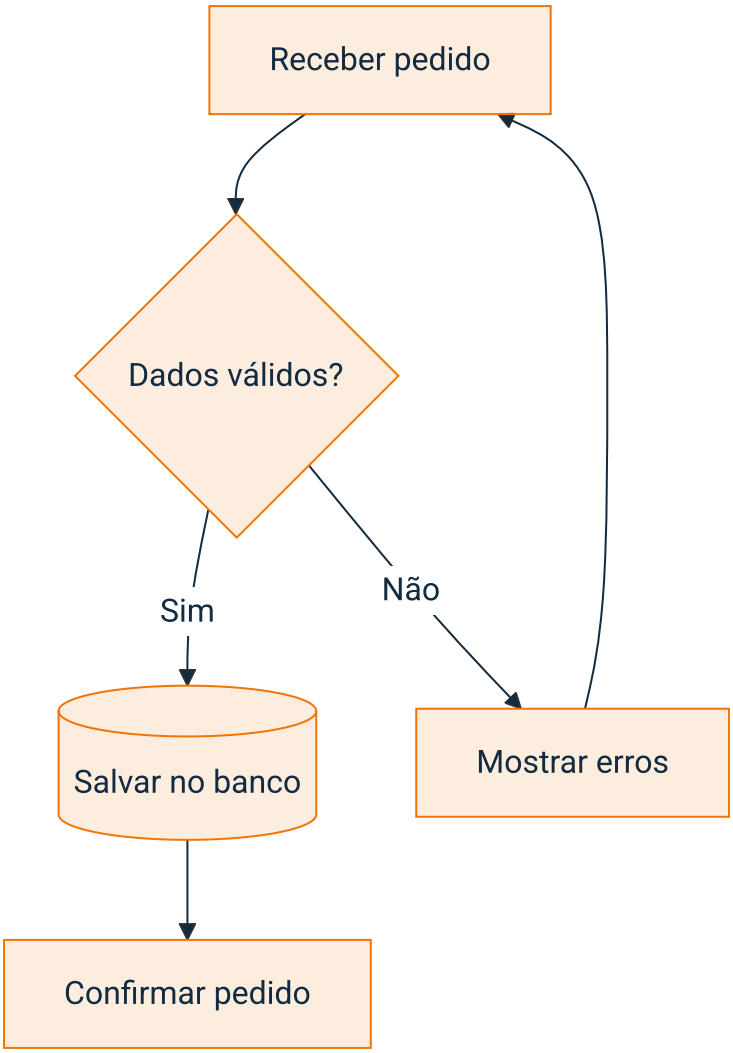
Exemplo dos diferentes tipos de conexão:



É permitido encadear conexões quando o fluxo é linear:



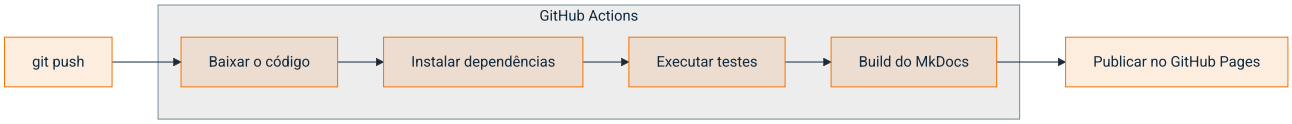
Quando houver uma condição, represente-a com um losango e dê um rótulo a cada caminho:



O retorno de `corrigir` para `receber` representa uma repetição do fluxo. Reutilizar o mesmo identificador conecta ao nó existente; criar um novo identificador cria outro nó.

4.4 Agrupar etapas relacionadas

`subgraph` desenha um contorno em volta de um grupo de nós. Use um identificador sem espaços e, se desejar, um título entre colchetes:

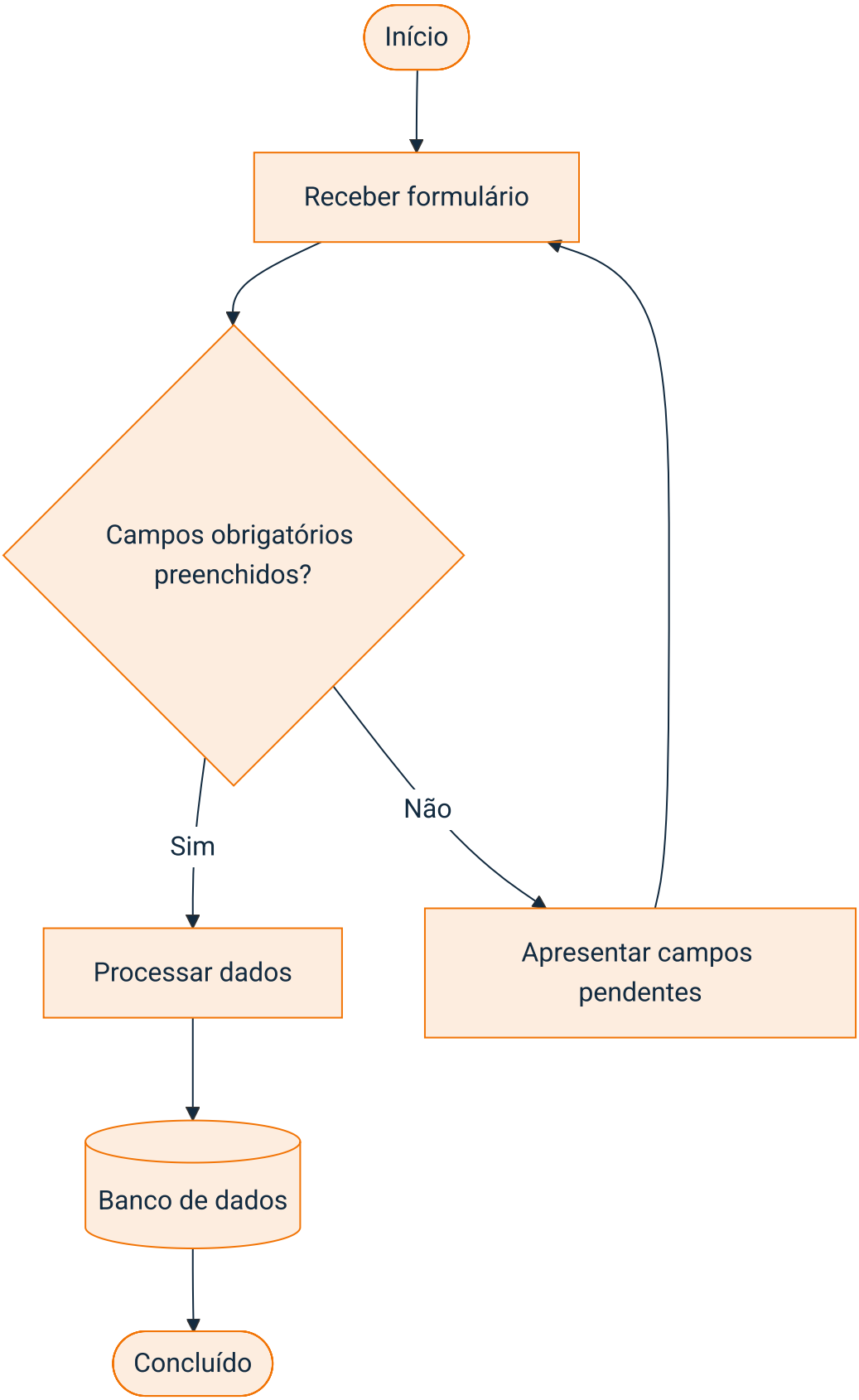


`direction TB` dentro do grupo sugere uma leitura de cima para baixo. Se nós do subgraph também estiverem conectados a nós externos, o Mermaid pode ignorar essa direção local e usar a direção do fluxograma principal. Um subgraph ajuda a separar áreas como navegador, API, banco de dados ou etapas de CI; não é necessário criar um grupo para cada nó.

4.5 Montar um diagrama completo a partir dos elementos

Este exemplo reúne início e fim, atividades, uma decisão, rótulos nas setas e um caminho de correção:

aws PARTNER Advanced Tier Services
• Immersion Day
• Solution Provider
• DevOps Services Competency
• Migration Services Competency
• Nonprofit Services Competency
• Public Sector Solution Provider
• Well-Architected Partner Program
• Solution Spark Public Sector Partners
• Amazon EC2 for Windows Server Delivery
dataRain FAR BEYOND THE CLOUD



Para criar seu próprio diagrama:

1. Liste as etapas e decisões em linguagem simples.

2. Escolha um tipo que comunique a intenção (por exemplo, retângulo para atividade e losango para decisão).
3. Dê a cada elemento um identificador curto e exclusivo.
4. Conecte-os na ordem do fluxo e nomeie os caminhos de decisão.
5. Execute `uv run mkdocs serve`, abra a página no navegador e ajuste a leitura antes de acrescentar cores ou detalhes.

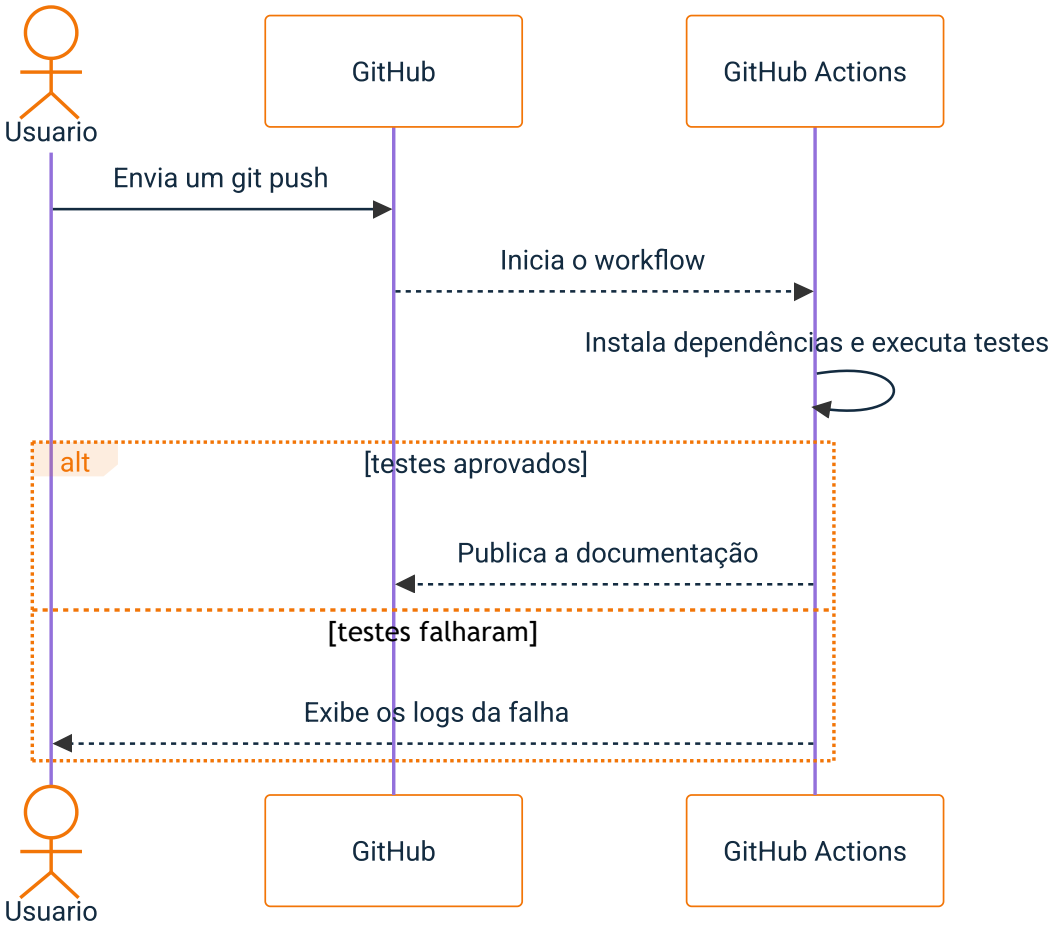
Evite usar `end` em letras minúsculas como identificador de nó em um fluxograma; essa palavra pode ser interpretada como fechamento de um grupo. Prefira `fim`, `concluido` ou `End`.

5. Outros tipos de diagrama

O Mermaid não se limita a fluxogramas. A primeira linha identifica o tipo do diagrama e a sintaxe dos elementos muda de acordo com o que se deseja explicar.

5.1 Diagrama de sequência

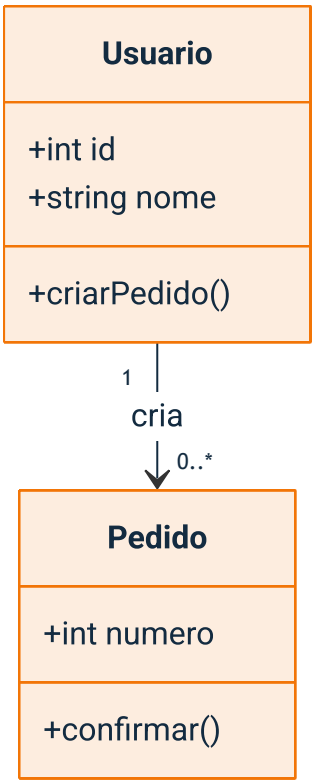
Mostra quem conversa com quem e em que ordem. `->>` indica uma mensagem e `-->>` costuma representar uma resposta:



Use `participant` para definir a ordem das colunas, `alt` e `else` para cenários alternativos e `loop` para representar uma repetição. As palavras `end` encerram esses blocos.

5.2 Diagrama de classes

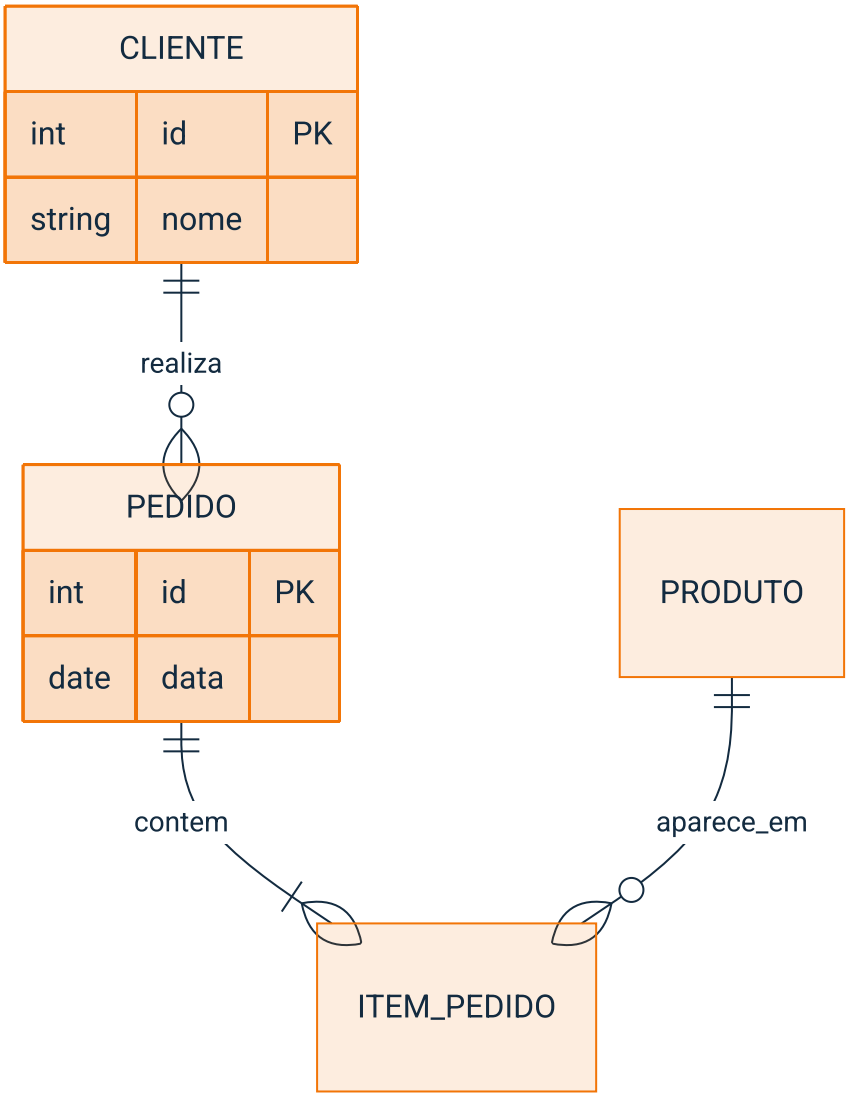
Mostra classes, atributos, métodos e relacionamentos:



Os sinais antes de um membro indicam visibilidade: `+` público, `-` privado, `#` protegido e `~` pacote. Entre classes, `--|>` representa herança, `*--` composição, `o--` agregação e `-->` associação.

5.3 Diagrama entidade-relacionamento

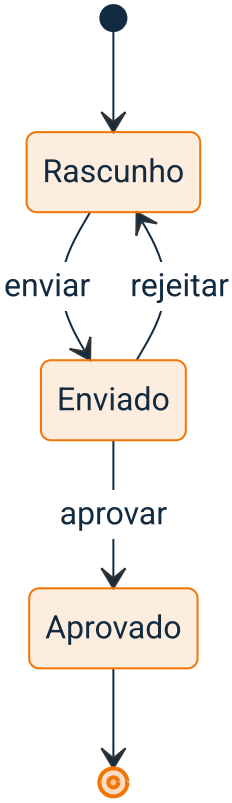
Mostra entidades e cardinalidade das relações, sendo útil para explicar um modelo de dados:



Nos extremos das relações, `||` significa exatamente um, `o|` zero ou um, `|{` um ou mais e `o{` zero ou mais. Por exemplo, `CLIENTE ||--o{ PEDIDO` indica que um cliente pode realizar zero ou vários pedidos.

5.4 Diagrama de estados

Representa os estados possíveis de um objeto e as transições entre eles. `[*]` marca o início ou o fim:



5.5 Outros formatos disponíveis

O Mermaid também tem sintaxe para cronogramas (`gantt`), gráficos de pizza (`pie`), histórico de branches (`gitGraph`), mapas mentais (`mindmap`), jornada do usuário (`journey`) e outros tipos. Consulte a galeria oficial de sintaxes para escolher o formato e ver as regras específicas de cada um.

O Material for MkDocs ajusta automaticamente cores e fontes da paleta do site para fluxogramas, sequências, classes, estados e diagramas entidade-relacionamento. Outros tipos podem renderizar, mas nem todos acompanham automaticamente o tema ou ficam legíveis em telas pequenas.

6. Personalizar cores e fontes

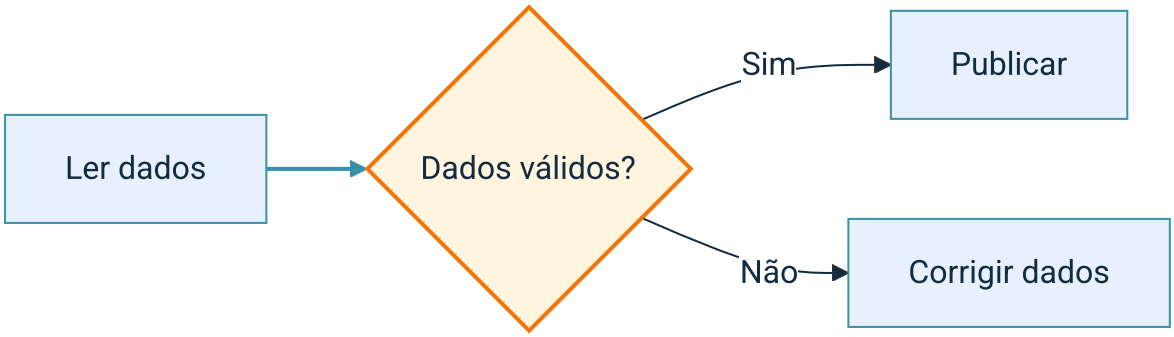
6.1 Usar as cores e fontes do tema MkDocs Material

Nesta documentação, o tema já define fontes e paletas clara e escura em `mkdocs.yml` ; o arquivo `docs/stylesheets/extra.css` define as cores institucionais. O Material aplica essas configurações automaticamente aos tipos de diagrama compatíveis.

Para mudar a identidade visual do site, altere a configuração existente em vez de fixar cores diferentes em cada diagrama. Consulte Personalização visual para ajustar a paleta, as fontes e o CSS do tema.

6.2 Colorir elementos específicos

`classDef` cria uma classe de estilo, `class` associa um ou mais nós a ela, e `linkStyle` altera uma conexão. Cores em hexadecimal ajudam a manter o resultado previsível:

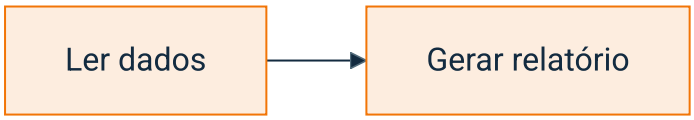


`linkStyle 0` estiliza a primeira conexão declarada; as conexões seguintes são numeradas na ordem em que aparecem. Para um ajuste pontual, também é possível usar `style validar fill:#FFF4DD,stroke:#F27507,color:#132B40`.

Estilos escritos diretamente no diagrama permanecem fixos nos modos claro e escuro. Verifique o contraste nos dois modos e não use apenas a cor para comunicar significado — mantenha rótulos como “Sim”, “Não”, “Erro” ou “Concluído”.

6.3 Ajustar o tema e a tipografia de um único diagrama

Em versões atuais do Mermaid, a configuração em frontmatter permite personalizar um diagrama. O tema `base` é o tema destinado a receber variáveis personalizadas:



`fontFamily` define a família tipográfica e `fontSize` o tamanho-base. A fonte precisa estar disponível no navegador; inclua alternativas comuns na lista para evitar substituições inesperadas. Para diagramas que não aceitem essa configuração, use a paleta global do Material e estilos `classDef`.

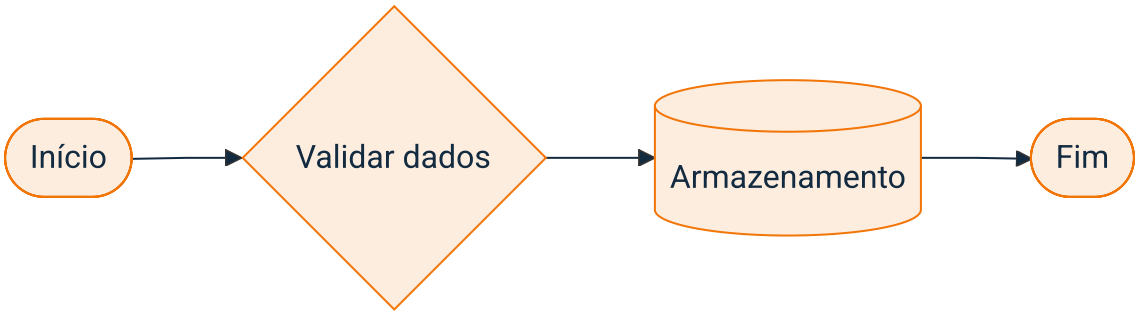
7. Ajustar tamanho, espaçamento e legibilidade

O diagrama é desenhado como SVG dentro da largura disponível na página. Quando há muitos nós ou textos longos, o navegador pode reduzir o desenho para fazê-lo caber. Antes de aumentar a fonte globalmente, ajuste o conteúdo e o layout:

- Escolha `LR` para uma sequência horizontal ou `TD` para etapas verticais.
- Reduza textos dentro dos nós; explique detalhes em um parágrafo ao lado.

- Divida processos grandes em diagramas menores e use subgraphs apenas para grupos relevantes.
- Evite cruzar setas; reorganize etapas e diminua conexões redundantes.
- Dê nomes distintos aos identificadores e deixe a direção do fluxo explícita.
- Confira a página em largura de desktop e de celular. O Mermaid Live Editor ajuda a testar a sintaxe, mas faça a conferência final no próprio MkDocs.

Para um fluxograma, o frontmatter também permite ajustar distância entre nós, distância entre níveis, largura para quebra de texto e formato das linhas:



Esses valores são um ponto de partida, não uma regra. Se o diagrama continuar ilegível, divida-o ou mantenha a largura natural com rolagem horizontal.

7.1 CSS responsivo opcional

O `mkdocs.yml` deste projeto já carrega `docs/stylesheets/extra.css`. Se os SVGs não se ajustarem bem à página, adicione estas regras ao final desse arquivo:

```
.md-typeset .mermaid {
  max-width: 100%;
  overflow-x: auto;
}

.md-typeset .mermaid svg {
  display: block;
  max-width: 100%;
  height: auto;
  margin: 0 auto;
}
```

Esse estilo permite que o SVG se adapte à largura do conteúdo. Se um fluxograma muito largo ficar pequeno demais, mantenha `overflow-x: auto` no contêiner e altere `max-width` do SVG para `none`; a página permitirá rolagem horizontal em vez de reduzir o desenho inteiro:

```
.md-typeset .mermaid svg {
  max-width: none;
}
```

Use essa segunda opção somente quando necessário: ela preserva o tamanho natural do desenho, mas pode exigir rolagem em telas pequenas.

7.2 Zoom e movimento interativos com `svg-pan-zoom`

Se a rolagem horizontal não for suficiente, a biblioteca JavaScript `svg-pan-zoom` adiciona zoom pela roda do mouse, movimento por arraste e, opcionalmente, controles visuais para ampliar, reduzir e centralizar o SVG. Ela funciona sobre os SVGs que o Mermaid já gera no navegador; não é necessário instalar um plugin Python do MkDocs.

Esta alternativa complementa a rolagem, mas não abre automaticamente um modo de tela cheia. Mantenha uma alternativa responsiva para telas pequenas e confira o resultado nos modos claro e escuro.

Passo 1: escolha um diagrama para testar

Este exemplo pode ser copiado para qualquer página Markdown do projeto. Depois da integração, será possível ampliar e arrastar o SVG:

````
```mermaid
flowchart LR
    push["Enviar alterações"] --> workflow["Iniciar GitHub Actions"]
    workflow --> teste["Executar testes"]
    teste -->|Aprovado| publicar["Publicar documentação"]
    teste -->|Falhou| corrigir["Consultar logs e corrigir"]
```
````

Passo 2: carregue a biblioteca antes do script de inicialização

No `mkdocs.yml`, acrescente os dois arquivos à lista `extra_javascript`. Preserve os scripts que já existem; a ordem importa: a biblioteca vem antes do script personalizado.

```
extra_javascript:
  - javascripts/mathjax.js
  - https://unpkg.com/mathjax@3/es5/tex-mml-chtml.js
  - https://cdn.jsdelivr.net/npm/svg-pan-zoom@3.6.2/dist/svg-pan-zoom.min.js
  - javascripts/mermaid-panzoom.js
```

O exemplo usa uma versão fixada para evitar atualizações inesperadas. Como a primeira URL depende de uma CDN, o navegador dos leitores precisa acessar a internet. Em redes privadas ou ambientes sem acesso à CDN, baixe a distribuição da versão escolhida para `docs/javascripts/vendor/` e use o caminho local:

```
extra_javascript:
  - javascripts/vendor/svg-pan-zoom.min.js
  - javascripts/mermaid-panzoom.js
```

Ao distribuir uma cópia local da biblioteca, mantenha também os avisos de licença BSD-2-Clause do projeto original.

Passo 3: inicialize os SVGs gerados pelo Mermaid

Crie `docs/javascripts/mermaid-panzoom.js` com o conteúdo abaixo. O `MutationObserver` detecta SVGs que o Mermaid renderiza depois do carregamento da página; `document$.subscribe` repete a busca após cada navegação instantânea do Material. Um `WeakSet` evita inicializar o mesmo SVG mais de uma vez.

```
(() => {
  const initializedDiagrams = new WeakSet();
  const diagramSelector = ".md-typeset .mermaid svg";

  function initializeDiagram(svg) {
    if (initializedDiagrams.has(svg)) {
      return;
    }

    window.svgPanZoom(svg, {
      zoomEnabled: true,
      panEnabled: true,
      mouseWheelZoomEnabled: true,
      dblClickZoomEnabled: true,
      controlIconsEnabled: true,
      fit: true,
      center: true,
      minZoom: 0.5,
      maxZoom: 10,
    });

    initializedDiagrams.add(svg);
  }

  function initializeDiagramsIn(node) {
    if (node.nodeType === Node.ELEMENT_NODE && node.matches(diagramSelector)) {
      initializeDiagram(node);
    }

    if (typeof node.querySelectorAll === "function") {
      node.querySelectorAll(diagramSelector).forEach(initializeDiagram);
    }
  }

  const observer = new MutationObserver((mutations) => {
    mutations.forEach((mutation) => {
      mutation.addedNodes.forEach(initializeDiagramsIn);
    });
  });

  observer.observe(document.body, {
    childList: true,
    subtree: true,
  });

  document$.subscribe(() => {
```

```
    requestAnimationFrame(() => initializeDiagramsIn(document));
  });

  initializeDiagramsIn(document);
})();
```

Os controles visuais são criados porque `controlIconsEnabled` está habilitado. Para remover os controles e usar apenas roda do mouse, duplo clique e arraste, desative essa opção; para limitar a ampliação, ajuste `minZoom` e `maxZoom`.

Passo 4: teste a integração

Execute o site local e teste o diagrama com a roda do mouse, duplo clique, arraste e controles visuais. Navegue para outra página e volte para confirmar que a inicialização continua funcionando com `navigation.instant`.

```
uv run mkdocs serve
```

Se o diagrama não receber os controles, abra o console do navegador e confira se a CDN ou o arquivo local da biblioteca carregou sem erro. Para redes restritas, prefira a cópia local. A biblioteca não transforma o diagrama em texto acessível por si só; descreva o fluxo também em texto e não dependa apenas das cores ou dos controles visuais.

8. Erros comuns e diagnóstico

Sintoma	O que conferir
O bloco aparece como código, em vez de diagrama	Use exatamente uma cerca ```` ```mermaid ```` e confirme `custom_fences` em `mkdocs.yml`.
O diagrama não renderiza	Revise a primeira linha (`flowchart`, `sequenceDiagram` etc.), os pares de delimitadores e as setas.
Um nó desaparece ou um grupo termina cedo	Evite o identificador `end` em minúsculas e use nomes exclusivos.
O SVG fica pequeno	Reduza o número de nós, encurte rótulos, mude a direção ou habilite rolagem horizontal.
O zoom/pan não aparece	Confirme que `svg-pan-zoom.min.js` carrega antes de `mermaid-panzoom.js`, que o seletor corresponde aos SVGs e que o console não mostra erros.

Sintoma	O que conferir
Cores ficam sem contraste	Confira os modos claro e escuro e prefira a paleta do Material em vez de cores fixas.
A página local não mostra a alteração	Deixe `uv run mkdocs serve` ativo e atualize o navegador; confira também o console do navegador se houver erro de sintaxe.

9. Alternativas conceituais: PlantUML e Graphviz

O Mermaid cobre bem fluxogramas, diagramas de sequência, de classes e vários outros tipos de diagrama comuns em documentação de software, mas não é a única ferramenta de diagramação como texto que existe no ecossistema. Duas alternativas frequentemente citadas são:

- **PlantUML**: focado especificamente em diagramas UML (casos de uso, sequência, classes, componentes, entre outros), com uma sintaxe própria e, em muitos casos, um nível de detalhe e de conformidade com a notação UML formal maior do que o Mermaid oferece. A limitação prática do PlantUML é que ele não é um interpretador JavaScript executado no navegador como o Mermaid: ele é escrito em Java e depende de um **binário Java** instalado no ambiente que gera a documentação (ou de um serviço externo de renderização) para transformar o texto do diagrama em uma imagem.
- **Graphviz**: focado em grafos genéricos — nós e arestas, com um algoritmo de layout automático particularmente forte para representar relações de dependência, hierarquias e estruturas de árvore. Diferente do PlantUML, o Graphviz não é baseado em Java, mas depende de um **binário externo** próprio, o comando `dot`, que precisa estar instalado no sistema operacional para converter a definição textual do grafo (a linguagem DOT) em uma imagem renderizada.

Nenhuma das duas ferramentas foi instalada neste projeto, e esta seção é apenas uma referência conceitual para o leitor que, em um projeto diferente, precise de um tipo de diagrama que o Mermaid não cobre bem (por exemplo, um diagrama de classes UML mais rigoroso ou um grafo de dependências complexo). A razão para não as adotar aqui é prática: tanto o PlantUML quanto o Graphviz exigiriam a instalação de um binário externo ao ambiente Python (uma máquina virtual Java, no caso do PlantUML, ou o executável `dot`, no caso do Graphviz), o que está fora do escopo de uma dependência instalável simplesmente via `uv add`, como todos os demais plugins deste tutorial. O Mermaid, por não precisar de nada além do JavaScript já embutido no tema Material, continua sendo a opção padrão deste projeto sempre que um diagrama é necessário.

10. Referências oficiais

- Sintaxe dos fluxogramas
- Diagramas de sequência

- Diagramas de classes
- Diagramas entidade-relacionamento
- Diagramas de estados
- Temas e variáveis de estilo
- Configuração de fluxogramas: espaçamento e layout
- Integração de diagramas no Material for MkDocs
- JavaScript adicional e navegação instantânea no Material for MkDocs
- Biblioteca `svg-pan-zoom`: instalação, opções e exemplos

2026-11-13 2026-11-11

uv: gerenciar projetos e dependências Python

`uv` é uma ferramenta para instalar e gerenciar versões do Python, criar projetos, resolver dependências e executar comandos dentro de ambientes virtuais. Em um projeto, o `pyproject.toml` descreve as dependências e configurações; o `uv.lock` registra a resolução exata usada para reproduzir o ambiente.

1. Instalar o uv

No Windows, instale pelo WinGet:

```
winget install --id=astral-sh.uv -e
```

Ou use o instalador oficial no PowerShell:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

No macOS ou Linux, o instalador oficial é:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Feche e reabra o terminal e confirme a instalação:

```
uv --version
```

Se o comando não for reconhecido, confirme se o diretório de instalação do `uv` está no `PATH` e abra um novo terminal. A instalação e as opções por sistema estão na documentação oficial de instalação do uv.

2. Criar um projeto com `uv init`

Crie uma pasta de projeto e inicialize nela:

```
uv init meu-projeto
cd meu-projeto
```

No template padrão atual para aplicação empacotada, o `uv init` cria arquivos semelhantes a estes:

```
meu-projeto/
├── .gitignore
├── .python-version
├── README.md
├── pyproject.toml
└── src/
```

```
    └── meu_projeto/
        └── __init__.py
```

- `.python-version` indica a versão padrão do Python para o projeto.
- `pyproject.toml` contém os metadados e as dependências.
- `src/meu_projeto/` é o pacote Python inicial. O sublinhado substitui o hífen do nome da distribuição.
- `.gitignore` exclui arquivos gerados, como `.venv`.
- `README.md` é o ponto de partida para a descrição do projeto.

O template pode variar conforme a versão do `uv` e as opções do comando. Para um script simples sem sistema de build, use `uv init --no-package meu-script`; esse modelo cria `main.py` em vez do pacote em `src/`. Para uma biblioteca distribuível, use `uv init --lib minha-biblioteca`.

O `uv init` não precisa criar o ambiente e o lockfile imediatamente. `uv lock`, `uv run` ou `uv sync` podem criar o `uv.lock`; já a pasta `.venv` é criada quando `uv run` ou `uv sync` prepara o ambiente. `uv lock` sozinho não cria nem instala um ambiente virtual. Depois desses comandos, a estrutura pode incluir:

```
meu-projeto/
├── .venv/       # Ambiente virtual local; não deve ir para o Git
└── uv.lock      # Versões resolvidas; deve ser versionado
```

3. Executar comandos e scripts

`uv run` prepara ou sincroniza o ambiente do projeto antes de executar um comando. Assim, não é necessário ativar `.venv` manualmente:

```
uv run python --version
uv run python main.py
```

Para executar uma ferramenta instalada como dependência do projeto:

```
uv run pytest
```

Para sincronizar o ambiente explicitamente, por exemplo para que o editor encontre o interpretador e as dependências:

```
uv sync
```

O ambiente do projeto fica em `.venv/`, ao lado de `pyproject.toml`. Use `uv add` para dependências persistentes; evite instalar pacotes de projeto manualmente dentro de `.venv`, pois uma sincronização posterior pode removê-los.

4. Configurar o `pyproject.toml`

`pyproject.toml` é o arquivo TOML padrão para metadados e configurações de projetos Python. O bloco `[project]` segue o padrão PEP 621; o `uv` também reconhece configurações próprias em `[tool.uv]`.

Exemplo de metadados, dependência de execução e grupos:

```toml
[project]
name = "meu-projeto"
version = "0.1.0"
description = "Descrição do projeto"
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "httpx>=0.27,<1",
]

[dependency-groups]
dev = [
    "pytest>=8",
    "ruff>=0.11",
]
docs = [
    "mkdocs>=1.6",
]

[project.optional-dependencies]
postgres = [
    "psycopg[binary]>=3.2",
]
```

O significado das seções:

- **`[project]`**: nome, versão, Python compatível e dependências necessárias para executar o projeto.
- **`dependencies`**: dependências obrigatórias de execução; são consideradas ao instalar o projeto e ao publicar uma distribuição.
- **`[dependency-groups]`**: ferramentas e bibliotecas locais de desenvolvimento, como testes, lint e documentação.
- **`[project.optional-dependencies]`**: recursos opcionais publicados como *extras*, que podem ser instalados por quem consome a biblioteca.
- **`[tool.uv]`**: opções específicas do `uv`, como fontes Git, índices ou grupos padrão.
- **`[build-system]`**: backend usado para empacotar o projeto. O template de `uv init` normalmente o configura; não é necessário alterá-lo para apenas adicionar dependências.

Você pode editar o TOML manualmente, mas os comandos `uv add` e `uv remove` ajudam a manter o arquivo e o lockfile sincronizados.

5. Adicionar e remover dependências

Adicione uma dependência necessária em execução:

```
uv add httpx
```

Isso acrescenta a dependência a `[project].dependencies`, atualiza `uv.lock` e sincroniza o ambiente. Para removê-la:

```
uv remove httpx
```

Para fixar uma faixa de versões, informe o requisito:

```
uv add "httpx>=0.27,<1"
```

Para executar uma ferramenta sem registrá-la permanentemente no projeto, use `uvx` (atalho para executar uma ferramenta em um ambiente isolado):

```
uvx ruff --version
```

6. Separar dependências de desenvolvimento e produção

Use **`project.dependencies`** para bibliotecas necessárias ao funcionamento do aplicativo e **`dependency-groups.dev`** para ferramentas de desenvolvimento, testes e lint. O grupo `dev` é local: não é publicado nos requisitos da distribuição.

Comandos equivalentes para gerenciar essas listas:

```
uv add fastapi
uv add --dev pytest ruff
uv add --group docs mkdocs
```

O resultado fica separado no `pyproject.toml`:

```
[project]
dependencies = [
    "fastapi>=0.115",
]

[dependency-groups]
dev = [
    "pytest>=8",
    "ruff>=0.11",
]
docs = [
    "mkdocs>=1.6",
```

```
]
```

Comportamento dos comandos de sincronização:

```
uv sync                 # dependências de execução + grupo dev (padrão)
uv sync --no-dev        # dependências de execução, sem o grupo dev
uv sync --locked --no-dev  # produção/CI: não altera o lockfile
uv sync --group docs    # inclui o grupo docs além dos grupos padrão
```

Por padrão, `uv sync` inclui o grupo `dev`. `--no-dev` o exclui; a opção `--locked` falha se `pyproject.toml` e `uv.lock` não estiverem sincronizados, em vez de atualizar o lockfile silenciosamente. `uv sync` também faz uma sincronização exata e pode remover pacotes que foram instalados manualmente, mas não estão declarados.

Use `[project.optional-dependencies]` quando o recurso opcional deve ser oferecido aos consumidores da biblioteca como *extra*:

```
uv add --optional postgres "psycopg[binary]>=3.2"
uv sync --extra postgres
```

Extras não são instalados por padrão. Diferentemente do grupo `dev`, eles fazem parte dos metadados publicados do projeto.

7. O que é `uv.lock` e como atualizá-lo

`uv.lock` é o lockfile gerenciado pelo `uv`. Enquanto `pyproject.toml` declara requisitos e faixas aceitáveis, o lockfile registra a resolução exata das dependências diretas e transitivas, as fontes dos pacotes e as condições por plataforma/Python. O formato é universal para que o mesmo arquivo possa orientar instalações em diferentes sistemas compatíveis.

O lockfile é criado ou atualizado por `uv lock`, `uv add`, `uv remove` e também automaticamente durante comandos de projeto como `uv run` e `uv sync`. **Não edite `uv.lock` manualmente**; altere o `pyproject.toml` ou use os comandos do `uv`.

Comandos úteis:

```
uv lock --check                  # verifica se o lockfile corresponde ao projeto
uv lock                          # resolve e atualiza o lockfile
uv lock --upgrade                # atualiza pacotes dentro dos requisitos declarados
uv lock --upgrade-package httpx  # atualiza apenas um pacote
uv sync --locked                 # instala sem modificar um lockfile desatualizado
```

Versione `uv.lock` junto com `pyproject.toml` para que desenvolvimento, CI e implantação usem a mesma resolução. Não versione `.venv/`.

8. Interpretar a estrutura deste repositório

O `pyproject.toml` deste tutorial declara os pacotes do site em `[project].dependencies`; a origem Git e a revisão fixada de `griffe-typedoc` ficam em `[tool.uv.sources]`. O `uv.lock` contém a resolução exata de todos eles. Como as ferramentas MkDocs fazem parte da tarefa principal deste repositório, elas estão declaradas como dependências do projeto, e não como grupo `dev`.

Para instalar o ambiente documentado, clone o repositório e execute:

```
uv sync
```

Isso lê os requisitos do TOML e as versões do lockfile; não é necessário repetir uma série de comandos `uv add` pacote por pacote.

9. Comandos principais

Comando	Uso
`uv init`	Cria a estrutura inicial de um projeto.
`uv add pacote`	Adiciona uma dependência de execução.
`uv add --dev pacote`	Adiciona uma dependência ao grupo de desenvolvimento.
`uv remove pacote`	Remove uma dependência.
`uv sync`	Sincroniza o ambiente com o projeto e o lockfile.
`uv run comando`	Executa um comando dentro do ambiente do projeto.
`uv lock`	Resolve e atualiza o lockfile.
`uv tree`	Mostra a árvore de dependências.
`uv python install 3.12`	Instala uma versão gerenciada do Python.
`uv python pin 3.12`	Define a versão padrão do projeto em `.python-version`.
`uv build`	Constrói os arquivos de distribuição em `dist/`.

Referências oficiais

- Instalação do uv
- Criar projetos com `uv init`
- Trabalhar em projetos
- Configurar dependências
- Lockfile e sincronização
- Estrutura do projeto e arquivos

2026-11-13 2026-11-13

Criar diagramas do Draw.io com Python e `drawpyo`

`drawpyo` é uma biblioteca Python para gerar arquivos de diagramas que podem ser abertos e editados no Draw.io (também chamado diagrams.net). O código cria o arquivo, a página, os elementos e as conexões; depois, o Draw.io pode exportar o resultado como SVG ou HTML.

Há dois tipos de recursos externos neste tutorial:

- **Bibliotecas `mxlibrary`**, como o arquivo `developer.xml` indicado aqui, são coleções de formas do Draw.io. O arquivo não é uma biblioteca Python nem um conjunto de ícones AWS.
- **Ícones SVG da AWS** são arquivos de imagem que podem ser incorporados aos elementos do diagrama. O exemplo AWS abaixo usa SVGs oficiais baixados do pacote da AWS.

1. Instalar e preparar o projeto

Crie um projeto Python e instale `drawpyo`:

```
uv init diagramas-cloud
cd diagramas-cloud
uv add "drawpyo @ git+https://github.com/MerrimanInd/drawpyo.git@0.2.6"
```

A versão 0.2.6 ou posterior é necessária para carregar bibliotecas `mxlibrary` externas. Se essa versão já estiver disponível no PyPI, também é possível instalá-la com `uv add "drawpyo>=0.2.6"`; consulte as releases para verificar a versão atual.

Crie uma pasta para scripts e outra para os arquivos gerados:

```
New-Item -ItemType Directory -Force scripts, docs\assets\diagrams
```

2. Criar o primeiro diagrama

Um arquivo Draw.io pode ter uma ou mais páginas. Cada página contém objetos (formas) e arestas (conexões). Salve o exemplo como `scripts/primeiro_diagrama.py`:

```
from pathlib import Path

import drawpyo

output_dir = Path("docs/assets/diagrams")
output_dir.mkdir(parents=True, exist_ok=True)

diagram = drawpyo.File()
diagram.file_path = str(output_dir.resolve())
```

```
diagram.file_name = "primeiro-diagrama.drawio"

page = drawpyo.Page(file=diagram)
page.name = "Fluxo inicial"

inicio = drawpyo.diagram.Object(
    page=page,
    value="Receber solicitação",
    position=(80, 120),
    width=160,
    height=60,
)
inicio.fillColor = "#E8F0FE"
inicio.strokeColor = "#3791A6"

processo = drawpyo.diagram.Object(
    page=page,
    value="Processar dados",
    position=(340, 120),
    width=160,
    height=60,
)
processo.fillColor = "#FFF4DD"
processo.strokeColor = "#F27507"

conexao = drawpyo.diagram.Edge(
    page=page,
    source=inicio,
    target=processo,
)
conexao.strokeColor = "#526B7A"
conexao.strokeWidth = 2

diagram.write()
print(f"Diagrama salvo em: {output_dir / diagram.file_name}")
```

Execute o script e abra o arquivo `.drawio` no Draw.io:

```
uv run python scripts/primeiro_diagrama.py
```

Use `Object` para criar formas, defina `position`, `width` e `height` para controlar a geometria e conecte os objetos com `Edge`. Os códigos hexadecimais definem as cores de preenchimento e da borda.

3. Carregar a biblioteca externa `developer.xml`

O arquivo `developer.xml` é uma biblioteca do formato nativo do Draw.io, chamada `mxlibrary`. Ela contém símbolos relacionados a desenvolvimento e integração. O projeto `drawpyo` consegue carregá-la de uma URL ou de um arquivo local.

Para registrar e usar uma forma, acrescente o código abaixo depois de criar `diagram` e `page`:

```
import drawpyo

developer_url = (
    "https://raw.githubusercontent.com/jgraph/drawio-libs/"
    "refs/heads/main/libs/integration/developer.xml"
)

drawpyo.register_mxlibrary("developer", developer_url)

ferramenta = drawpyo.diagram.object_from_library(
    page=page,
    library="developer",
    obj_name="App Developing",
    value="Desenvolvimento",
    position=(210, 240),
)

diagram.write()
```

Os nomes das formas dependem do conteúdo da biblioteca e precisam corresponder ao nome registrado nela. Liste os nomes disponíveis antes de escolher um:

```
formas = drawpyo.load_mxlibrary(developer_url)
print("\n".join(list(formas)[:30]))
```

Para evitar depender da disponibilidade da URL e de alterações futuras no branch `main`, baixe uma cópia revisada da biblioteca para o projeto e registre o caminho local:

```
drawpyo.register_mxlibrary(
    "developer",
    "assets/libraries/developer.xml",
)
```

> **Importante:** `developer.xml` não contém os ícones de serviços AWS. Para representar uma arquitetura AWS, use os SVGs oficiais na próxima etapa ou outra biblioteca AWS compatível com o formato `mxlibrary`.

4. Criar um diagrama AWS usando ícones SVG

4.1 Obter os ícones oficiais

Baixe o pacote atual em AWS Architecture Icons. Extraia o pacote e copie apenas os SVGs necessários para `assets/aws/`. Neste exemplo, salve os arquivos selecionados com estes nomes:

```
assets/
└── aws/
    ├── api-gateway.svg
    ├── lambda.svg
    └── dynamodb.svg
```

Os nomes são escolhas locais para o exemplo: copie o SVG equivalente no pacote e renomeie-o. A AWS atualiza os ícones periodicamente; consulte a página oficial para obter a versão mais recente e respeite os termos de uso e avisos incluídos no pacote.

4.2 Incorporar os SVGs no arquivo editável

O exemplo lê cada SVG local, codifica seus bytes em Base64 e os coloca no estilo `image` do Draw.io. Assim, os ícones ficam incorporados ao `.drawio`; o arquivo não depende de URLs de imagem externas quando for aberto em outra máquina.

Salve como `scripts/diagrama_aws.py`:

```
import base64
from pathlib import Path

import drawpyo


def adicionar_icone_svg(page, caminho_svg, rotulo, posicao):
    svg_base64 = base64.b64encode(
        Path(caminho_svg).read_bytes()
    ).decode("ascii")

    icone = drawpyo.diagram.Object(page=page, value="")
    icone.position = posicao
    icone.width = 64
    icone.height = 64
    icone.apply_style_string(
        "shape=image;"
        "aspect=fixed;"
        "imageAspect=0;"
        "verticalLabelPosition=bottom;"
        "verticalAlign=top;"
        f"image=data:image/svg+xml,{svg_base64};"
    )

    legenda = drawpyo.diagram.Object(
        page=page,
        value=rotulo,
        position=(posicao[0] - 35, posicao[1] + 74),
        width=134,
        height=36,
    )
    legenda.text_format.align = "center"
    legenda.fillColor = "none"
    legenda.strokeColor = "none"

    return icone


output_dir = Path("docs/assets/diagrams")
output_dir.mkdir(parents=True, exist_ok=True)
```

```
diagram = drawpyo.File()
diagram.file_path = str(output_dir.resolve())
diagram.file_name = "aws-arquitetura.drawio"

page = drawpyo.Page(file=diagram)
page.name = "API, processamento e dados"

regiao = drawpyo.diagram.Object(
    page=page,
    value="AWS | us-east-1",
    position=(45, 70),
    width=570,
    height=210,
)
regiao.rounded = True
regiao.fillColor = "#F8F9FA"
regiao.strokeColor = "#879196"

api = adicionar_icone_svg(
    page,
    "assets/aws/api-gateway.svg",
    "Amazon API Gateway",
    (90, 145),
)
funcao = adicionar_icone_svg(
    page,
    "assets/aws/lambda.svg",
    "AWS Lambda",
    (275, 145),
)
banco = adicionar_icone_svg(
    page,
    "assets/aws/dynamodb.svg",
    "Amazon DynamoDB",
    (460, 145),
)

for origem, destino in ((api, funcao), (funcao, banco)):
    conexao = drawpyo.diagram.Edge(
        page=page,
        source=origem,
        target=destino,
    )
    conexao.strokeColor = "#545B64"
    conexao.strokeWidth = 2

diagram.write()
print(f"Diagrama AWS salvo em: {output_dir / diagram.file_name}")
```

Gere o arquivo e abra `docs/assets/diagrams/aws-arquitetura.drawio` no Draw.io:

```
uv run python scripts/diagrama_aws.py
```

Se o Draw.io não renderizar um SVG, valide que o arquivo é um SVG independente e bem formado. Prefira os SVGs oficiais, sem scripts ou referências externas. Os dados Base64 aumentam o tamanho do arquivo;

incorpore somente os ícones usados.

4.3 Gerar o fluxo usando as imagens do projeto

Este passo a passo monta o fluxo solicitado:

```
user.png -> imagem1.png -> Api_gateway.png -> lambda.png -> s3.png
```

4.3.1 Organizar as imagens e o script

Salve os cinco PNGs dentro de `docs/assets/`. Os nomes devem corresponder exatamente aos usados pelo script:

Etapa	Arquivo de entrada
Usuário	`docs/assets/user.png`
Imagem enviada	`docs/assets/imagem1.png`
API Gateway	`docs/assets/Api_gateway.png`
Lambda	`docs/assets/lambda.png`
S3	`docs/assets/s3.png`

O diretório `docs/assets/diagrams/` receberá o diagrama gerado. O script cria esse diretório automaticamente; para criá-lo manualmente no PowerShell:

```
New-Item -ItemType Directory -Force docs\assets\diagrams
```

Salve o script em `examples/diagrama_fluxo_imagens.py`, na raiz do repositório. A estrutura relevante fica assim:

```
repositorio/
├── examples/
│   └── diagrama_fluxo_imagens.py
└── docs/
    └── assets/
        ├── user.png
        ├── imagem1.png
        ├── Api_gateway.png
        ├── lambda.png
        ├── s3.png
        └── diagrams/
            └── fluxo-imagens.drawio  (gerado pelo script)
```

O código resolve os caminhos a partir da localização do próprio script, portanto pode ser executado de outras pastas sem alterar os caminhos dos PNGs. Ele incorpora os bytes de cada imagem no `.drawio` em Base64, acrescenta uma legenda e cria uma seta para cada ligação:

```python
import base64
from pathlib import Path

import drawpyo


project_root = Path(__file__).resolve().parents[1]
assets_dir = project_root / "docs" / "assets"
output_dir = assets_dir / "diagrams"
output_dir.mkdir(parents=True, exist_ok=True)

diagram = drawpyo.File()
diagram.file_path = str(output_dir.resolve())
diagram.file_name = "fluxo-imagens.drawio"

page = drawpyo.Page(file=diagram)
page.name = "Usuário até S3"

etapas = [
    ("user.png", "Usuário"),
    ("imagem1.png", "Imagem 1"),
    ("Api_gateway.png", "API Gateway"),
    ("lambda.png", "Lambda"),
    ("s3.png", "S3"),
]

icones = []
posicao_inicial_x = 55
distancia_entre_etapas = 170
posicao_y = 120

for indice, (nome_arquivo, rotulo) in enumerate(etapas):
    caminho_imagem = assets_dir / nome_arquivo
    if not caminho_imagem.is_file():
        raise FileNotFoundError(f"Imagem não encontrada: {caminho_imagem}")

    imagem_base64 = base64.b64encode(caminho_imagem.read_bytes()).decode("ascii")
    x = posicao_inicial_x + indice * distancia_entre_etapas
```

```
    icone = drawpyo.diagram.Object(page=page, value="")
    icone.position = (x, posicao_y)
    icone.width = 72
    icone.height = 72
    icone.apply_style_string(
        "shape=image;"
        "aspect=fixed;"
        "imageAspect=0;"
        f"image={imagem_base64};"
    )
    icones.append(icone)

    legenda = drawpyo.diagram.Object(
        page=page,
        value=rotulo,
        position=(x - 30, posicao_y + 82),
        width=132,
        height=32,
    )
    legenda.text_format.align = "center"
    legenda.fillColor = "none"
    legenda.strokeColor = "none"

for origem, destino in zip(icones, icones[1:]):
    conexao = drawpyo.diagram.Edge(page=page, source=origem, target=destino)
    conexao.strokeColor = "#526B7A"
    conexao.strokeWidth = 2
    conexao.apply_style_string("endArrow=block;")

diagram.write()
print(f"Diagrama salvo em: {output_dir / diagram.file_name}")
```

4.3.2 Configurar a dependência do `uv`

As dependências do projeto ficam em `pyproject.toml`; o `uv.lock` registra as versões resolvidas. Não é necessário criar nem editar um `uv.toml` para adicionar `drawpyo`: ele é um arquivo de configuração do `uv`, não o manifesto de dependências. As configurações equivalentes também podem ficar na seção `[tool.uv]` do `pyproject.toml`. Consulte a documentação oficial do `uv` sobre dependências e sobre arquivos de configuração.

Para manter `drawpyo` como dependência do repositório, execute na raiz:

```
uv add "drawpyo @ git+https://github.com/MerrimanInd/drawpyo.git@0.2.6"
```

O comando atualiza `pyproject.toml` e `uv.lock`. Como a biblioteca vem de uma release do GitHub, o `uv` registra o nome da dependência em `[project].dependencies` e sua origem em `[tool.uv.sources]`:

```
[project]
dependencies = [
    # Preserve as demais dependências já existentes no projeto.
    "drawpyo",
]
```

```
[tool.uv.sources]
drawpyo = { git = "https://github.com/MerrimanInd/drawpyo.git", rev = "0.2.6" }
```

Não substitua as dependências nem outras configurações já existentes pelo exemplo; acrescente a dependência e a origem Git indicadas. A referência `rev = "0.2.6"` fixa a release que disponibiliza o suporte a bibliotecas `mxlibrary` descrito neste tutorial. Se não quiser adicionar a biblioteca ao projeto, use a execução temporária abaixo; ela não altera `pyproject.toml` nem `uv.lock`.

```
uv run --no-project --with "drawpyo @
git+https://github.com/MerrimanInd/drawpyo.git@0.2.6" python
examples/diagrama_fluxo_imagens.py
```

Se adicionou a dependência ao projeto, execute-o assim:

```
uv run python examples/diagrama_fluxo_imagens.py
```

4.3.3 Conferir e publicar o resultado

Ao terminar, o terminal informa o caminho do arquivo criado:

```
docs/assets/diagrams/fluxo-imagens.drawio
```

O diagrama contém cinco imagens, cinco legendas e quatro conexões direcionais. Abra `docs/assets/diagrams/fluxo-imagens.drawio` no Draw.io para conferir ou ajustar o layout. Como os PNGs estão incorporados no arquivo, o diagrama continua abrindo mesmo se os arquivos de entrada forem movidos.

Para publicar uma versão estática na documentação, abra o `.drawio` no Draw.io e exporte como SVG para `docs/assets/diagrams/fluxo-imagens.svg`. Então, inclua a imagem na página:

```
![Fluxo do usuário até o armazenamento S3](assets/diagrams/fluxo-imagens.svg)
```

O arquivo `.drawio` também pode ser compartilhado como fonte editável:

```
[Baixar o diagrama editável](assets/diagrams/fluxo-imagens.drawio)
```

4.3.4 O que foi adicionado ao repositório

Para reproduzir o exemplo em outra cópia do repositório, mantenha os cinco PNGs nos caminhos listados e o script em `examples/diagrama_fluxo_imagens.py`. Instale `drawpyo` como dependência do projeto conforme o passo anterior ou use o comando temporário com `uv run --no-project`. O `.drawio` é gerado pelo script e pode ser mantido no repositório como fonte editável.

Neste exemplo, foram adicionados o script e o diagrama gerado, e a documentação desta seção foi atualizada. Os PNGs de entrada já estão em `docs/assets/`. A execução temporária usada para gerar o

exemplo não alterou `pyproject.toml` nem `uv.lock`; use `uv add` se quiser registrar a dependência permanentemente. A página `drawpyo.md` já está na navegação do `mkdocs.yml` e o diretório `docs/assets/` já é publicado pelo MkDocs, portanto não é preciso alterar a navegação nem a configuração do MkDocs. Inclua os cinco PNGs, o script e o `.drawio` no controle de versão para que outros usuários recebam todos os arquivos ao clonar o repositório.

5. Exportar para XML, SVG ou HTML

O arquivo `.drawio` gerado pelo `drawpyo` é a fonte editável do diagrama. Para criar formatos de publicação, instale o aplicativo desktop do Draw.io e use sua interface em **File** → **Export as** ou a linha de comando.

Exportar XML

O comando abaixo gera um XML sem compressão, útil para inspecionar o modelo ou integrar com outras ferramentas:

```
drawio --export --format xml --uncompressed `
  --output docs/assets/diagrams/aws-arquitetura.xml `
  docs/assets/diagrams/aws-arquitetura.drawio
```

O `.drawio` original já deve ser preservado como fonte editável; o XML exportado é uma representação adicional.

Exportar SVG

SVG é vetorial e costuma ser a melhor opção para incorporar diagramas estáticos em documentação. Use `--embed-diagram` para incluir também os dados do diagrama no SVG e `--embed-svg-images` para incorporar imagens SVG:

```
drawio --export --format svg --transparent `
  --embed-diagram --embed-svg-images `
  --output docs/assets/diagrams/aws-arquitetura.svg `
  docs/assets/diagrams/aws-arquitetura.drawio
```

Exportar HTML interativo

O exportador HTML inclui um visualizador com zoom e opções de interação:

```
drawio --export --format html `
  --output docs/assets/diagrams/aws-arquitetura.html `
  docs/assets/diagrams/aws-arquitetura.drawio
```

O HTML exportado carrega o visualizador JavaScript hospedado pelo Draw.io e precisa de conexão à internet. Verifique as opções antes de publicar diagramas internos ou confidenciais. No Windows, se `drawio` não estiver no `PATH`, substitua o comando pelo caminho do executável `draw.io.exe` instalado na máquina.

6. Incorporar o diagrama no MkDocs

Mantenha os arquivos que serão publicados dentro de `docs/`, por exemplo em `docs/assets/diagrams/`. O MkDocs copia os arquivos de imagem e diagramas para o site gerado.

Mostrar SVG estático

Em qualquer página Markdown, use um caminho relativo para a imagem:

```
![Arquitetura AWS: API Gateway, Lambda e DynamoDB](assets/diagrams/aws-arquitetura.svg)
```

O SVG continua nítido ao ampliar. Também é possível disponibilizar a fonte editável:

```
[Baixar a fonte editável do Draw.io](assets/diagrams/aws-arquitetura.drawio)
```

Incorporar o visualizador HTML

Para exibir o HTML interativo dentro de uma página, use um `iframe`. O exemplo abaixo é relativo a uma página Markdown na raiz de `docs/`; ajuste `../assets/` se a página estiver em uma subpasta:

```
<iframe
  src="../assets/diagrams/aws-arquitetura.html"
  title="Diagrama interativo da arquitetura AWS"
  width="100%"
  height="620"
  loading="lazy"
  style="border: 0;">
</iframe>
```

Ou forneça um link para abrir o visualizador em uma página separada:

```
[Abrir o diagrama AWS interativo](assets/diagrams/aws-arquitetura.html)
```

Para documentação pública, o SVG estático é mais simples e não depende de JavaScript externo. Use HTML interativo quando precisar de zoom ou das funções do visualizador e quando a conexão ao serviço Draw.io for permitida.

7. Resumo do fluxo de trabalho

1. Instale `drawpyo` 0.2.6 ou posterior para usar bibliotecas `mxlibrary` externas.
2. Crie `File`, `Page`, objetos e conexões em Python.
3. Use `register_mxlibrary` para bibliotecas Draw.io como `developer.xml`.
4. Baixe SVGs AWS oficiais e incorpore-os como `image` se precisar criar diagramas AWS pelo código.
5. Salve a fonte `.drawio` em `docs/assets/diagrams/`.
6. Exporte SVG para documentação estática ou HTML para um visualizador interativo.
7. Referencie o SVG ou o HTML na página MkDocs e teste com `uv run mkdocs serve`.

Referências

- Repositório e exemplos de `drawpyo`
- Documentação de bibliotecas de formas do `drawpyo`
- Biblioteca `developer.xml`
- Pacote oficial de ícones da AWS
- Formatos de exportação do Draw.io
- Exportar SVG
- Exportar HTML
- Opções da linha de comando do Draw.io Desktop

2026-11-13 2026-11-13

Exportar a documentação MkDocs para PDF

O MkDocs gera o site em HTML. Para produzir PDFs com o mesmo conteúdo, este projeto usa o plugin `mkdocs-exporter`, que renderiza as páginas com um navegador controlado pelo Playwright. O plugin pode criar um PDF por página e, opcionalmente, reunir as páginas em um único PDF.

Neste projeto, o plugin e o Playwright já estão declarados em `pyproject.toml`, e a exportação de PDFs individuais já está habilitada em `mkdocs.yml`. Para gerar um arquivo único com toda a documentação, também é necessário configurar o agregador, conforme o passo 3.

1. Como funciona a exportação

O processo é:

1. O MkDocs converte os arquivos Markdown em páginas HTML.
2. O `mkdocs-exporter` abre essas páginas em um navegador sem interface gráfica, usando Playwright.
3. O navegador imprime cada página em PDF.
4. Se o agregador estiver habilitado, o plugin reúne os PDFs das páginas em um documento.
5. Os arquivos resultantes ficam dentro da pasta de saída do site, normalmente `site/`.

O HTML continua sendo o site publicado; o PDF é um formato adicional para baixar ou distribuir.

2. Instalar as dependências

Em um projeto novo, instale o MkDocs, o plugin e o Playwright com o gerenciador de pacotes escolhido. Para um projeto que usa `uv`:

```
uv add mkdocs mkdocs-exporter playwright
```

Neste repositório essas dependências já estão listadas em `pyproject.toml`. Para sincronizar o ambiente:

```
uv sync
```

O Playwright precisa de um navegador instalado. No Windows, instale o Chromium:

```
uv run playwright install chromium
```

No Ubuntu usado por uma GitHub Action, instale o Chromium e as dependências do sistema:

```
- name: Instalar o Chromium do Playwright
```

```
    run: uv run playwright install --with-deps chromium
```

3. Habilitar PDFs individuais e o PDF completo

O plugin é configurado na lista `plugins` do `mkdocs.yml`. Mantenha os outros plugins do projeto e atualize a entrada existente de `exporter`; não crie uma segunda entrada `exporter`.

```
plugins:
  # Mantenha aqui search, mkdocstrings e os demais plugins já usados.
  - exporter:
      formats:
        pdf:
          enabled: true
          aggregator:
            enabled: true
            output: documentacao-completa.pdf
```

Com `enabled: true`, o plugin também gera PDFs individuais para as páginas incluídas. O bloco `aggregator` combina essas páginas em `site/documentacao-completa.pdf` após o build.

Se estiver usando capas individuais e quiser controlar quais aparecem na combinação, configure `covers`:

```
          aggregator:
            enabled: true
            output: documentacao-completa.pdf
            covers: limits
```

O modo `limits` mantém a capa inicial do primeiro documento e a capa final do último. Outras opções incluem `all`, `none`, `front`, `back` e `book`. Sem capas configuradas, deixe essa opção de fora.

Por padrão, todas as páginas são consideradas para exportação. Para excluir uma página específica, adicione front matter no início do arquivo Markdown:

```
---
pdf: false
---

# Página que só deve aparecer no site HTML
```

4. Personalizar o layout de impressão

Crie uma folha de estilos para PDF. Ela é separada do CSS usado na tela e pode definir tamanho do papel, margens, paginação e comportamento de imagens e blocos de código.

Por exemplo, crie `docs/stylesheets/pdf.scss`:

```
@page {
  size: A4;
  margin: 18mm 15mm 20mm;

  @bottom-right {
    content: counter(page);
    color: #526B7A;
  }
}

@media print {
  h1,
  h2,
  h3 {
    break-after: avoid;
  }

  img,
  svg {
    max-width: 100%;
    height: auto;
  }

  table,
  blockquote {
    break-inside: avoid;
  }

  pre {
    white-space: pre-wrap;
    overflow-wrap: anywhere;
  }
}
```

Registre o arquivo na configuração do exportador:

```
  - exporter:
      formats:
        pdf:
          enabled: true
          stylesheets:
            - docs/stylesheets/pdf.scss
          aggregator:
            enabled: true
            output: documentacao-completa.pdf
```

Se o YAML já tiver uma entrada de `exporter`, acrescente `stylesheets` dentro de `formats.pdf` nela. Teste o resultado com tabelas largas, imagens, diagramas e exemplos de código extensos; CSS de impressão pode exigir ajustes diferentes do CSS do site.

5. Adicionar uma capa opcional

O plugin aceita modelos HTML/Jinja para capas. Por exemplo, crie `docs/overrides/pdf-capa.html.j2`:

```
<section class="pdf-cover">
  <h1>Teste MkDocs</h1>
  <p>Exportação para PDF</p>
</section>
```

Configure o modelo no mesmo bloco do PDF:

```
  - exporter:
      formats:
        pdf:
          enabled: true
          covers:
            front: docs/overrides/pdf-capa.html.j2
          aggregator:
            enabled: true
            output: documentacao-completa.pdf
            covers: limits
```

Ajuste a aparência da capa na folha `pdf.scss`. Para uma documentação que não precisa de capa, omita `covers` e mantenha a configuração mais simples.

6. Gerar e conferir os arquivos localmente

Depois de instalar dependências e configurar o navegador, execute o build:

```
uv run mkdocs build --strict
```

Confira se o site e o PDF completo foram gerados:

```
Test-Path site\index.html
Test-Path site\documentacao-completa.pdf
Get-Item site\documentacao-completa.pdf | Select-Object Name, Length
```

Abra `site/documentacao-completa.pdf` em um leitor de PDF e confira:

- se as páginas aparecem na ordem da navegação;
- se caracteres acentuados, links, imagens e fontes estão corretos;
- se tabelas e blocos de código não ultrapassam as margens;
- se os diagramas estão legíveis e as quebras de página são adequadas.

Para documentos longos, prefira diagramas SVG ou imagens estáticas. Diagramas interativos, `iframe`s e conteúdo que depende de serviços externos podem não ser adequados para impressão.

7. Publicar o PDF com o GitHub Pages

O fluxo do GitHub Actions deve instalar o navegador, executar o build e publicar toda a pasta `site/`. Como o PDF agregado é criado dentro dessa pasta, ele acompanha o HTML no artefato do Pages.

O workflow deste projeto já possui passos para instalar Node.js e TypeDoc, sincronizar o `uv`, instalar Chromium, executar `mkdocs build --strict` e publicar `site/`. Portanto, depois de habilitar o agregador em `mkdocs.yml`, o PDF será incluído na publicação do Pages.

Além do PDF criado pelo agregador, este projeto gera pela Action um PDF completo com cabeçalho, faixa de rodapé e botão de download na documentação. Esse fluxo usa o conversor local e está documentado em Conversão PDF local; o arquivo é publicado em `assets/documentacao-local.pdf` e também fica disponível nos artefatos da execução da Action.

Um fluxo mínimo para um projeto semelhante é:

```yaml
- name: Instalar dependências Python
  run: uv sync

- name: Instalar o Chromium do Playwright
  run: uv run playwright install --with-deps chromium

- name: Gerar o site e os PDFs
  run: uv run mkdocs build --strict

- name: Publicar o site, incluindo o PDF
  uses: actions/upload-pages-artifact@v3
  with:
    path: site
```

Após a publicação, o PDF fica disponível na raiz do site, por exemplo, em `https://<usuario>.github.io/<repositorio>/documentacao-completa.pdf`. Ajuste o endereço ao domínio e ao caminho configurados para o Pages.

8. Problemas comuns

O build informa que não encontrou o navegador

Instale o Chromium na máquina local ou adicione o passo de instalação do Playwright ao workflow:

```
uv run playwright install chromium
```

O site é gerado, mas o PDF completo não aparece

Confirme se `aggregator.enabled` está dentro de `plugins → exporter → formats → pdf`, se o caminho `output` está correto e se o build terminou sem erros.

Uma página não aparece no PDF

Verifique se ela tem front matter `pdf: false`. Essa opção desativa a exportação da página.

Diagramas ou imagens ficam cortados

Use SVG ou PNG estático, limite a largura no CSS de impressão e evite depender de `iframe`s ou de JavaScript externo. Gere novamente o PDF após cada ajuste no estilo.

O build falha antes de começar a exportação

O exportador depende de um build MkDocs bem-sucedido. Neste projeto, páginas TypeScript também requerem Node.js e o comando `typedoc`; o workflow configura essas dependências antes de executar MkDocs. Em outros projetos, resolva primeiro os erros do build HTML.

Referências

- MkDocs Exporter: geração de PDFs
- MkDocs Exporter: agregar páginas em um PDF
- Playwright: instalação de navegadores
- MkDocs: configuração

2026-11-13 2026-11-13

Conversão local do site HTML para PDF

Esta página ensina a converter toda a documentação HTML já gerada pelo MkDocs em um PDF único, mantendo a ordem da navegação, o estilo claro, imagens e diagramas estáticos. O script inclui cabeçalho com o logo DataRain, rodapé em todas as páginas e marcadores no PDF para as seções.

O conversor é útil quando você quer processar localmente o site pronto ou gerar um PDF com marcadores próprios. Para a geração integrada diretamente ao `mkdocs build`, veja também Exportação para PDF, que documenta o `mkdocs-exporter` e seu agregador.

Baixar a documentação completa em PDF

1. Como a conversão funciona

O arquivo `examples/expotr_to_pdf.py`:

1. Lê a ordem e os títulos de `nav` no `mkdocs.yml`.
2. Acrescenta as demais páginas Markdown e HTML encontradas em `docs_dir` que não estejam listadas em `nav`.
3. Abre as páginas HTML do diretório `site/` em Chromium por um servidor HTTP local.
4. Usa as folhas de estilo, imagens, fontes e diagramas já publicados no site gerado.
5. Imprime cada página em PDF com o estilo claro e junta os resultados.
6. Repete o cabeçalho e o rodapé em todas as páginas físicas e cria marcadores hierárquicos.
7. Grava o PDF em `site/assets/documentacao-local.pdf`, caminho usado pelo botão de download.

As páginas listadas em `nav` aparecem primeiro e mantêm sua ordem e agrupamento. Outras páginas Markdown/HTML que o MkDocs tenha gerado são acrescentadas depois, em ordem de caminho. O servidor HTTP é local e temporário; ele serve os arquivos durante a conversão e é encerrado ao final. O HTML original não é modificado.

2. Instalar os requisitos

O script utiliza Playwright para renderizar as páginas, PyYAML para ler a navegação e `pypdf` para combinar os PDFs. As três bibliotecas estão declaradas diretamente nas dependências deste projeto; Playwright já era usado pelo `mkdocs-exporter`.

Sincronize as dependências e instale o Chromium:

```
uv sync
uv run playwright install chromium
```

Se estiver usando outro projeto, adicione as bibliotecas diretamente:

```
uv add playwright pypdf pyyaml
uv run playwright install chromium
```

3. Gerar primeiro o site HTML

O conversor não compila Markdown: ele lê HTML que já foi gerado. Na raiz do projeto, execute:

```
uv run mkdocs build
uv run python examples\expotr_to_pdf.py `
  --site-dir site `
  --output site\assets\documentacao-local.pdf
```

Confirme que o diretório e as páginas principais existem:

```
Test-Path site\index.html
Get-ChildItem site -Filter *.html -Recurse | Select-Object -First 10 FullName
```

Se o build falhar, corrija os erros do MkDocs antes de executar a conversão.

4. Definir o endereço do site para os links internos

Links externos são preservados como links clicáveis no PDF. Para que links entre páginas apontem ao site publicado, configure `site_url` no `mkdocs.yml` ou informe `base_url` ao executar o script.

Exemplo de configuração — substitua pelo endereço real do seu site:

```
site_url: https://<usuario>.github.io/<repositorio>/
```

Ou informe o endereço diretamente no comando. O `base_url` deve terminar com `/`:

```
uv run python examples\expotr_to_pdf.py `
  --site-dir site `
  --output site\assets\documentacao-local.pdf `
  --config mkdocs.yml `
  --base-url "https://<usuario>.github.io/<repositorio>/"
```

Se o endereço não for informado, o script avisa que os links internos podem não apontar ao site publicado.

5. Executar a função Python diretamente

O script também pode ser importado e usado em outro código Python:

```python
from pathlib import Path

from examples.expotr_to_pdf import export_to_pdf

arquivo_pdf = export_to_pdf(
    site_dir=Path("site"),
    output_pdf=Path("site/assets/documentacao-local.pdf"),
    mkdocs_config=Path("mkdocs.yml"),
    base_url="https://<usuario>.github.io/<repositorio>/",
)

print(f"PDF criado em: {arquivo_pdf}")
```

Os parâmetros são:

Parâmetro	Finalidade
`site_dir`	Pasta do site HTML gerado; por padrão, `site/`.
`output_pdf`	Destino do PDF; por padrão, `site/assets/documentacao-local.pdf`.
`mkdocs_config`	Caminho para o `mkdocs.yml`, usado para ler a navegação e `site_url`.
`base_url`	Endereço publicado usado para reescrever links internos; opcional se `site_url` estiver configurado.

O `nav` do `mkdocs.yml` precisa estar definido para o script manter a ordem e os agrupamentos. Todas as páginas Markdown/HTML geradas sob `docs_dir` são convertidas; páginas não listadas em `nav` entram no final. Arquivos estáticos, como CSS e imagens, são recursos das páginas e não viram páginas independentes.

6. Disponibilizar o botão de download

O botão no início desta página usa um link relativo para `assets/documentacao-local.pdf` e o atributo HTML `download`. O caminho relativo também funciona quando o GitHub Pages publica o site em uma subpasta do domínio. O PDF precisa existir nesse caminho no site publicado; por isso, o comando acima grava o arquivo em `site/assets/` depois do build.

No Chrome/Chromium, clicar no botão normalmente baixa o arquivo. O navegador ainda pode abri-lo se as preferências do usuário determinarem esse comportamento. O link direto continua disponível para baixar ou abrir o PDF.

7. Gerar e publicar com GitHub Actions

O workflow `.github/workflows/docs.yml` executa os passos nesta ordem:

1. Instala as dependências do projeto e o Chromium do Playwright.
2. Executa `mkdocs build --strict` para gerar o HTML e copiar os assets para `site/`.
3. Executa `examples/expotr_to_pdf.py` e grava o PDF em `site/assets/documentacao-local.pdf`.
4. Disponibiliza o PDF como artefato para download na execução da Action por 30 dias.
5. Envia toda a pasta `site/` como artefato do GitHub Pages; o HTML, as imagens e o PDF são publicados juntos.

Para configurar a publicação, abra o repositório no GitHub e vá a **Settings** → **Pages** → **Build and deployment** → **Source**. Selecione **GitHub Actions**. Depois, envie as alterações para a branch `main` ou execute o workflow manualmente pela aba **Actions**. Aguarde a execução terminar com sucesso e abra a URL publicada pelo job; o botão da página **Conversão PDF local** baixa o arquivo gerado naquela execução. Para baixar diretamente da execução, abra o resumo do job e selecione o artefato `documentacao-completa-pdf` em **Artifacts**.

O PDF não precisa ser regenerado nem commitado em cada alteração: a Action o gera durante o build e publica o resultado. Para testá-lo localmente, use os comandos do passo 3 e abra `site/assets/documentacao-local.pdf`. Se quiser versionar uma cópia no repositório, copie a saída local para `docs/assets/documentacao-local.pdf` e inclua essa alteração no commit.

8. Preservar imagens, diagramas, estilos e referências

O script serve a pasta inteira do site gerado durante a renderização. Por isso, caminhos relativos de CSS, imagens e diagramas estáticos continuam disponíveis ao navegador. Verifique se os arquivos usados estão dentro de `docs/` antes do build, para que sejam copiados para `site/`.

- **Imagens e SVGs:** são carregados pelo Chromium e incorporados visualmente nas páginas do PDF.
- **Diagramas:** prefira imagens SVG/PNG exportadas. Um `.drawio` pode continuar disponível como link para download, mas não é uma imagem renderizada.
- **Links externos:** continuam clicáveis no PDF.
- **Links entre páginas:** são reescritos para `base_url` ou para o `site_url` do projeto.
- **Referências e ordem:** os PDFs são combinados seguindo o `nav`; os marcadores refletem os grupos e títulos da navegação.

- **JavaScript e iframes:** conteúdo remoto ou interativo pode não estar pronto para impressão; prefira uma exportação estática do diagrama para o PDF.

O Chromium aplica o modo de impressão e as regras `@media print` das páginas. O script força o esquema claro do Material, usa as cores de identidade visual DataRain, oculta a navegação do site e expande a área principal.

9. Cabeçalho e rodapé DataRain

O conversor lê o logo configurado em `theme.logo` no `mkdocs.yml`. Neste projeto, o caminho aponta para `assets/logo.png`, que é copiado para `site/assets/logo.png` durante o build. O logo é incorporado ao PDF como imagem, então não depende de acesso externo.

O cabeçalho é repetido em todas as páginas físicas e contém o logo DataRain. Para o rodapé, o script incorpora `docs/assets/footer.png`, que o MkDocs copia para `site/assets/footer.png`. Mantenha esse arquivo nesse caminho. O esquema claro usa fundo branco, texto azul-marinho e o teal/laranja da paleta clara do tema:

```yaml
theme:
  name: material
  logo: assets/logo.png
  palette:
    - scheme: default
      primary: custom
      accent: custom
```

O cabeçalho/rodapé e a folha de estilos estão definidos em `examples/expotr_to_pdf.py`. O tamanho do papel está configurado como A4, com margens reservadas para que o conteúdo não fique sob o logo ou a faixa do rodapé.

10. Conferir o PDF

Depois da execução, valide a existência e o tamanho do arquivo:

```powershell
Test-Path site\assets\documentacao-local.pdf
Get-Item site\assets\documentacao-local.pdf | Select-Object Name, Length
```

Abra o PDF e confira:

1. A ordem das páginas e os marcadores correspondem ao `nav`.
2. Imagens e diagramas não foram cortados.
3. Tabelas e blocos de código respeitam as margens.
4. Links externos e internos abrem os destinos esperados.

5. A acentuação e a paginação estão corretas.

11. Problemas comuns

O script não encontra `site/` ou uma página HTML

Gere o site antes de converter e confirme que todos os documentos do `nav` foram incluídos no build:

```
uv run mkdocs build
```

O Chromium não está instalado

Instale o navegador usado pelo Playwright:

```
uv run playwright install chromium
```

O PDF não tem marcadores ou está fora de ordem

Confirme que `mkdocs.yml` tem uma seção `nav` válida e que as páginas listadas existem no HTML gerado. Páginas fora de `nav` aparecem após as páginas navegadas, ordenadas por caminho.

O cabeçalho não mostra o logo DataRain

Confirme que `theme.logo` aponta para uma imagem dentro do site gerado e que o arquivo existe, por exemplo, `site/assets/logo.png`. Verifique também se `docs/assets/footer.png` foi copiado para `site/assets/footer.png`.

Links internos não apontam para o site publicado

Configure `site_url` no `mkdocs.yml` ou passe o endereço completo ao parâmetro `--base-url`. Isso também é importante para sites GitHub Pages publicados em uma subpasta do domínio.

Diagramas interativos não aparecem

Converta-os para SVG ou PNG e inclua a imagem na documentação antes de executar `mkdocs build`. Iframes e recursos dependentes de serviços remotos não são garantidos no PDF.

Referências

- Playwright para Python: geração de PDF

- pypdf: combinar documentos
- MkDocs: configuração da navegação

2026-11-13 2026-11-13

Taskipy: automatizar tarefas do projeto

Taskipy é um executor de tarefas para projetos Python. Você cadastra comandos com nomes curtos no `pyproject.toml` e os executa de forma consistente no terminal e no GitHub Actions. Por exemplo, uma tarefa chamada `test` pode executar o pytest, e outra chamada `lint` pode executar o Ruff.

Neste tutorial, os comandos usam `uv` para instalar dependências e executar tarefas no ambiente do projeto.

1. Instalar Taskipy com uv

Abra o terminal na pasta que contém `pyproject.toml` e adicione Taskipy ao grupo de desenvolvimento:

```
uv add --dev taskipy
```

O comando atualiza `pyproject.toml` e `uv.lock`. Confirme a instalação:

```
uv run task --help
```

Taskipy é uma ferramenta de desenvolvimento, não uma dependência necessária para a aplicação funcionar em produção.

2. Criar uma tarefa no `pyproject.toml`

As tarefas são declaradas em `[tool.taskipy.tasks]`. Cada chave vira o nome de uma tarefa e o valor é o comando executado:

```
[tool.taskipy.tasks]
hello = "python -c \"print('Olá, Taskipy!')\""
```

Execute-a dentro do ambiente gerenciado pelo uv:

```
uv run task hello
```

Para listar as tarefas configuradas:

```
uv run task --list
```

Taskipy não substitui `uv run`: o uv seleciona o ambiente virtual e o Taskipy escolhe qual comando executar nesse ambiente.

3. Integrar Taskipy, Ruff e pytest

Instale as ferramentas usadas nos exemplos:

```
uv add --dev taskipy ruff pytest pytest-cov
```

Adicione tarefas para formatação, lint, testes e cobertura:

```
[tool.taskipy.tasks]
lint = "ruff check src tests"
format = "ruff format src tests"
format_check = "ruff format --check src tests"
test = "pytest -q"
coverage = "pytest --cov=meu_pacote --cov-report=term-missing --cov-report=html"
check = "task lint && task format_check && task test"
```

Substitua `src` e `tests` pelos diretórios usados no seu projeto. Troque `meu_pacote` pelo nome importável do pacote Python — normalmente com sublinhados, mesmo que o nome de distribuição use hífens.

Agora é possível executar cada etapa separadamente:

```
uv run task lint
uv run task format
uv run task test
uv run task coverage
```

Ou executar as verificações de uma só vez:

```
uv run task check
```

O comando `check` chama as tarefas na ordem indicada. Se uma etapa falhar, o processo termina com erro, o que também faz o GitHub Actions marcar a execução como falha.

4. Passar argumentos e criar tarefas compostas

Argumentos escritos depois do nome da tarefa são encaminhados ao comando configurado. Por exemplo, para selecionar um subconjunto de testes:

```
uv run task test -k cadastro
```

Uma tarefa composta pode chamar outras tarefas. No exemplo anterior, `check` combina lint, conferência de formatação e testes com `&&`; se uma etapa falhar, as seguintes não são executadas.

Taskipy também reconhece tarefas `pre_<nome>` e `post_<nome>` como ganchos antes e depois de uma tarefa. Use-os quando essa dependência fizer sentido para o fluxo; para verificações explícitas de CI, uma tarefa composta como `check` costuma ser mais fácil de entender.

5. Executar as tarefas no GitHub Actions

Crie `.github/workflows/qualidade.yml`. O workflow abaixo instala uv, sincroniza o ambiente usando o lockfile e executa as tarefas definidas no TOML:

```yaml
name: Qualidade do código

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  qualidade:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Instalar uv
        uses: astral-sh/setup-uv@v9
        with:
          enable-cache: true

      - name: Instalar dependências
        run: uv sync --locked

      - name: Executar verificações
        run: uv run task check
```

Versione `pyproject.toml` e `uv.lock`. A opção `--locked` faz o job falhar se os arquivos não estiverem sincronizados, em vez de alterar o lockfile durante o CI. Depois de enviar o workflow ao GitHub, acompanhe o resultado na aba **Actions** do repositório.

Referências oficiais

- Taskipy no GitHub: instalação, tarefas, argumentos e hooks
- uv em GitHub Actions
- Ruff: linter e formatador
- pytest: testes e cobertura

2026-11-13 2026-11-13

Ruff: analisar e formatar código Python

Ruff reúne duas ferramentas relacionadas, mas diferentes:

- **Linter**: analisa o código e aponta problemas, como imports não utilizados, erros comuns e divergências das regras selecionadas.
- **Formatador**: reescreve o código para aplicar um estilo consistente.

O linter é executado com `ruff check`; o formatador, com `ruff format`. Um não substitui o outro: formatar o código não corrige todos os problemas encontrados pelo linter.

1. Instalar Ruff com uv

No terminal, a partir da pasta que contém `pyproject.toml`, adicione Ruff ao grupo de desenvolvimento:

```
uv add --dev ruff
```

Confira a versão instalada:

```
uv run ruff --version
```

2. Executar o linter

Analise todos os arquivos Python nas pastas indicadas:

```
uv run ruff check src tests
```

Ruff informa o arquivo, a linha e o código da regra para cada problema. Para verificar uma regra específica, por exemplo imports não utilizados (`F401`):

```
uv run ruff check --select F401 src tests
```

Para tentar aplicar automaticamente as correções seguras:

```
uv run ruff check --fix src tests
```

Revise o diff depois de corrigir. Ruff pode oferecer correções não seguras com `--unsafe-fixes`; elas podem alterar o comportamento e devem ser avaliadas antes de aplicar:

```
uv run ruff check --fix --unsafe-fixes src tests
```

Para ver a explicação de uma regra:

```
uv run ruff rule F401
```

3. Formatar o código

Formate os arquivos Python:

```
uv run ruff format src tests
```

Para apenas verificar se já estão formatados — sem modificar os arquivos — use:

```
uv run ruff format --check src tests
```

Para visualizar as alterações propostas sem gravá-las:

```
uv run ruff format --diff src tests
```

`ruff format` grava as alterações; `ruff format --check` é apropriado para CI, pois retorna erro quando encontra arquivos que precisam ser formatados.

4. Configurar Ruff no `pyproject.toml`

Ruff lê sua configuração do mesmo `pyproject.toml` usado pelo uv:

```toml
[tool.ruff]
line-length = 88
target-version = "py312"
extend-exclude = ["build", "dist", "htmlcov", "site"]

[tool.ruff.lint]
select = ["E4", "E7", "E9", "F", "I"]

[tool.ruff.format]
quote-style = "double"
indent-style = "space"
```

O que essas opções fazem:

- **`line-length`**: comprimento de linha usado pelo linter e pelo formatador.
- **`target-version`**: versão mínima do Python considerada na análise; `py312` corresponde ao Python 3.12.
- **`extend-exclude`**: complementa a lista de arquivos e diretórios ignorados.

- **`lint.select`**: escolhe os grupos de regras; `E` vem do pycodestyle, `F` do Pyflakes e `I` verifica a organização dos imports.
- **`format.quote-style`** e **`format.indent-style`**: definem o estilo de aspas e indentação.

O Ruff também aceita ajustes como `lint.ignore` para ignorar regras e `lint.per-file-ignores` para exceções restritas a certos arquivos. Prefira corrigir o código ou limitar a exceção ao arquivo necessário em vez de desativar uma regra para o projeto inteiro.

5. Integrar Ruff ao Taskipy

Depois de instalar Taskipy com `uv add --dev taskipy`, configure tarefas no mesmo TOML:

```toml
[tool.taskipy.tasks]
lint = "ruff check src tests"
lint_fix = "ruff check --fix src tests"
format = "ruff format src tests"
format_check = "ruff format --check src tests"
```

Execute as tarefas sem precisar lembrar todos os argumentos:

```
uv run task lint
uv run task lint_fix
uv run task format
uv run task format_check
```

Para verificações automáticas, use `lint` e `format_check`, que apenas analisam o código. Reserve `lint_fix` e `format` para quando quiser gravar as correções localmente.

6. Executar as verificações no GitHub Actions

O GitHub Actions pode usar as mesmas tarefas sem duplicar os comandos do projeto. No arquivo `.github/workflows/qualidade.yml`, depois de configurar checkout e uv:

```yaml
      - name: Instalar dependências
        run: uv sync --locked

      - name: Verificar lint
        run: uv run task lint

      - name: Conferir formatação
        run: uv run task format_check
```

Se um comando retornar erro, o job falha e os detalhes ficam nos logs da execução em **GitHub → Actions**. Consulte Taskipy para o workflow completo.

Referências oficiais

- Documentação do Ruff
- Linter: regras e opções
- Formatador
- Configurações disponíveis
- Taskipy: executar Ruff com tarefas

2026-11-13 2026-11-13

pytest: escrever e executar testes

Um **teste** é um pequeno programa que verifica se outra parte do programa se comporta como esperado. Ele prepara uma entrada, executa uma função e compara o resultado com uma expectativa. Quando uma mudança quebra esse comportamento, o teste ajuda a detectar o problema antes de chegar ao usuário.

pytest é uma ferramenta que encontra e executa testes Python, apresenta erros com contexto e oferece recursos como fixtures e parametrização.

1. Instalar pytest e pytest-cov com uv

No terminal, a partir da pasta que contém `pyproject.toml`, instale o pytest e o plugin de cobertura como dependências de desenvolvimento:

```
uv add --dev pytest pytest-cov
```

O `pytest-cov` adiciona opções de cobertura ao pytest e usa a biblioteca Coverage.py para medir quais partes do código foram executadas.

2. Organizar o código e os testes

Uma estrutura comum para uma biblioteca Python é:

```
meu-projeto/
├── pyproject.toml
├── src/
│   └── calculadora/
│       ├── __init__.py
│       └── operacoes.py
└── tests/
    └── test_operacoes.py
```

Crie a função que será testada em `src/calculadora/operacoes.py`:

```python
def somar(a: float, b: float) -> float:
    return a + b


def dividir(a: float, b: float) -> float:
    if b == 0:
        raise ValueError("O divisor não pode ser zero.")
    return a / b
```

O pytest procura, por padrão, arquivos com nomes como `test_*.py` ou `*_test.py` e funções cujo nome começa com `test_`. Crie `tests/test_operacoes.py`:

```
import pytest

from calculadora.operacoes import dividir, somar


def test_somar_dois_numeros():
    assert somar(2, 3) == 5


@pytest.mark.parametrize(
    ("a", "b", "esperado"),
    [
        (6, 2, 3),
        (9, 3, 3),
        (-8, 2, -4),
    ],
)
def test_dividir(a, b, esperado):
    assert dividir(a, b) == esperado


def test_dividir_por_zero_informa_erro():
    with pytest.raises(ValueError, match="zero"):
        dividir(10, 0)
```

Neste exemplo, `assert` compara o resultado com o esperado; `pytest.raises` verifica se a função lança a exceção esperada; e `@pytest.mark.parametrize` executa o mesmo teste com várias combinações de dados.

3. Executar e selecionar testes

Execute todos os testes na raiz do projeto:

```
uv run pytest
```

Opções comuns para controlar a execução:

```
uv run pytest -v                          # saída detalhada
uv run pytest -q                          # saída resumida
uv run pytest tests/test_operacoes.py     # seleciona um arquivo
uv run pytest tests/test_operacoes.py::test_somar_dois_numeros
uv run pytest -k dividir                  # seleciona testes pelo nome
uv run pytest -x                          # para na primeira falha
uv run pytest --collect-only              # mostra testes encontrados sem executá-los
```

Uma execução bem-sucedida mostra um resumo semelhante a este:

```
============================= test session starts ==============================
collected 5 items

tests/test_operacoes.py .....                                            [100%]

============================== 5 passed in 0.08s ===============================
```

Se um teste falhar, o pytest mostra a asserção que falhou e os valores observados, facilitando o diagnóstico.

4. Usar fixtures e arquivos temporários

Uma **fixture** prepara um valor ou recurso que um teste precisa. pytest fornece, por exemplo, `tmp_path`, uma pasta temporária exclusiva para cada teste:

```python
def test_gravar_arquivo_temporario(tmp_path):
    arquivo = tmp_path / "resultado.txt"
    arquivo.write_text("processado", encoding="utf-8")

    assert arquivo.read_text(encoding="utf-8") == "processado"
```

Também é possível criar fixtures próprias. pytest as encontra pelo nome do argumento da função de teste:

```python
import pytest


@pytest.fixture
def lista_vazia():
    return []


def test_lista_vazia_comeca_sem_itens(lista_vazia):
    assert lista_vazia == []
```

Fixtures ajudam a evitar a repetição de preparação de dados. Cada teste deve continuar claro e, sempre que possível, independente dos demais.

5. Configurar pytest no `pyproject.toml`

É possível configurar a descoberta de testes e o relatório básico no mesmo arquivo do uv:

```toml
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra"
```

`testpaths` indica onde procurar testes quando você executa `uv run pytest` sem informar um caminho. `addopts = "-ra"` inclui no resumo informações sobre testes ignorados ou marcados como esperados

para falhar.

6. O que significa cobertura de testes

Cobertura estima quanto do código foi executado enquanto os testes rodavam. A cobertura de linhas indica quais linhas executáveis foram percorridas; a cobertura de ramificações também verifica caminhos alternativos, como os lados verdadeiro e falso de um `if`.

Uma cobertura alta **não prova** que o programa está correto: um teste pode executar uma linha sem verificar se o resultado está certo. Use a cobertura para encontrar código sem testes e complemente a métrica com casos que verifiquem comportamentos importantes, entradas inválidas e limites.

7. Gerar relatório no terminal e HTML

Execute os testes medindo o pacote `calculadora` e gere um relatório com as linhas não cobertas e outro em HTML:

```
uv run pytest \
  --cov=calculadora \
  --cov-report=term-missing \
  --cov-report=html
```

No Windows PowerShell, se preferir evitar a continuação de linha com barra invertida, execute em uma linha:

```
uv run pytest --cov=calculadora --cov-report=term-missing --cov-report=html
```

O pytest-cov imprime o resumo no terminal e cria `htmlcov/index.html`. Abra o relatório localmente pelo navegador ou sirva a pasta na raiz do projeto:

```
uv run python -m http.server 8000 --directory htmlcov
```

Depois acesse `http://localhost:8000` e interrompa o servidor com `Ctrl+C` quando terminar. O relatório HTML mostra os arquivos e destaca as linhas executadas e não executadas.

Outros relatórios úteis:

```
uv run pytest --cov=calculadora --cov-report=xml
uv run pytest --cov=calculadora --cov-branch --cov-report=term-missing
uv run pytest --cov=calculadora --cov-fail-under=80
```

- `--cov-report=xml` gera `coverage.xml`, formato frequentemente consumido por serviços de CI e cobertura.
- `--cov-branch` mede também as ramificações.

- `--cov-fail-under=80` faz a execução retornar erro se a cobertura ficar abaixo de 80%; o valor é um exemplo e deve ser definido pelo projeto.

Para obter os relatórios de terminal e HTML na mesma execução, informe os dois valores de `--cov-report`, como no primeiro exemplo.

8. Configurar cobertura e ignorar arquivos gerados

O `pyproject.toml` também pode conter opções de Coverage.py:

```
[tool.coverage.run]
branch = true
source = ["calculadora"]

[tool.coverage.report]
show_missing = true
fail_under = 80
```

`source` limita a medição ao pacote indicado; `branch` habilita a análise de ramificações; `show_missing` lista linhas não cobertas; e `fail_under` define um limite mínimo opcional.

Os arquivos de cobertura são resultados gerados. Não os versione no Git:

```
.coverage
.coverage.*
coverage.xml
htmlcov/
```

9. Executar testes e publicar o relatório no GitHub Actions

Para rodar testes e gerar relatórios em CI, adicione etapas como estas a um job que já tenha feito checkout do repositório e instalado uv:

```
      - name: Instalar dependências
        run: uv sync --locked

      - name: Executar testes e cobertura
        run: >-
          uv run pytest
          --cov=calculadora
          --cov-report=term-missing
          --cov-report=xml
          --cov-report=html

      - name: Disponibilizar relatório HTML
        if: always()
        uses: actions/upload-artifact@v4
        with:
```

```
          name: cobertura-html
          path: htmlcov/
          if-no-files-found: ignore
```

Depois da execução, abra **GitHub** → **Actions** → **execução do workflow** → **Artifacts** para baixar a pasta HTML e abrir `index.html`. O relatório do terminal e o resultado dos testes também ficam nos logs do job. Para executar os mesmos comandos como tarefas locais e no CI, consulte Taskipy.

Referências oficiais

- Documentação do pytest
- Como executar pytest
- Fixtures
- Parametrização de testes
- pytest-cov: relatórios de cobertura
- Taskipy: integrar testes em tarefas

2026-11-13 2026-11-13

Playwright: automação de navegadores

Playwright é uma biblioteca para controlar navegadores reais por código. Com Python, ela pode abrir páginas, preencher formulários, verificar conteúdo, tirar capturas de tela, baixar arquivos e gerar PDFs. Também é usada para testes automatizados de ponta a ponta (*end-to-end*).

Este projeto já declara o pacote Python `playwright` no `pyproject.toml`. O script Preview com Playwright o usa para abrir a documentação local no Chromium, e o plugin `mkdocs-exporter` o usa para gerar PDFs. Esta página apresenta os conceitos e exemplos para outros projetos.

1. Instalar a biblioteca e o Chromium

O Playwright é composto por duas partes: a biblioteca Python e os navegadores que ela controla. Instale ambas:

```
uv add playwright
uv run playwright install chromium
```

O primeiro comando registra o pacote no projeto; o segundo baixa o Chromium compatível com a versão instalada do Playwright. Em um projeto que usa `pip`, ative o ambiente virtual e execute:

```
python -m pip install playwright
playwright install chromium
```

Depois de atualizar a versão do Playwright, execute novamente `playwright install chromium`: cada versão da biblioteca espera versões específicas dos binários dos navegadores. Para instalar Firefox ou WebKit, troque `chromium` pelo navegador desejado.

Em Linux, especialmente em integração contínua, instale também as dependências de sistema necessárias:

```
uv run playwright install --with-deps chromium
```

`--with-deps` instala bibliotecas do sistema e é voltado a ambientes Linux; no Windows e macOS, normalmente basta instalar o navegador com `playwright install chromium`. Consulte a documentação oficial de instalação de navegadores para requisitos atualizados.

2. Iniciar e configurar o Chromium

Por padrão, `playwright.chromium.launch()` usa o Chromium baixado pelo próprio Playwright. `headless=True` executa sem janela — ideal para CI e geração de PDFs — e `headless=False` abre uma

janela visível para depuração:

```
from playwright.sync_api import sync_playwright

with sync_playwright() as playwright:
    browser = playwright.chromium.launch(headless=False)
    try:
        page = browser.new_page(
            viewport={"width": 1366, "height": 768},
            device_scale_factor=1,
        )
        page.goto("https://example.com", wait_until="domcontentloaded")
        print(page.title())
    finally:
        browser.close()
```

O navegador é o processo do Chromium; uma **página** corresponde a uma aba. Para manter cookies e configurações isolados entre sessões, crie um contexto e uma página dentro dele:

```
context = browser.new_context(
    viewport={"width": 1280, "height": 800},
    locale="pt-BR",
)
page = context.new_page()
```

Feche o contexto quando terminar (`context.close()`) e, por fim, feche o navegador. Se precisar usar o Google Chrome ou o Microsoft Edge instalado na máquina, configure `channel="chrome"` ou `channel="msedge"` em `launch()`. Para a maioria dos casos, prefira o Chromium gerenciado pelo Playwright; não é necessário apontar manualmente para o executável.

```
browser = playwright.chromium.launch(channel="chrome", headless=True)
```

3. Navegar, interagir e validar uma página

Use localizadores semânticos, como `get_by_role` e `get_by_label`. As ações e asserções do Playwright aguardam automaticamente que o elemento esteja pronto, evitando pausas arbitrárias com `time.sleep()`:

```
from playwright.sync_api import expect, sync_playwright

with sync_playwright() as playwright:
    browser = playwright.chromium.launch()
    page = browser.new_page()

    page.goto("https://playwright.dev/python/docs/intro")
    expect(
        page.get_by_role("heading", name="Installation")
    ).to_be_visible()

    browser.close()
```

Para interagir com formulários, localize o campo pelo rótulo e o botão pela função e pelo nome:

```
page.get_by_label("E-mail").fill("pessoa@example.com")
page.get_by_label("Senha").fill("senha-de-teste")
page.get_by_role("button", name="Entrar").click()
expect(page.get_by_text("Bem-vindo")).to_be_visible()
```

Prefira `get_by_role`, `get_by_label` e outros localizadores voltados ao usuário. Eles tornam os testes mais próximos da forma como uma pessoa usa o site e menos dependentes da estrutura interna do HTML.

4. Capturas de tela e depuração visual

Uma captura pode incluir a área visível ou a página inteira. Crie a pasta de destino antes de salvar:

```
from pathlib import Path
from playwright.sync_api import sync_playwright

Path("capturas").mkdir(exist_ok=True)

with sync_playwright() as playwright:
    browser = playwright.chromium.launch()
    page = browser.new_page()
    page.goto("https://example.com")
    page.screenshot(path="capturas/pagina-inteira.png", full_page=True)
    browser.close()
```

Para abrir o navegador e observar as ações durante o desenvolvimento, use `headless=False`. Também é possível desacelerar as ações:

```
browser = playwright.chromium.launch(headless=False, slow_mo=250)
```

`slow_mo` é útil para depuração interativa; remova-o em testes normais e em CI. Para gerar código de automação interativamente a partir dos cliques no navegador, use o gravador do Playwright:

```
uv run playwright codegen https://example.com
```

5. Downloads e emulação de dispositivos

Para downloads, aguarde o evento antes de clicar; assim o script captura o arquivo iniciado pela ação:

```
from pathlib import Path

Path("downloads").mkdir(exist_ok=True)

with page.expect_download() as download_event:
    page.get_by_role("link", name="Baixar relatório").click()
```

```
download = download_event.value
download.save_as(Path("downloads") / download.suggested_filename)
```

Playwright também pode configurar um contexto com parâmetros de um dispositivo móvel:

```
device = playwright.devices["iPhone 13"]
context = browser.new_context(**device)
page = context.new_page()
page.goto("https://example.com")
```

Isso emula características como viewport, user agent e densidade de pixels. Use-o para verificar layouts responsivos; a emulação não substitui testes em aparelhos físicos quando o comportamento do hardware é relevante.

6. Gerar PDF com Chromium

O método `page.pdf()` imprime a página atual para PDF e está disponível no Chromium em modo headless. Por padrão, ele usa estilos de impressão; `print_background=True` inclui cores e imagens de fundo:

```
from pathlib import Path
from playwright.sync_api import sync_playwright

Path("exportados").mkdir(exist_ok=True)

with sync_playwright() as playwright:
    browser = playwright.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://example.com", wait_until="load")
    page.evaluate("document.fonts.ready")
    page.pdf(
        path="exportados/pagina.pdf",
        format="A4",
        print_background=True,
        prefer_css_page_size=True,
    )
    browser.close()
```

`prefer_css_page_size=True` permite que regras CSS `@page` determinem o tamanho e as margens quando existirem. Se precisar da aparência de tela em vez de estilos de impressão, configure `page.emulate_media(media="screen")` antes de chamar `page.pdf()`.

PDF da documentação MkDocs neste projeto

Não é necessário criar um script próprio para exportar cada página deste site. O `mkdocs-exporter` já está configurado no `mkdocs.yml` e usa o Playwright para renderizar as páginas com o tema e então gerar PDFs.

Após instalar o Chromium e executar o build, o plugin disponibiliza a exportação nas páginas. Veja a configuração e o uso na página Plugins.

Para configurar a mesma integração em outro projeto MkDocs, habilite o plugin no `mkdocs.yml` e gere o site:

```
plugins:
  - search
  - exporter:
      formats:
        pdf:
          enabled: true
```

```
uv run playwright install chromium
uv run mkdocs build
```

O Playwright também abre o site local no script Preview com Playwright. Para executar o fluxo completo de preview, incluindo build e servidor HTTP, use `uv run docs_preview.py`; para exportar páginas, use o recurso do `mkdocs-exporter`.

7. API síncrona e assíncrona

Os exemplos acima usam a API síncrona (`playwright.sync_api`), que costuma ser mais simples para scripts pequenos. Em aplicações assíncronas, use a API `async_api` e aguarde cada operação:

```
import asyncio
from playwright.async_api import async_playwright


async def main():
    async with async_playwright() as playwright:
        browser = await playwright.chromium.launch()
        page = await browser.new_page()
        await page.goto("https://example.com")
        print(await page.title())
        await browser.close()


asyncio.run(main())
```

Não misture chamadas síncronas e assíncronas na mesma sessão do Playwright.

8. Boas práticas e diagnóstico

- **Instale o navegador compatível:** se o erro indicar que o executável não foi encontrado, execute `uv run playwright install chromium`. Repita após atualizar o pacote.

- **Em CI, use headless:** não abra uma janela com `headless=False` em um runner sem ambiente gráfico.
- **Aguarde condições, não tempos fixos:** use `expect(locator).to_be_visible()`, `locator.wait_for()` ou eventos específicos em vez de `sleep`.
- **Feche os recursos:** feche páginas/contextos e o navegador mesmo quando ocorrer uma exceção, usando `try/finally` ou o fluxo de limpeza adequado ao script.
- **Investigue falhas com rastreamento:** grave traces do contexto e abra-os com o visualizador do Playwright:

```python
from playwright.sync_api import sync_playwright

with sync_playwright() as playwright:
    browser = playwright.chromium.launch()
    context = browser.new_context()
    context.tracing.start(screenshots=True, snapshots=True, sources=True)
    page = context.new_page()
    page.goto("https://example.com")
    context.tracing.stop(path="trace.zip")
    browser.close()
```

```
uv run playwright show-trace trace.zip
```

O trace pode incluir capturas de tela, snapshots do DOM e fontes; evite compartilhá-lo se contiver dados privados de usuários.

Referências oficiais

- Playwright para Python
- Instalação de navegadores
- Localizadores
- Capturas de tela
- Gerar PDFs com `page.pdf()`

2026-11-13 2026-11-13

Preview com Playwright

1. Introdução

O arquivo `docs_preview.py`, na raiz do projeto, é um script auxiliar que automatiza todo o fluxo de "gerar e abrir" a documentação. Sem ele, conferir o resultado final do site exigiria pelo menos três passos manuais: rodar `uv run mkdocs build` (ou `mkdocs serve`), iniciar algum servidor HTTP apontando para a pasta `site/` (lembrando que abrir `site/index.html` direto via `file://` quebra a busca do tema Material, como já explicado na página Exemplos do notebook) e, por fim, abrir o navegador manualmente no endereço correto.

O `docs_preview.py` atualiza a página `docs/referencias.md` a partir das dependências declaradas, builda a documentação, inicia um servidor HTTP local servindo a pasta `site/` e abre a página inicial em uma janela real do navegador Chromium, já maximizada, esperando apenas que você aperte Enter no terminal quando quiser fechar tudo. É um script pensado para quem quer apenas *ver o resultado*, sem se preocupar com os comandos individuais de build e servidor.

Para conhecer outras funções do Playwright — interações com páginas, capturas de tela, downloads e geração de PDFs — consulte o capítulo Playwright.

2. Instalação do Playwright

Antes de rodar o script, é preciso instalar duas coisas em sequência: o pacote Python do Playwright e o navegador que ele controla. São etapas diferentes, e as duas são obrigatórias.

```
uv add playwright
```

Esse comando adiciona o pacote Python `playwright` às dependências do projeto, registrando-o no `pyproject.toml` e instalando-o no ambiente gerenciado pelo `uv` — da mesma forma que `uv add mkdocs` faz com o MkDocs. O pacote `playwright` instalado aqui é apenas a **biblioteca de automação**: o código Python que sabe como conversar com um navegador e controlá-lo (abrir páginas, clicar, esperar, fechar etc.).

```
uv run playwright install chromium
```

Esse segundo comando baixa e instala o **binário do navegador Chromium** que o Playwright vai controlar. Essa é uma etapa separada e obrigatória porque o pacote Python instalado no passo anterior não inclui o navegador em si — ele só contém a API que sabe *como* controlar um navegador, não o navegador executável. O Playwright suporta vários motores de navegador (Chromium, Firefox, WebKit), e cada um precisa ser baixado individualmente através do comando `playwright install <nome-do-navegador>`.

Sem rodar esse segundo comando, o pacote Python estaria instalado, mas qualquer tentativa de abrir um navegador falharia, porque não haveria nenhum executável de Chromium disponível para ser lançado.

Na prática, você normalmente só precisa rodar os dois comandos acima uma vez por máquina. O próprio `docs_preview.py` também verifica e garante a instalação do Chromium automaticamente a cada execução (através da função `ensure_playwright_browser_installed`, descrita mais abaixo), então mesmo que você esqueça o segundo comando, o script tenta corrigir isso por conta própria na primeira vez que for executado.

3. O que é o Playwright e por que ele é usado aqui

O Playwright é uma biblioteca de automação de navegador, originalmente criada para testes end-to-end de aplicações web. Com ela, é possível escrever código que abre um navegador real (Chromium, Firefox ou WebKit), navega até uma URL, interage com elementos da página (clica em botões, preenche formulários, espera elementos aparecerem) e inspeciona o resultado — tudo de forma programática, sem precisar de um ser humano clicando manualmente.

No contexto do `docs_preview.py`, o Playwright não é usado para testar nada: ele é usado apenas pela sua capacidade de **abrir uma janela de navegador real e controlada por código**. Em vez de instruir o usuário a abrir o navegador manualmente e digitar um endereço, o script usa o Playwright para:

- lançar uma instância do Chromium já maximizada (`--start-maximized`);
- navegar automaticamente até o endereço do servidor local que acabou de ser iniciado;
- manter essa janela aberta até o usuário decidir encerrar (pressionando Enter no terminal), encerrando o navegador de forma limpa em seguida.

Ou seja, o Playwright é a peça que transforma o fluxo de "build + preview" em um único comando de terminal: ao rodar o script, a documentação é gerada e a janela do navegador já aparece na tela, sem nenhuma ação manual adicional.

4. Documentação das funções via mkdocstrings

O `docs_preview.py` define sete funções públicas, incluindo `generate_references_markdown`, que gera a página de referências; as funções têm docstrings em pt-BR no estilo Google. Em vez de repetir essas docstrings manualmente nesta página — o que criaria o mesmo risco de desatualização discutido na página Exemplos de código —, usamos uma única diretiva do `mkdocstrings` apontando para o **módulo inteiro**, em vez de uma diretiva por função:

```
::: docs_preview
```

Como o `docs_preview.py` está localizado na raiz do projeto — exatamente o mesmo lugar onde está o `teste.py`, usado como exemplo na página de Exemplos de código —, a configuração `paths: [.]` já

existente em `plugins.mkdocstrings.handlers.python` no `mkdocs.yml` também resolve o módulo `docs_preview` sem precisar de nenhuma configuração extra. Essa configuração diz ao handler Python do `mkdocstrings` para procurar módulos a partir da raiz do projeto, e tanto `teste.py` quanto `docs_preview.py` moram nesse mesmo diretório.

Ao apontar a diretiva para o módulo inteiro (`docs_preview`), em vez de uma função específica (como `docs_preview.main`), o `mkdocstrings` importa o módulo e renderiza automaticamente a documentação de **todas** as funções públicas definidas nele, uma após a outra, cada uma com sua assinatura e suas seções `Args` / `Returns` / `Raises` extraídas da docstring real. Abaixo, a diretiva é inserida de verdade na página, para que o `mkdocstrings` a processe:

`docs_preview`

Atualiza referências, builda a documentação e abre o resultado no navegador.

Uso

uv run docs_preview.py uv run docs_preview.py --generate-references

Este script é um "script uv" autocontido (metadados PEP 723 no topo do arquivo): ao executá-lo com `uv run docs_preview.py`, o `uv` cria automaticamente um ambiente efêmero só com o `playwright` instalado — não é necessário rodar `uv add` / `pip install` manualmente antes.

Passo a passo

1. Gera docs/referencias.md a partir das dependências declaradas no pyproject.toml ou requirements.txt.
2. Executa o `mkdocs build` do `.venv` do projeto diretamente (grupo de dependências `docs`: mkdocs, mkdocs-material, mkdocstrings), gerando o site estático em `site/`. Chama o binário direto em vez de `uv run mkdocs build` para não depender da sincronização do `uv`, que trava com frequência quando o `.venv` está dentro de uma pasta sincronizada pelo OneDrive.
3. Garante que o navegador Chromium do Playwright esteja instalado (`playwright install chromium`), baixando-o na primeira execução caso ainda não exista.
4. Serve o diretório `site/` localmente via `http.server` (necessário para a busca do Material funcionar — abrir `index.html` direto como `file://` bloqueia o carregamento do índice de busca).
5. Abre a página inicial em um navegador Chromium controlado pelo Playwright e mantém a janela aberta até você apertar Enter no terminal.

`ensure_playwright_browser_installed()`

Garante que o navegador Chromium do Playwright esteja instalado.

Baixa o Chromium na primeira execução (pode levar alguns minutos); em execuções seguintes, o Playwright detecta que já está instalado e a chamada retorna quase imediatamente.

Returns:

Name	Type	Description
`None`	`None`	Esta função não retorna valor.

Source code in `docs_preview.py`

```
337 def ensure_playwright_browser_installed() -> None:
338     """Garante que o navegador Chromium do Playwright esteja instalado.
339 
340     Baixa o Chromium na primeira execução (pode levar alguns minutos);
341     em execuções seguintes, o Playwright detecta que já está instalado
342     e a chamada retorna quase imediatamente.
343 
344     Returns:
345         None: Esta função não retorna valor.
346     """
347     print("==> Verificando instalação do Chromium (Playwright)...")
348 
349     subprocess.run([sys.executable, "-m", "playwright", "install", "chromium"],
    check=False)
```

`find_free_port()`

Encontra uma porta TCP livre na máquina local.

Returns:

Name	Type	Description
`int`	`int`	Número de uma porta TCP livre em `127.0.0.1`.

Source code in `docs_preview.py`

```python
def find_free_port() -> int:
    """Encontra uma porta TCP livre na máquina local.

    Returns:
        int: Número de uma porta TCP livre em ``127.0.0.1``.
    """
    with closing(socket.socket(socket.AF_INET, socket.SOCK_STREAM)) as sock:
        sock.bind(("127.0.0.1", 0))
        return sock.getsockname()[1]
```

`generate_references_markdown(project_root=PROJECT_ROOT)`

Gera `docs/referencias.md` com referências extraídas das dependências.

Lê as dependências de `pyproject.toml` (incluindo origens Git definidas em `tool.uv.sources`) ou, se não houver uma lista de dependências, usa `requirements.txt`. Para dependências GitHub com revisão fixada, cria um link para o commit; para pacotes sem URL direta, cria uma referência à página correspondente no PyPI. A página gerada também documenta a instalação com `uv sync` e inclui esta função via `mkdocstrings`.

A função roda antes do build em `run_mkdocs_build` e no workflow de publicação. Também pode ser chamada isoladamente com `uv run docs_preview.py --generate-references`.

Parameters:

Name	Type	Description	Default
`project_root`	`Path`	Raiz do projeto que contém os arquivos de dependências.	`PROJECT_ROOT`

Returns:

Name	Type	Description
`Path`	`Path`	Caminho do arquivo Markdown gerado.

Raises:

Type	Description
`FileNotFoundError`	Se não houver um `requirements.txt` de fallback.
`ValueError`	Se as dependências estiverem vazias ou malformadas.

Source code in `docs_preview.py`

```python
def generate_references_markdown(project_root: Path = PROJECT_ROOT) -> Path:
    """Gera ``docs/referencias.md`` com referências extraídas das dependências.

    Lê as dependências de ``pyproject.toml`` (incluindo origens Git definidas
    em ``tool.uv.sources``) ou, se não houver uma lista de dependências,
    usa ``requirements.txt``. Para dependências GitHub com revisão fixada, cria
    um link para o commit; para pacotes sem URL direta, cria uma referência à
    página correspondente no PyPI. A página gerada também documenta a
    instalação com ``uv sync`` e inclui esta função via ``mkdocstrings``.

    A função roda antes do build em ``run_mkdocs_build`` e no workflow de
    publicação. Também pode ser chamada isoladamente com
    ``uv run docs_preview.py --generate-references``.

    Args:
        project_root: Raiz do projeto que contém os arquivos de dependências.

    Returns:
        Path: Caminho do arquivo Markdown gerado.

    Raises:
        FileNotFoundError: Se não houver um ``requirements.txt`` de fallback.
        ValueError: Se as dependências estiverem vazias ou malformadas.
    """
    project_root = project_root.resolve()
    dependencies = _dependency_references(project_root)
    rows = [
        "| Pacote | Requisito declarado | Referência |",
        "|---|---|---|",
    ]

    for name, specifier, reference in dependencies:
        safe_name = name.replace("|", r"\|")
        safe_specifier = (specifier or "sem versão mínima").replace("|", r"\|")
        safe_reference = reference.replace("|", r"\|")
        rows.append(
            f"| `{safe_name}` | `{safe_specifier}` | [Abrir referência]({safe_reference})
|"
        )

    content = "\n".join(
        [
            "---",
            "pdf: false",
            "---",
            "",
            "# Referências de pacotes Python",
            "",
            "Esta página é gerada automaticamente a partir das dependências do projeto.",
            "Para atualizar a lista manualmente, execute `uv run docs_preview.py --
generate-references`.",
            "",
            *rows,
            "",
            "## Instalar os pacotes",
            "",
```

```
            "Na raiz do repositório, instale as dependências e as versões registradas no
lockfile com:",
            "",
            "```bash",
            "uv sync",
            "````",
            "",
            "## Função de extração",
            "",
            "A função abaixo lê o `pyproject.toml` ou o `requirements.txt` e gera esta
página:",
            "",
            "::: docs_preview.generate_references_markdown",
            "",
        ]
    )

    output_path = project_root / "docs" / "referencias.md"
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_text(content, encoding="utf-8")
    return output_path
```

`main()`

Executa o preview ou gera apenas a página de referências.

Por padrão, atualiza as referências, executa o build do MkDocs e abre o site no Chromium. Com `--generate-references`, atualiza `docs/referencias.md` e encerra sem abrir o navegador.

Returns:

Name	Type	Description
`None`	`None`	Esta função não retorna valor.

Source code in `docs_preview.py`

```python
def main() -> None:
    """Executa o preview ou gera apenas a página de referências.

    Por padrão, atualiza as referências, executa o build do MkDocs e abre o
    site no Chromium. Com ``--generate-references``, atualiza
    ``docs/referencias.md`` e encerra sem abrir o navegador.

    Returns:
        None: Esta função não retorna valor.
    """
    parser = argparse.ArgumentParser(description="Builda ou atualiza a documentação.")
    parser.add_argument(
        "--generate-references",
        action="store_true",
        help="Gera docs/referencias.md a partir das dependências do projeto.",
    )
    args = parser.parse_args()

    if args.generate_references:
        print(f"==> Referências atualizadas em: {generate_references_markdown()}")
        return

    run_mkdocs_build()

    port = find_free_port()
    server = start_local_server(SITE_DIR, port)

    url = f"http://127.0.0.1:{port}/"

    print(f"==> Servindo {SITE_DIR} em {url}")

    ensure_playwright_browser_installed()

    try:
        open_in_browser(url)
    finally:
        print("==> Encerrando servidor local...")
        server.shutdown()
```

`open_in_browser(url)`

Abre `url` em um navegador Chromium controlado pelo Playwright.

Mantém o navegador aberto até o usuário apertar Enter no terminal, depois encerra o navegador e o contexto do Playwright de forma limpa.

Parameters:

Name	Type	Description	Default
`url`	`str`	URL a ser aberta (ex.: `"http://127.0.0.1:8000/"`).	*required*

Returns:

Name	Type	Description
`None`	`None`	Esta função não retorna valor.

Source code in `docs_preview.py`

```python
def open_in_browser(url: str) -> None:
    """Abre `url` em um navegador Chromium controlado pelo Playwright.

    Mantém o navegador aberto até o usuário apertar Enter no terminal,
    depois encerra o navegador e o contexto do Playwright de forma
    limpa.

    Args:
        url (str): URL a ser aberta (ex.: ``"http://127.0.0.1:8000/"``).

    Returns:
        None: Esta função não retorna valor.
    """
    from playwright.sync_api import sync_playwright

    print(f"==> Abrindo {url} no navegador (Playwright)...")

    with sync_playwright() as playwright:

        browser = playwright.chromium.launch(
            headless=False,
            args=["--start-maximized"],
        )

        # `no_viewport=True` faz a pagina usar o tamanho real da janela do
        # navegador (maximizada) em vez do viewport fixo padrao do
        # Playwright (1280x720) - sem isso, a janela fica maximizada mas a
        # pagina continua renderizada como se a tela fosse pequena.
        context = browser.new_context(no_viewport=True)
        page = context.new_page()
        page.goto(url)

        input("\nNavegador aberto. Pressione Enter aqui para fechar...\n")

        browser.close()
```

`run_mkdocs_build()`

Atualiza as referências e executa o `mkdocs build` do projeto.

Chama o executável `mkdocs` dentro de `.venv` diretamente, em vez de `uv run mkdocs build`: o `uv run` faz uma sincronizacao do ambiente antes de rodar, e isso trava com frequencia quando o `.venv` fica dentro de uma pasta sincronizada pelo OneDrive (o OneDrive bloqueia arquivos do `.venv` durante a sincronizacao, e o `uv` recebe "Acesso negado" ao tentar reinstalar pacotes). Antes de executar o binário do MkDocs, gera `docs/referencias.md` a partir de `pyproject.toml` ou `requirements.txt`.

Raises:

Type	Description
`SystemExit`	Se o `mkdocs` nao existir em `.venv` (rode `uv sync --group docs` primeiro), ou se o build falhar (codigo de saida diferente de 0), ou se o diretorio `site/` nao existir apos o build.
`FileNotFoundError`	Se a página de referências não puder ser gerada porque não há um arquivo de dependências de fallback.
`ValueError`	Se as dependências estiverem ausentes ou malformadas.

Returns:

Name	Type	Description
`None`	`None`	Esta funcao nao retorna valor.

Source code in `docs_preview.py`

```
253  def run_mkdocs_build() -> None:
254      """Atualiza as referências e executa o ``mkdocs build`` do projeto.
255
256      Chama o executável ``mkdocs`` dentro de ``.venv`` diretamente, em vez
257      de ``uv run mkdocs build``: o ``uv run`` faz uma sincronizacao do
258      ambiente antes de rodar, e isso trava com frequencia quando o `.venv`
259      fica dentro de uma pasta sincronizada pelo OneDrive (o OneDrive
260      bloqueia arquivos do `.venv` durante a sincronizacao, e o `uv`
261      recebe "Acesso negado" ao tentar reinstalar pacotes). Antes de executar
262      o binário do MkDocs, gera ``docs/referencias.md`` a partir de
263      ``pyproject.toml`` ou ``requirements.txt``.
264
265      Raises:
266          SystemExit: Se o `mkdocs` nao existir em `.venv` (rode
267              `uv sync --group docs` primeiro), ou se o build falhar
268              (codigo de saida diferente de 0), ou se o diretorio `site/`
269              nao existir apos o build.
270          FileNotFoundError: Se a página de referências não puder ser gerada
271              porque não há um arquivo de dependências de fallback.
272          ValueError: Se as dependências estiverem ausentes ou malformadas.
273
274      Returns:
275          None: Esta funcao nao retorna valor.
276      """
277      mkdocs_bin = (
278          PROJECT_ROOT / ".venv" / "Scripts" / "mkdocs.exe"
279          if sys.platform == "win32"
280          else PROJECT_ROOT / ".venv" / "bin" / "mkdocs"
281      )
282
283      if not mkdocs_bin.exists():
284          print(f"mkdocs nao encontrado em: {mkdocs_bin}")
285          print("Rode `uv sync --group docs` na raiz do projeto primeiro.")
286          raise SystemExit(1)
287
288      references_path = generate_references_markdown()
289      print(f"==> Referências atualizadas em: {references_path}")
290      print(f"==> Executando: {mkdocs_bin} build")
291
292      result = subprocess.run([str(mkdocs_bin), "build"], cwd=str(PROJECT_ROOT))
293
294      if result.returncode != 0:
295          print("Build do MkDocs falhou. Abortando.")
296          raise SystemExit(result.returncode)
297
298      if not SITE_DIR.exists():
299          print(f"Diretório do site não encontrado em: {SITE_DIR}")
300          raise SystemExit(1)
301
302      print(f"==> Build concluído em: {SITE_DIR}")
```

`start_local_server(directory, port)`

Inicia um servidor HTTP local servindo `directory`, em background.

Parameters:

Name	Type	Description	Default
`directory`	`Path`	Diretório a ser servido (ex.: `site/`).	*required*
`port`	`int`	Porta TCP onde o servidor vai escutar.	*required*

Returns:

Name	Type	Description
`ThreadingHTTPServer`	`ThreadingHTTPServer`	Instância do servidor já rodando em uma thread separada (`daemon=True`, não bloqueia o encerramento do script).

Source code in `docs_preview.py`

```python
316 def start_local_server(directory: Path, port: int) -> ThreadingHTTPServer:
317     """Inicia um servidor HTTP local servindo `directory`, em background.
318
319     Args:
320         directory (Path): Diretório a ser servido (ex.: `site/`).
321         port (int): Porta TCP onde o servidor vai escutar.
322
323     Returns:
324         ThreadingHTTPServer: Instância do servidor já rodando em uma
325             thread separada (`daemon=True`, não bloqueia o encerramento
326             do script).
327     """
328     handler = partial(SimpleHTTPRequestHandler, directory=str(directory))
329     server = ThreadingHTTPServer(("127.0.0.1", port), handler)
330
331     thread = threading.Thread(target=server.serve_forever, daemon=True)
332     thread.start()
333
334     return server
```

Função `generate_references_markdown`

Essa função mantém `docs/referencias.md` sincronizado com as dependências do projeto. Ela prioriza a lista `project.dependencies` do `pyproject.toml`, usa `tool.uv.sources` para apontar dependências Git à origem/revisão configurada e gera links do PyPI para os demais pacotes. Se o TOML não declarar dependências, usa `requirements.txt`; arquivos incluídos com `-r` também são lidos.

O arquivo gerado contém a tabela de dependências, o comando `uv sync` e uma diretiva do `mkdocstrings` para exibir a própria implementação da função. No preview local, ela roda antes do build. No GitHub Actions, o workflow a executa explicitamente antes de `mkdocs build`, para que a versão publicada inclua sempre a lista atual.

Para gerar apenas a página, sem iniciar o preview:

```
uv run docs_preview.py --generate-references
```

5. Como executar o script

Com o Playwright e o Chromium instalados, basta rodar:

```
uv run docs_preview.py
```

O script atualiza `docs/referencias.md` antes de iniciar o build. A execução isolada da função, sem abrir o navegador, está descrita na subseção anterior.

O `docs_preview.py` é um "script uv" autocontido: ele tem metadados no formato PEP 723 escritos em um comentário no topo do próprio arquivo, declarando que precisa do pacote `playwright>=1.47`. Graças a esses metadados, o `uv run` consegue criar automaticamente um ambiente Python efêmero contendo apenas essa dependência e executar o script dentro dele — sem precisar que você rode `uv add` ou `uv sync` manualmente antes. Isso é diferente de `teste.py`, por exemplo, que depende das dependências já declaradas no `pyproject.toml` do projeto.

Ao ser executado, o script segue o seguinte fluxo, na ordem:

1. **Geração de referências**: lê `pyproject.toml` ou, como alternativa, `requirements.txt` e atualiza `docs/referencias.md`.
2. **Build do MkDocs**: chama o executável `mkdocs` dentro do `.venv` do projeto diretamente (em vez de `uv run mkdocs build`), gerando o site estático atualizado em `site/`.
3. **Servidor HTTP local**: encontra uma porta TCP livre na máquina e inicia um servidor HTTP em segundo plano, servindo o conteúdo de `site/` em `http://127.0.0.1:<porta>/`.
4. **Abertura no Chromium**: garante que o Chromium do Playwright esteja instalado e, em seguida, abre a URL do servidor local em uma janela do Chromium maximizada, controlada pelo Playwright.
5. **Espera por Enter**: o terminal exibe a mensagem "Navegador aberto. Pressione Enter aqui para fechar..." e o script fica parado nesse ponto até você apertar Enter; só então o navegador é fechado e o servidor HTTP local é encerrado.

2026-11-13 2026-11-11

Dockerfile e outras linguagens

1. Por que não existe um handler de mkdocstrings para estas linguagens

Nas páginas Exemplos de código e Preview com Playwright, o plugin `mkdocstrings` gerou documentação automaticamente a partir do código-fonte real do projeto, usando diretivas como `teste.calcular_media`, `calcular-media` e `docs_preview`. Isso só é possível porque o `mkdocstrings` depende de um **handler** específico para cada linguagem — um componente capaz de ler o código-fonte, entender sua estrutura (módulos, funções, parâmetros, tipos) e extrair uma documentação estruturada a partir de comentários no formato de docstring:

- o handler `python` sabe interpretar docstrings no estilo Google (seções `Args`, `Returns`, `Raises`), como visto em `teste.calcular_media`;
- o handler `typescript` sabe interpretar comentários JSDoc (`@param`, `@returns`, `@throws`), delegando a leitura real do código para o TypeDoc, como visto em `calcular-media`.

Dockerfile, JavaScript puro (sem um toolchain de documentação configurado), YAML, Terraform, Shell e SQL **não possuem um handler oficial equivalente disponível neste projeto**. Não é que essas linguagens sejam "piores" ou que seus arquivos não possam ser documentados — é que não existe, aqui, um parser de docstring estruturado e configurado para extrair `Args` / `Returns` / `Raises` (ou equivalente) a partir de um `Dockerfile`, de um script Shell ou de uma consulta SQL, por exemplo.

Diante dessa limitação, há três caminhos possíveis para mostrar esse código na documentação:

1. **Copiar e colar o conteúdo manualmente** em um bloco de código estático — a abordagem mais simples, mas que corre o risco de ficar desatualizada em relação ao arquivo real, exatamente como já discutido para o bloco estático de `calcular_media` na página de Exemplos de código.
2. **Tentar usar o `mkdocstrings`** — não é uma opção aqui, já que não há handler configurado para nenhuma dessas linguagens neste projeto.
3. **Incorporar o conteúdo real do arquivo diretamente na página**, através da extensão `pymdownx.snippets` — a abordagem adotada nesta página, e explicada em detalhe a seguir.

2. Ferramentas complementares: Hadolint e Docker Buildx

Antes de entrar na técnica de incorporação, vale mencionar duas ferramentas comumente usadas junto de um `Dockerfile`, embora nenhuma das duas tenha sido instalada ou executada como parte desta documentação — elas são citadas aqui apenas como prática recomendada, para quem quiser ir além da leitura do código.

aws PARTNER Advanced Tier Services
Immersion Day · Solution Provider · DevOps Services Competency · Migration Services Competency
Nonprofit Services Competency · Public Sector Solution Provider · Well-Architected Partner Program
Solution Spark Public Sector Partners · Amazon EC2 for Windows Server Delivery
dataRain FAR BEYOND THE CLOUD

- **Hadolint**: um linter dedicado a arquivos `Dockerfile`. Ele analisa o arquivo em busca de problemas comuns de boas práticas, como ordenação ineficiente de camadas (o que invalida o cache de build sem necessidade), uso de `apt-get install` sem `--no-install-recommends`, falta de pinagem de versão em imagens base (`FROM python:3.12-slim` em vez de `FROM python` sem tag) e outras recomendações equivalentes às de um linter de código tradicional, só que voltadas especificamente para instruções Docker. Ele pode ser executado sem instalação local, via container, apontando o próprio `Dockerfile` para dentro dele pela entrada padrão:

```
docker run --rm -i hadolint/hadolint < Dockerfile
```

- **Docker Buildx**: a extensão de build do Docker usada para efetivamente construir a imagem a partir do `Dockerfile`, com suporte a multi-estágio, cache avançado e múltiplas plataformas. Rodar um build é a forma mais direta de confirmar que o `Dockerfile` realmente funciona, não apenas que ele está bem escrito:

```
docker buildx build .
```

Nenhum desses dois comandos foi executado neste ambiente — não há Docker instalado nesta máquina de documentação —, então o `Dockerfile` desta página foi revisado manualmente, mas não teve seu lint ou seu build validados automaticamente por essas ferramentas.

3. A técnica de incorporação: `pymdownx.snippets`

A extensão `pymdownx.snippets`, parte do pacote `pymdown-extensions`, adiciona uma sintaxe especial que pode ser usada **dentro** de um bloco de código Markdown: a linha `--8<-- "caminho/arquivo"`. Quando o MkDocs processa a página, ele substitui essa linha pelo conteúdo real do arquivo apontado, mantendo o restante do bloco de código (as crases de abertura/fechamento e a linguagem indicada para destaque de sintaxe) exatamente como foi escrito. Por exemplo:

```
```dockerfile
# ------------------------------------------------------------------------
# Estágio 1: builder — instala as dependências do projeto em um ambiente
# isolado, usando o gerenciador uv e os arquivos de lock já existentes
# (pyproject.toml / uv.lock), antes de copiar o código-fonte da aplicação.
# ------------------------------------------------------------------------
FROM python:3.12-slim AS builder

# Copia o binário oficial do uv (já compilado) para dentro da imagem, em vez
# de instalá-lo via pip, mantendo o estágio de build mais rápido e leve.
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

# Define o diretório de trabalho onde o projeto será montado dentro da imagem.
WORKDIR /app

# Copia primeiro apenas os arquivos de manifesto de dependências, para que o
# cache de camadas do Docker seja reaproveitado quando só o código mudar.
COPY pyproject.toml uv.lock ./
```
```

```
# Instala exatamente as dependências travadas em uv.lock (--frozen), sem os
# grupos de desenvolvimento (--no-dev), mantendo o ambiente final mínimo.
RUN uv sync --frozen --no-dev

# Copia o código-fonte real da aplicação (main.py) e do exemplo documentado
# via mkdocstrings (teste.py) para dentro da imagem builder.
COPY main.py teste.py ./

# ---------------------------------------------------------------------------
# Estágio 2: runtime — imagem final, menor, contendo apenas o ambiente
# virtual já resolvido pelo uv e o código necessário para executar a
# aplicação, sem as ferramentas de build usadas no estágio anterior.
# ---------------------------------------------------------------------------
FROM python:3.12-slim AS runtime

# Define o mesmo diretório de trabalho usado no estágio builder.
WORKDIR /app

# Copia o ambiente virtual já criado pelo uv no estágio builder, evitando
# reinstalar dependências e reduzindo o tamanho e o tempo de build da
# imagem final.
COPY --from=builder /app/.venv /app/.venv

# Copia o código-fonte já validado no estágio builder.
COPY --from=builder /app/main.py /app/teste.py ./

# Adiciona o ambiente virtual ao PATH, para que o comando `python` resolva
# para o interpretador e os pacotes instalados pelo uv, sem precisar
# ativar manualmente o ambiente virtual.
ENV PATH="/app/.venv/bin:$PATH"

# Define o comando padrão executado quando o container é iniciado. Este
# projeto não possui um pacote `src/` de aplicação (a pasta `src/`
# existente é o pacote de exemplo TypeScript de outra parte da
# documentação, não a aplicação), então o entrypoint real é o script
# `main.py` na raiz do projeto.
CMD ["python", "main.py"]

```
```

O resultado, depois de processado pelo `mkdocs build`, é um bloco de código `dockerfile` contendo o conteúdo inteiro do arquivo `Dockerfile` da raiz do projeto, com o destaque de sintaxe apropriado para essa linguagem.

Isso só funciona sem configuração adicional por página porque a extensão `pymdownx.snippets` está configurada com a opção `base_path: .` na seção `markdown_extensions` do `mkdocs.yml`:

```
markdown_extensions:
  - pymdownx.snippets:
      base_path: .
```

O `base_path: .` diz ao `pymdownx.snippets` que todo caminho usado em `--8<-- "..."` deve ser resolvido **a partir da raiz do projeto** — o mesmo diretório em que os comandos `mkdocs build` / `mkdocs`
```

`serve` são executados. É por isso que `--8<-- "Dockerfile"` encontra o `Dockerfile` da raiz, e `--8<-- "examples/exemplo.js"` encontra o arquivo dentro da pasta `examples/`, sem que cada página precise declarar seu próprio caminho base.

Duas características importantes dessa técnica:

- **O conteúdo é sempre lido do arquivo real a cada build.** Diferente de copiar e colar manualmente o texto de um arquivo dentro da página Markdown, a linha `--8<-- "caminho/arquivo"` não armazena nenhum conteúdo — ela é apenas uma referência. Toda vez que `mkdocs build` (ou `mkdocs serve`) roda, o MkDocs lê o arquivo naquele exato momento e insere seu conteúdo atual no lugar da referência. Se o `Dockerfile` for alterado no futuro, a próxima vez que o site for reconstruído já mostrará a versão atualizada, sem que ninguém precise editar esta página.
- **É uma incorporação de texto bruto, não uma extração estruturada.** Essa é a diferença principal entre `pymdownx.snippets` e o `mkdocstrings`, usado nas páginas anteriores. O `mkdocstrings` interpreta o código-fonte (importa o módulo Python ou usa o TypeDoc para processar o código-fonte TypeScript) e gera um HTML estruturado com cabeçalho, assinatura e seções separadas para `Args` / `Returns` / `Raises`, a partir de uma docstring. O `pymdownx.snippets`, por outro lado, não entende o conteúdo do arquivo: ele simplesmente copia todo o texto do arquivo, caractere a caractere, para dentro do bloco de código, sem extrair assinatura, parâmetros ou qualquer outra informação estruturada — é o mesmo texto que estaria lá se alguém tivesse colado manualmente, só que sempre sincronizado com o arquivo real.

4. Dockerfile

O `Dockerfile` deste projeto, na raiz, usa **dois estágios de build**:

- `builder`: instala as dependências do projeto com `uv sync --frozen --no-dev`, a partir de `pyproject.toml` e `uv.lock`, e copia o código-fonte da aplicação.
- `runtime`: copia apenas o ambiente virtual já resolvido e o código-fonte do estágio `builder`, sem as ferramentas usadas durante a instalação, resultando em uma imagem final mais leve. O comando padrão do container é `CMD ["python", "main.py"]`.

Abaixo, o conteúdo do `Dockerfile` é incorporado de verdade na página, através de `--8<-- "Dockerfile"`:

```
# ------------------------------------------------------------------------
# Estágio 1: builder — instala as dependências do projeto em um ambiente
# isolado, usando o gerenciador uv e os arquivos de lock já existentes
# (pyproject.toml / uv.lock), antes de copiar o código-fonte da aplicação.
# ------------------------------------------------------------------------
FROM python:3.12-slim AS builder

# Copia o binário oficial do uv (já compilado) para dentro da imagem, em vez
# de instalá-lo via pip, mantendo o estágio de build mais rápido e leve.
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

# Define o diretório de trabalho onde o projeto será montado dentro da imagem.
```

```
WORKDIR /app

# Copia primeiro apenas os arquivos de manifesto de dependências, para que o
# cache de camadas do Docker seja reaproveitado quando só o código mudar.
COPY pyproject.toml uv.lock ./

# Instala exatamente as dependências travadas em uv.lock (--frozen), sem os
# grupos de desenvolvimento (--no-dev), mantendo o ambiente final mínimo.
RUN uv sync --frozen --no-dev

# Copia o código-fonte real da aplicação (main.py) e do exemplo documentado
# via mkdocstrings (teste.py) para dentro da imagem builder.
COPY main.py teste.py ./

# ---------------------------------------------------------------------------
# Estágio 2: runtime — imagem final, menor, contendo apenas o ambiente
# virtual já resolvido pelo uv e o código necessário para executar a
# aplicação, sem as ferramentas de build usadas no estágio anterior.
# ---------------------------------------------------------------------------
FROM python:3.12-slim AS runtime

# Define o mesmo diretório de trabalho usado no estágio builder.
WORKDIR /app

# Copia o ambiente virtual já criado pelo uv no estágio builder, evitando
# reinstalar dependências e reduzindo o tamanho e o tempo de build da
# imagem final.
COPY --from=builder /app/.venv /app/.venv

# Copia o código-fonte já validado no estágio builder.
COPY --from=builder /app/main.py /app/teste.py ./

# Adiciona o ambiente virtual ao PATH, para que o comando `python` resolva
# para o interpretador e os pacotes instalados pelo uv, sem precisar
# ativar manualmente o ambiente virtual.
ENV PATH="/app/.venv/bin:$PATH"

# Define o comando padrão executado quando o container é iniciado. Este
# projeto não possui um pacote `src/` de aplicação (a pasta `src/`
# existente é o pacote de exemplo TypeScript de outra parte da
# documentação, não a aplicação), então o entrypoint real é o script
# `main.py` na raiz do projeto.
CMD ["python", "main.py"]
```

5. Outras linguagens

Esta seção reúne os cinco arquivos de exemplo do diretório `examples/`, cada um representando uma linguagem sem handler de mkdocstrings configurado neste projeto. Cada subseção incorpora o conteúdo real do arquivo correspondente.

JavaScript

O arquivo `examples/exemplo.js` define uma função `calcularMedia` em JavaScript puro — sem TypeScript, sem anotações de tipo e sem um toolchain de documentação como o TypeDoc, por isso não há como gerar sua documentação via `mkdocstrings` (diferente do exemplo equivalente em TypeScript, visto na página de Exemplos de código).

```javascript
// exemplo.js — função simples em JavaScript, incorporada na documentação
// via pymdownx.snippets (não existe handler de mkdocstrings para JS puro).

// Calcula a média aritmética de uma lista de números.
function calcularMedia(valores) {
  if (valores.length === 0) {
    throw new Error("A lista não pode estar vazia.");
  }
  return valores.reduce((soma, valor) => soma + valor, 0) / valores.length;
}

module.exports = { calcularMedia };
```

YAML

O arquivo `examples/exemplo.yaml` é um pequeno arquivo de configuração, no mesmo formato usado pelo próprio `mkdocs.yml` deste projeto.

```yaml
# exemplo.yaml — configuração simples incorporada na documentação.
app:
  name: teste-mkdocks
  version: "1.0.0"
  logging:
    level: info
    format: json
```

Terraform

O arquivo `examples/exemplo.tf` declara um recurso Terraform simples. Embora a extensão do arquivo seja `.tf`, o destaque de sintaxe usado abaixo é `hcl` — a sigla de *HashiCorp Configuration Language*, a linguagem de configuração em que o Terraform é escrito (não existe uma linguagem de destaque chamada literalmente "terraform" nos realces de sintaxe usados pelo MkDocs).

```hcl
# exemplo.tf — recurso Terraform simples incorporado na documentação.
resource "local_file" "exemplo" {
  filename = "${path.module}/exemplo-output.txt"
  content  = "Gerado pelo exemplo Terraform da documentação."
}
```

Shell

O arquivo `examples/exemplo.sh` é um script Shell simples, que builda e publica uma imagem Docker — unindo conceitualmente esta página à seção sobre o `Dockerfile` acima.

```
#!/usr/bin/env bash
# exemplo.sh — script simples de build/deploy incorporado na documentação.
set -euo pipefail

echo "Construindo a imagem Docker..."
docker build -t teste-mkdocks:latest .

echo "Publicando a imagem no registry..."
docker push teste-mkdocks:latest
```

SQL

O arquivo `examples/exemplo.sql` contém uma criação de tabela e uma consulta simples, calculando a média de uma coluna numérica — o mesmo tipo de cálculo da função `calcular_media` / `calcularMedia`, só que expresso em SQL.

```
-- exemplo.sql — criação de tabela e consulta simples incorporada na
-- documentação.
CREATE TABLE medias (
    id SERIAL PRIMARY KEY,
    valor NUMERIC NOT NULL
);

SELECT AVG(valor) AS media
FROM medias;
```

6. Resumo

Linguagem	Arquivo	Destaque de sintaxe
Dockerfile	`Dockerfile`	`dockerfile`
JavaScript	`examples/exemplo.js`	`javascript`
YAML	`examples/exemplo.yaml`	`yaml`
Terraform	`examples/exemplo.tf`	`hcl`
Shell	`examples/exemplo.sh`	`bash`

Linguagem	Arquivo	Destaque de sintaxe
SQL	`examples/exemplo.sql`	`sql`

Em todos os seis casos, o conteúdo exibido acima nesta página não foi copiado e colado: cada bloco de código contém uma linha `--8<-- "caminho/arquivo"`, resolvida pela extensão `pymdownx.snippets` a cada build a partir da raiz do projeto (`base_path: .`). Isso garante que, se qualquer um desses seis arquivos for alterado, esta página mostrará automaticamente a versão mais recente na próxima vez que `mkdocs build` ou `mkdocs serve` for executado.

2026-11-11 2026-11-11

Extensões pymdownx

Cada demonstração segue o mesmo fluxo: primeiro há um bloco expansível com o **Markdown que o autor deve escrever**; em seguida, a **saída renderizada pelo MkDocs** aparece na página. A saída é mostrada como o leitor a verá no site, não como HTML bruto.

1. Introdução

O pacote `pymdown-extensions` já está instalado neste projeto (ele foi adicionado ao `pyproject.toml` na Tarefa 9.1, para suportar a incorporação de arquivos via `pymdownx.snippets`, usada na página Dockerfile e outras linguagens). Esse pacote reúne um conjunto bem maior de extensões Markdown do que apenas `snippets` — a maioria delas pensada especificamente para documentação técnica e exemplos de código, que é exatamente o caso deste tutorial.

Esta página cataloga as **8 extensões `pymdownx` mais relevantes para código** usadas (ou disponíveis) neste projeto. Quatro delas já apareceram em páginas anteriores e são apenas revisadas rapidamente aqui, com um link de volta para onde foram detalhadas. As outras quatro — `pymdownx.tabbed`, `pymdownx.tasklist`, `pymdownx.details` e `pymdownx.emoji` — são o foco desta página, cada uma com um exemplo real, processado pelo MkDocs no momento do build (não apenas um trecho ilustrativo de Markdown).

2. As 8 extensões

Extensão	Finalidade
`pymdownx.highlight`	Destaque de sintaxe (*syntax highlighting*) em blocos de código.
`pymdownx.superfences`	Blocos de código avançados, com suporte a aninhamento e a cercas customizadas (como os diagramas Mermaid).
`pymdownx.snippets`	Incorporação do conteúdo real de arquivos do projeto dentro de blocos de código, via `--8<--`.
`pymdownx.tabbed`	Abas para alternar entre conteúdos relacionados (ex.: o mesmo exemplo em linguagens diferentes).
`pymdownx.inlinehilite`	Destaque de sintaxe para código citado em linha, entre crase simples (`` `assim` ``).

Extensão	Finalidade
`pymdownx.tasklist`	Checklists técnicos, com caixas de marcação renderizadas de verdade.
`pymdownx.details`	Seções expansíveis/colapsáveis.
`pymdownx.emoji`	Emojis e ícones.

As quatro primeiras linhas da tabela já foram apresentadas em detalhe em páginas anteriores deste tutorial:

- `pymdownx.highlight` e `pymdownx.superfences` — introduzidas na página Configuração do mkdocs.yml e usadas em praticamente todo bloco de código deste tutorial, incluindo o diagrama Mermaid da página Diagramas e arquitetura, que depende do `custom_fences` do `superfences`.
- `pymdownx.snippets` — detalhada em profundidade na página Dockerfile e outras linguagens, incluindo a sintaxe `--8<--` e a opção `base_path: .`.
- `pymdownx.inlinehilite` — habilitada no `mkdocs.yml` desde a Tarefa 1.1 e usada implicitamente sempre que um nome de arquivo, comando ou identificador é citado entre crase simples ao longo deste tutorial (por exemplo, `mkdocs.yml` ou `calcular_media` em qualquer página).

Os exemplos abaixo mostram, para cada extensão, primeiro o Markdown fonte e depois o resultado visível no HTML.

Bloco de código com `pymdownx.highlight` e `pymdownx.superfences`

Declare a linguagem depois das crases para habilitar o destaque de sintaxe:

Markdown fonte do bloco de código

````
```python
print("Olá, mundo!")
```
````

Saída renderizada no HTML do MkDocs:

```python
print("Olá, mundo!")
```

O `superfences` também permite aninhar blocos de código dentro de uma caixa:

Markdown fonte de código dentro de uma caixa

````
!!! note "Exemplo"
    ```python
    print("Olá, mundo!")
    ```
````

Saída renderizada no HTML do MkDocs:

Exemplo

```
print("Olá, mundo!")
```

Incorporar um arquivo com `pymdownx.snippets`

Use `--8<--` dentro do bloco de código para incluir o arquivo existente. Neste projeto, `base_path: .` resolve o caminho a partir da raiz:

Markdown fonte da incorporação

````
```javascript
// exemplo.js — função simples em JavaScript, incorporada na documentação
// via pymdownx.snippets (não existe handler de mkdocstrings para JS puro).

// Calcula a média aritmética de uma lista de números.
function calcularMedia(valores) {
  if (valores.length === 0) {
    throw new Error("A lista não pode estar vazia.");
  }
  return valores.reduce((soma, valor) => soma + valor, 0) / valores.length;
}

module.exports = { calcularMedia };

```
````

Saída renderizada no HTML do MkDocs: o conteúdo atual de `examples/exemplo.js` é inserido durante o build:

```javascript
// exemplo.js — função simples em JavaScript, incorporada na documentação
// via pymdownx.snippets (não existe handler de mkdocstrings para JS puro).

// Calcula a média aritmética de uma lista de números.
function calcularMedia(valores) {
  if (valores.length === 0) {
```

```
    throw new Error("A lista não pode estar vazia.");
  }
  return valores.reduce((soma, valor) => soma + valor, 0) / valores.length;
}

module.exports = { calcularMedia };
```

As demais demonstrações da página seguem a mesma sequência: Markdown fonte seguido do resultado processado pelo MkDocs.

Código inline colorido com `pymdownx.inlinehilite`

Para destacar a sintaxe de um pequeno trecho de código dentro de uma frase, use `#!` seguido da linguagem dentro das crases.

Markdown fonte

```
Código Python em uma frase: `#!python print("Olá, mundo!")`
```

Saída renderizada no HTML do MkDocs:

Código Python em uma frase: `print("Olá, mundo!")`

3. Abas com `pymdownx.tabbed`

A extensão `pymdownx.tabbed` transforma uma sequência de blocos marcados com `=== "Título"` em abas clicáveis, todas visíveis na mesma área da página — o leitor alterna entre elas sem precisar rolar a página para comparar o conteúdo. Ela está configurada no `mkdocs.yml` com a opção `alternate_style: true`, que faz as abas usarem o estilo visual do tema Material (uma faixa de botões acima do conteúdo) em vez do estilo padrão da extensão.

A sintaxe é:

- cada aba começa com `=== "Título da aba"`, na margem esquerda da linha;
- o conteúdo da aba vem na(s) linha(s) seguinte(s), indentado com **4 espaços** — exatamente como um item de lista Markdown aninhado. Esse conteúdo pode ser qualquer Markdown válido, incluindo um bloco de código inteiro, como nos exemplos abaixo;
- uma linha em branco (sem indentação) separa uma aba da próxima.

O Markdown fonte abaixo cria três abas com a mesma instrução em linguagens diferentes:

Markdown fonte das abas

````
=== "Python"

    ```python
    print("Hello World")
    ```

=== "JavaScript"

    ```javascript
    console.log("Hello World");
    ```

=== "PowerShell"

    ```powershell
    Write-Host "Hello World"
    ```
````

Saída renderizada no HTML do MkDocs:

Python

```python
print("Hello World")
```

JavaScript

```javascript
console.log("Hello World");
```

PowerShell

```powershell
Write-Host "Hello World"
```

4. Checklist com `pymdownx.tasklist`

A extensão `pymdownx.tasklist` transforma itens de lista Markdown escritos como `- [x]` ou `- [ ]` em caixas de marcação **de verdade** — não apenas o texto literal `[x]` / `[ ]`, mas um elemento HTML renderizado visualmente como uma caixa de seleção. `[x]` marca o item como concluído (caixa marcada) e `[ ]` (um espaço entre os colchetes) marca o item como pendente (caixa vazia).

Escreva os itens como uma lista comum, usando `[x]` para concluído e `[ ]` para pendente:

Markdown fonte do checklist

```
- [x] Instalar o MkDocs e o tema Material
- [x] Configurar o `mkdocs.yml`
- [ ] Publicar o site no GitHub Pages
```

Saída renderizada no HTML do MkDocs:

- [x] Instalar o MkDocs e o tema Material
- [x] Configurar o `mkdocs.yml`
- [ ] Publicar o site no GitHub Pages

No `mkdocs.yml` deste projeto, `pymdownx.tasklist` está configurada com a opção `custom_checkbox: true`. Sem essa opção, a extensão renderizaria um `<input type="checkbox" disabled>` HTML comum — a caixa de marcação padrão do navegador, com a aparência do sistema operacional do leitor. Com `custom_checkbox: true`, o tema Material substitui essa caixa padrão pelos ícones customizados do próprio tema (um quadrado com marca de visto para itens concluídos, um quadrado vazio para pendentes), mantendo a aparência visual consistente com o restante do site, em vez de depender do estilo nativo de cada navegador/sistema operacional.

5. Seção expansível com `pymdownx.details`

A extensão `pymdownx.details` cria uma seção de conteúdo que começa **recolhida** (escondida) e só é revelada quando o leitor clica em seu título — equivalente ao elemento HTML `<details>` / `<summary>`, mas escrito em uma sintaxe parecida com a dos admonitions (as caixas de aviso `!!! nota` / `!!! atenção` já usadas em páginas anteriores deste tutorial).

A diferença de sintaxe entre as duas é um único caractere no marcador de abertura:

- `!!! tipo "Título"` (três pontos de exclamação) — usado pela extensão `admonition` — cria uma caixa de aviso **sempre visível**, com todo o conteúdo exposto desde o carregamento da página.
- `??? tipo "Título"` (três pontos de interrogação) — usado pela extensão `pymdownx.details` — cria a mesma caixa visualmente, mas **colapsada por padrão**: o leitor só vê o título até clicar nele para expandir o conteúdo.

O Markdown fonte da seção recolhível é:

Markdown fonte da seção recolhível

```
??? note "Clique para expandir"
    Este conteúdo começa escondido e só aparece quando o leitor clica no título.
```

Saída renderizada no HTML do MkDocs: a caixa abaixo começa recolhida; clique no título para expandi-la.

Clique para expandir

Este conteúdo começa escondido e só aparece quando o leitor clica no título.

Esse comportamento é útil para conteúdo complementar ou opcional — como um trecho de log de erro, uma explicação mais longa para quem quiser se aprofundar, ou uma seção "veja também" — que não precisa competir por atenção com o conteúdo principal da página, mas que continua disponível a um clique de distância.

6. Emoji com `pymdownx.emoji`

A extensão `pymdownx.emoji` substitui códigos no formato `:nome-do-emoji:` pelo ícone correspondente, renderizado como imagem/SVG em vez de depender da fonte de emoji nativa do sistema operacional ou navegador do leitor. No `mkdocs.yml` deste projeto, ela está configurada com:

Ver a configuração no mkdocs.yml

```
markdown_extensions:
  - pymdownx.emoji:
      emoji_index: !!python/name:material.extensions.emoji.twemoji
      emoji_generator: !!python/name:material.extensions.emoji.to_svg
```

- `emoji_index` aponta para o índice de nomes de emoji do Twemoji (o conjunto de emojis de código aberto originalmente criado pelo Twitter, distribuído junto com o tema Material), que é a tabela usada para traduzir um nome como `rocket` para o emoji correspondente.
- `emoji_generator` define que cada emoji encontrado é gerado como uma imagem SVG embutida na página, em vez de depender de uma fonte de emoji instalada no sistema do leitor — garantindo que o emoji tenha a mesma aparência em qualquer navegador ou sistema operacional.

Markdown fonte do exemplo de emoji

```
:rocket: Documentação publicada com sucesso!
```

Saída renderizada no HTML do MkDocs:

🚀 Documentação publicada com sucesso!

Nesse parágrafo, `:rocket:` foi substituído pelo ícone de um foguete no momento em que esta página foi processada pelo `mkdocs build` / `mkdocs serve` — o texto `:rocket:` em si não aparece na versão final renderizada do site.

7. Caixas de nota vs. âncoras de referência: nem tudo é pymdownx

Até aqui, boa parte das extensões desta página pertence ao namespace `pymdownx`. Mas nem todo recurso visual de "caixa" ou "âncora" vem desse namespace — vale esclarecer a origem de cada um antes de seguir para as próximas seções.

Caixas de nota e aviso com `admonition`

A extensão nativa `admonition` cria caixas destacadas com `!!! tipo "Título"`. O tema Material escolhe automaticamente a cor e o ícone de acordo com o tipo; não é necessário inserir um emoji ou configurar o ícone manualmente.

Escreva cada caixa com o tipo, o título entre aspas e o conteúdo indentado com quatro espaços. Este exemplo também mostra como incluir um bloco de código:

Markdown fonte de todas as caixas

````
!!! note "Nota"
    Use para registrar contexto ou uma informação complementar.

!!! abstract "Resumo"
    Use para resumir uma seção ou apresentar uma visão geral.

!!! info "Informação"
    Use para apresentar informação neutra ou uma tarefa pendente.

!!! tip "Dica"
    Use para sugerir uma boa prática ou um atalho.

!!! success "Concluído"
    Use para indicar um resultado esperado ou uma etapa bem-sucedida.

!!! question "Pergunta"
    Use para destacar uma pergunta frequente ou uma resposta.

!!! warning "Atenção"
    Use para destacar uma condição que pode causar um problema.

!!! failure "Falha"
    Use para descrever uma operação que não foi concluída.

!!! danger "Perigo"
    Use para alertar sobre uma ação destrutiva ou uma falha grave.

!!! bug "Bug"
    Use para documentar um defeito conhecido.

!!! example "Exemplo"
    Use para introduzir um exemplo ou caso de uso.

!!! quote "Citação"
    Use para destacar uma citação.

!!! info "Comando de exemplo"
    ```bash
    uv run mkdocs build
    ```
````

Saída renderizada no HTML do MkDocs:

Nota

Use para registrar contexto ou uma informação complementar.

Resumo

Use para resumir uma seção ou apresentar uma visão geral.

Informação

Use para apresentar informação neutra ou uma tarefa pendente.

Dica

Use para sugerir uma boa prática ou um atalho.

Concluído

Use para indicar um resultado esperado ou uma etapa bem-sucedida.

Pergunta

Use para destacar uma pergunta frequente ou uma resposta.

Atenção

Use para destacar uma condição que pode causar um problema.

Falha

Use para descrever uma operação que não foi concluída.

Perigo

Use para alertar sobre uma ação destrutiva ou uma falha grave.

Bug

Use para documentar um defeito conhecido.

Exemplo

Use para introduzir um exemplo ou caso de uso.

Citação

Use para destacar uma citação.

Comando de exemplo

```
uv run mkdocs build
```

Cada tipo seleciona um ícone e uma aparência. O texto entre aspas define o título, que pode ser escrito em português, e o conteúdo deve ficar indentado quatro espaços. Estes são os nomes dos tipos e seus aliases:

Tipo visual	Tipos alternativos
`note`	—
`abstract`	`summary`, `tldr`
`info`	`todo`
`tip`	`hint`, `important`
`success`	`check`, `done`
`question`	`help`, `faq`
`warning`	`caution`, `attention`
`failure`	`fail`, `missing`
`danger`	`error`
`bug`	—

Tipo visual	Tipos alternativos
`example`	—
`quote`	—

Os nomes da coluna **Tipos alternativos** podem ser usados no lugar do tipo visual e geram a mesma caixa e o mesmo ícone. Por exemplo, `!!! summary "Resumo"` equivale a `!!! abstract "Resumo"`.

Caixas recolhíveis: existe uma variante expansível, habilitada por `pymdownx.details`.

- `admonition` (extensão nativa do Python-Markdown) cria caixas sempre visíveis usando `!!! tipo "Título"`.
- `pymdownx.details` cria caixas recolhíveis usando `??? tipo "Título"`; o leitor clica no título para expandir o conteúdo.

Para iniciar a caixa recolhível já aberta, acrescente `+` ao marcador:

Markdown fonte das caixas recolhíveis

```
??? note "Começa recolhida"
    O leitor clica no título para ver este conteúdo.

???+ tip "Aberta por padrão"
    O leitor pode recolher este conteúdo clicando no título.
```

Saída renderizada no HTML do MkDocs:

Começa recolhida

O leitor clica no título para ver este conteúdo.

Aberta por padrão

O leitor pode recolher este conteúdo clicando no título.

Âncoras de seção com `toc`

A extensão `toc` gera IDs a partir dos títulos e, com `permalink: true`, mostra um ícone de link ao lado deles. Clique nesse ícone para copiar o endereço direto da seção. Também é possível criar um link para o

título manualmente usando seu fragmento.

Primeiro, habilite a opção no `mkdocs.yml`:

Configuração no mkdocs.yml

```
markdown_extensions:
  - toc:
      permalink: true
```

Depois, escreva o título e o link no documento:

Markdown fonte da âncora

```
#### Exemplo de âncora

[Ir para o exemplo](#exemplo-de-ancora)
```

Saída renderizada no HTML do MkDocs:

Exemplo de âncora

Ir para o exemplo

Neste exemplo, o MkDocs cria o ID `exemplo-de-ancora` a partir do título; o ícone de permalink é gerado automaticamente.

Notas de rodapé com `footnotes`

A extensão `footnotes` cria uma referência numerada no texto e a explicação correspondente no fim da página. O número leva à nota; a seta na nota leva de volta à frase original:

Markdown fonte da nota de rodapé

```
Este parágrafo contém uma nota de rodapé.[^ancora-exemplo]

[^ancora-exemplo]: Esta é a explicação apresentada no rodapé.
```

Saída renderizada no HTML do MkDocs:

Este parágrafo contém uma nota de rodapé.[1]

Para usar as âncoras do sumário ou as notas de rodapé, habilite as extensões no arquivo `mkdocs.yml`:

Ver a configuração do sumário e das notas de rodapé

```
markdown_extensions:
  - toc:
      permalink: true
  - footnotes
```

`admonition`, `toc` e `footnotes` são extensões nativas do Python-Markdown; `pymdownx.details` vem do pacote `pymdown-extensions`. O fato de uma extensão produzir uma caixa ou um ícone não determina de qual pacote ela vem.

8. Extensões de referência: `abbr`, `def_list`, `footnotes`, `tables`, `md_in_html`

Estas extensões nativas do Python-Markdown cobrem necessidades comuns de documentação de referência: abreviações, listas de definição, notas de rodapé, tabelas e Markdown dentro de elementos HTML.

`abbr`

A extensão `abbr` permite definir uma abreviação uma única vez (em qualquer ponto do documento, geralmente ao final) e faz com que toda ocorrência dessa sigla no texto da página ganhe um tooltip explicativo ao passar o mouse por cima, sem precisar marcar cada ocorrência manualmente.

Markdown fonte das abreviações

```
Este site usa HTML e CSS para estruturar e estilizar o conteúdo.

*[HTML]: HyperText Markup Language
*[CSS]: Cascading Style Sheets
```

Saída renderizada no HTML do MkDocs: passe o cursor sobre as siglas para ver suas definições.

Este site usa HTML e CSS para estruturar e estilizar o conteúdo.

`def_list`

A extensão `def_list` cria listas de definição — um termo seguido de sua explicação, com a sintaxe `Termo` em uma linha e `: Definição` (dois espaços após os dois-pontos) na linha seguinte:

Markdown fonte da lista de definições

```
MkDocs
:   Gerador de site estático a partir de arquivos Markdown.

Material
:   Tema visual para o MkDocs.
```

Saída renderizada no HTML do MkDocs:

MkDocs

: Gerador de site estático a partir de arquivos Markdown.

Material

: Tema visual para o MkDocs.

`footnotes`

A extensão `footnotes` cria notas de rodapé com âncora de ida-e-volta: a marcação `[^1]` no meio do texto gera um número de referência sobrescrito e clicável, que leva até a explicação correspondente no final da página; e a explicação, por sua vez, traz uma seta que leva de volta ao ponto exato do texto onde a nota foi referenciada.

Markdown fonte da nota de rodapé

```
O MkDocs é escrito em Python[^1].

[^1]: Uma linguagem de programação de alto nível.
```

Saída renderizada no HTML do MkDocs:

O MkDocs é escrito em Python[2].

`tables`

A extensão `tables` interpreta a sintaxe de tabelas do Markdown (linhas separadas por `|`, com uma linha de `---` definindo o cabeçalho) e a renderiza como uma tabela HTML:

Markdown fonte da tabela

```
| Extensão | Namespace |
|---|---|
| `abbr` | nativa |
| `pymdownx.mark` | `pymdownx` |
```

Saída renderizada no HTML do MkDocs:

Extensão	Namespace
`abbr`	nativa
`pymdownx.mark`	`pymdownx`

`md_in_html`

A extensão `md_in_html` permite que o Markdown seja processado dentro de um elemento HTML, desde que o elemento tenha o atributo `markdown="1"`. Isso é útil quando a estrutura HTML é necessária, mas o conteúdo deve continuar usando formatação Markdown:

Markdown fonte dentro do HTML

```
<div markdown="1">

Este parágrafo dentro de HTML contém **negrito** e `código inline`.

</div>
```

Saída renderizada no HTML do MkDocs:

Este parágrafo dentro de HTML contém **negrito** e `código inline`.

Sem o atributo `markdown="1"`, o conteúdo dentro do elemento HTML pode ser tratado como HTML bruto, sem a conversão da sintaxe Markdown.

9. Extensões de formatação de texto: `mark`, `caret`, `tilde`, `keys`, `critic`, `magiclink`, `smartsymbols`

Estas sete extensões do pacote `pymdown-extensions` adicionam sintaxes curtas para formatações de texto que o Markdown puro não cobre.

`pymdownx.mark`

Marca um trecho como destacado (equivalente ao elemento HTML `<mark>`), com a sintaxe `==texto==`:

Markdown fonte do destaque

```
Este é um ==texto destacado==.
```

Saída renderizada no HTML do MkDocs:

Este é um texto destacado.

`pymdownx.caret`

A sintaxe `texto^sobrescrito^` cria um sobrescrito, útil para fórmulas como potências (`H^2^O`). A sintaxe `^^texto^^` (com os dois acentos circunflexos duplicados) cria sublinhado (`sublinhado`).

Markdown fonte de sobrescrito e sublinhado

```
A água é H^2^O.
Este texto está ^^sublinhado^^.
```

Saída renderizada no HTML do MkDocs:

A água é H^2O. Este texto está sublinhado.

`pymdownx.tilde`

A sintaxe `~~texto~~` cria texto tachado. A sintaxe `texto~subscrito~` cria subscrito, útil para fórmulas químicas (`H~2~O`).

Markdown fonte de tachado e subscrito

```
Este texto está ~~tachado~~.
A fórmula da água é H~2~O.
```

Saída renderizada no HTML do MkDocs:

Este texto está ~~tachado~~. A fórmula da água é H_2O.

`pymdownx.keys`

A sintaxe `++tecla1+tecla2++` renderiza uma combinação de teclas como teclas de teclado estilizadas:

Markdown fonte da combinação de teclas

```
Pressione ++ctrl+shift+p++ para abrir a paleta de comandos.
```

Saída renderizada no HTML do MkDocs:

Pressione <kbd>^ Ctrl</kbd>+<kbd>⇧ Shift</kbd>+<kbd>P</kbd> para abrir a paleta de comandos.

`pymdownx.critic`

Marca revisões no estilo "controle de alterações": o autor indica um trecho removido e outro inserido, que ficam visíveis simultaneamente para fins de revisão.

Markdown fonte da revisão

```
O MkDocs {~~era~>é~~} fácil de usar.
```

Saída renderizada no HTML do MkDocs:

O MkDocs ~~era~~é fácil de usar.

`pymdownx.magiclink`

Converte automaticamente qualquer URL solta no texto em um link clicável, sem a necessidade da sintaxe `[texto](url)`:

Markdown fonte do link automático

```
Veja mais em https://www.mkdocs.org.
```

Saída renderizada no HTML do MkDocs:

Veja mais em https://www.mkdocs.org.

`pymdownx.smartsymbols`

Converte sequências específicas de caracteres digitados em símbolos tipográficos equivalentes. Por exemplo, `-->` é convertido em `→` e `(c)` é convertido em `©`.

Markdown fonte dos símbolos

```
O fluxo segue de A --> B, e o conteúdo é (c) deste projeto.
```

Saída renderizada no HTML do MkDocs:

O fluxo segue de A → B, e o conteúdo é © deste projeto.

10. Numeração e destaque de linhas de código

As opções `linenums` e `hl_lines` fazem parte da configuração de `pymdownx.highlight` (já habilitada neste projeto desde a Tarefa 9.4) e funcionam em conjunto com `pymdownx.superfences`, podendo ser aplicadas a qualquer bloco de código cercado por três crases, indicando as opções entre aspas logo após o nome da linguagem:

Ver o Markdown usado com numeração e destaque

````
```python linenums="1" hl_lines="2 4"
import pandas as pd

df = pd.read_csv("dados.csv")
print(df.head())
```
````

Saída renderizada no HTML do MkDocs:

```python
import pandas as pd

```

```
3  df = pd.read_csv("dados.csv")
4  print(df.head())
```

`linenums="1"` numera as linhas do bloco a partir de 1, exibindo a numeração à esquerda do código. `hl_lines="2 4"` destaca visualmente (com um fundo diferenciado) as linhas 2 e 4 do bloco — nesse exemplo, a linha da leitura do CSV e a linha do `print`.

11. Fórmulas matemáticas com Arithmatex

A extensão `pymdownx.arithmatex`, configurada neste projeto com `generic: true`, prepara o HTML de fórmulas escritas em LaTeX para que um motor de renderização matemática no navegador possa processá-las — mas **sozinha ela não renderiza nada visualmente**. É necessário também configurar a opção raiz `extra_javascript` apontando para esse motor de renderização, o que já foi feito neste projeto: `docs/javascripts/mathjax.js` (configuração customizada) somado ao script do CDN do MathJax 3 (ver `mkdocs.yml`). Sem esse segundo passo, o HTML preparado pelo Arithmatex ficaria pronto, mas nenhuma fórmula apareceria de fato na página.

Ver o Markdown usado para as fórmulas

```
Fórmula em bloco:

\[ \text{média} = \frac{\sum valores}{n} \]

Fórmula em linha: \(E = mc^2\).
```

Saída renderizada no HTML do MkDocs:

Fórmula em bloco:

$$\text{média} = \frac{\sum valores}{n}$$

Fórmula em linha, no meio de uma frase: a famosa equação $E = mc^2$ relaciona massa e energia.

12. Quantas extensões habilitar?

Markdown fonte da caixa de recomendação

```
!!! tip "Comece pequeno"
    Não habilite todas as extensões do pacote `pymdown-extensions` de uma vez só, mesmo que isso seja tecnicamente possível. Comece pelo conjunto essencial para documentação técnica — `pymdownx.highlight`, `pymdownx.superfences`, `pymdownx.snippets`, `pymdownx.tabbed`, `pymdownx.details` e `pymdownx.tasklist` — e adicione as demais (como as extensões de referência, as de formatação de texto e o Arithmatex, catalogadas nesta página) apenas quando a necessidade real aparecer no conteúdo sendo escrito. Isso mantém o `mkdocs.yml` organizado e evita acumular funcionalidades redundantes ou nunca utilizadas.
```

Saída renderizada no HTML do MkDocs:

Comece pequeno

Não habilite todas as extensões do pacote `pymdown-extensions` de uma vez só, mesmo que isso seja tecnicamente possível. Comece pelo conjunto essencial para documentação técnica — `pymdownx.highlight`, `pymdownx.superfences`, `pymdownx.snippets`, `pymdownx.tabbed`, `pymdownx.details` e `pymdownx.tasklist` — e adicione as demais (como as extensões de referência, as de formatação de texto e o Arithmatex, catalogadas nesta página) apenas quando a necessidade real aparecer no conteúdo sendo escrito. Isso mantém o `mkdocs.yml` organizado e evita acumular funcionalidades redundantes ou nunca utilizadas.

Esta caixa também é um exemplo de `admonition`: `!!! tip "Título"` escolhe o tipo, e o tema Material adiciona o ícone correspondente. Veja a seção Caixas de nota e aviso para outros tipos e exemplos.

---

1. Esta é a explicação apresentada no rodapé. ↩
2. Uma linguagem de programação de alto nível. ↩

2026-11-13 2026-11-11

Outros plugins

1. Introdução

As páginas anteriores já cobriram imagens, diagramas, integração com GitHub e as extensões `pymdownx` de formatação de conteúdo. Esta página reúne um grupo de plugins complementares que não se encaixam diretamente em nenhum desses temas, mas que resolvem problemas comuns de qualquer site MkDocs em produção:

- **`macros`**: permite usar variáveis e lógica Jinja2 reutilizável dentro das páginas Markdown.
- **`hooks` (redirecionamentos nativos)**: resolve o problema de URLs antigas quebradas quando uma página é renomeada, sem depender de um pacote externo controverso.
- **`minify`**: reduz o tamanho do HTML gerado, sem alterar sua aparência.
- **`awesome-pages`**: permite organizar a navegação por pasta através de arquivos `.pages` — com uma ressalva importante para este projeto, explicada na seção 5.
- **`exporter`**: exporta páginas do site para PDF, reaproveitando o Playwright já usado na página Preview com Playwright.

Todos os cinco já estão instalados e configurados no `mkdocs.yml` real deste projeto. As seções a seguir explicam cada um, mostrando a configuração efetivamente usada.

2. Macros (`mkdocs-macros-plugin`)

Instalação

```
uv add mkdocs-macros-plugin
```

Configuração

```yaml
plugins:
  - macros:
      module_name: macros_mkdocs
```

O que ele faz

O `mkdocs-macros-plugin` transforma cada página Markdown em um template Jinja2 antes de renderizá-la. Isso significa que, além do Markdown comum, é possível usar variáveis e expressões entre chaves duplas — a mesma sintaxe `{{ ... }}` do Jinja2 — diretamente no texto da página.

> As ocorrências de chaves duplas mostradas nesta seção estão envolvidas por um recurso do próprio Jinja2 que bloqueia sua substituição (sem gerar nenhuma marcação visível no texto renderizado), para que apareçam como sintaxe literal nesta página, em vez de serem interpretadas e substituídas pelo `mkdocs-macros-plugin` durante o build.

Duas fontes de variáveis costumam ser usadas:

- **Variáveis automáticas do próprio MkDocs**, como `{{ page.title }}` (o título da página atual) ou `{{ config.site_name }}` (o `site_name` definido no `mkdocs.yml`), disponíveis sem nenhuma configuração extra.
- **Variáveis customizadas**, definidas pelo autor do site em um arquivo Python que declara uma função `define_env(env)`. Essa função recebe um objeto `env` e registra variáveis dentro de `env.variables`. Por convenção, o plugin procura esse arquivo em `main.py` na raiz do projeto — mas neste projeto `main.py` já é a aplicação real (`def main(): print(...)`), e não um módulo de macros. Por isso a configuração acima usa explicitamente a opção `module_name: macros_mkdocs`, apontando para um arquivo dedicado, `macros_mkdocs.py`, em vez de deixar o plugin tentar importar `main.py` como se fosse seu módulo de macros:

```
# macros_mkdocs.py
def define_env(env):
    env.variables["minha_variavel"] = "valor"
```

Depois de registrada, a variável pode ser usada em **qualquer página Markdown** do site:

```
O valor configurado é: {{ minha_variavel }}
```

No momento do `mkdocs build`, o plugin substitui `{{ minha_variavel }}` pelo texto `valor` antes de o Markdown ser convertido em HTML. O mesmo mecanismo também permite registrar macros (funções Jinja2 chamáveis, via `env.macro`), não apenas variáveis simples — úteis para gerar trechos repetitivos de conteúdo a partir de uma única definição central, em vez de copiar o mesmo texto em várias páginas.

3. Redirecionamentos sem dependência externa (`hooks`)

Por que o `mkdocs-redirects` foi evitado

A escolha natural para redirecionar URLs antigas de página costuma ser o pacote `mkdocs-redirects`. Esse pacote, porém, passou a declarar o `properdocs` como dependência a partir da versão 1.2.3. O `properdocs` é um fork/continuação concorrente do próprio MkDocs, criado após uma disputa de governança no projeto original.

Para deixar claro: o `properdocs` **não é malware** nem um pacote malicioso. O problema não é de segurança — é de escopo e de risco de manutenção. Este projeto já usa o MkDocs original diretamente (`mkdocs`, `mkdocs-material`) e não tem motivo para trazer, como dependência transitiva de um plugin de redirecionamento, um fork inteiro e concorrente da própria ferramenta de documentação que o projeto usa.

É uma dependência desnecessária para a funcionalidade desejada, e controversa o suficiente para ser evitada.

A alternativa: `hooks` nativo do MkDocs

Em vez do `mkdocs-redirects`, este projeto usa a funcionalidade `hooks`, nativa do próprio MkDocs desde a versão 1.4 e documentada oficialmente em mkdocs.org/user-guide/configuration/#hooks. Essa funcionalidade permite registrar um ou mais scripts Python locais como handlers dos eventos internos do MkDocs (como `on_post_build`), **sem instalar nenhum pacote adicional do PyPI** — o script é só um arquivo `.py` do próprio repositório.

A configuração, no `mkdocs.yml` deste projeto, é:

```
hooks:
  - hooks/redirects.py
```

O script `hooks/redirects.py`

O conteúdo real do arquivo, na raiz do projeto, é:

```
"""hooks/redirects.py — Native_Redirect_Hook.

Gera páginas de redirecionamento HTML (meta refresh) para caminhos antigos
de página, sem depender do pacote `mkdocs-redirects` (nem, por
consequência, do pacote `properdocs`). Referenciado na opção `hooks` do
mkdocs.yml.

Documentação oficial da funcionalidade `hooks` do MkDocs (disponível desde a
versão 1.4, sem exigir nenhum pacote PyPI adicional):
https://www.mkdocs.org/user-guide/configuration/#hooks
"""

import os

# Mapeamento de caminho antigo (relativo a docs_dir) -> caminho atual.
REDIRECTS = {
    "preview.md": "playwright-preview.md",
}


def _write_redirect_html(target_path: str, destination_url: str) -> None:
    """Escreve um arquivo HTML de redirecionamento em `target_path`."""
    os.makedirs(os.path.dirname(target_path), exist_ok=True)
    content = (
        "<!DOCTYPE html>\n"
        "<html lang=\"pt-BR\">\n"
        "<head>\n"
        "  <meta charset=\"utf-8\">\n"
        f"  <meta http-equiv=\"refresh\" content=\"0; url={destination_url}\">\n"
        f"  <link rel=\"canonical\" href=\"{destination_url}\">\n"
```

```
            "  <title>Redirecionando...</title>\n"
            "</head>\n"
            "<body>\n"
            f"  Redirecionando para <a href=\"{destination_url}\">{destination_url}</a>...\n"
            "</body>\n"
            "</html>\n"
        )
        with open(target_path, "w", encoding="utf-8") as f:
            f.write(content)


def on_post_build(config, **kwargs):
    """Evento `on_post_build` do MkDocs: gera uma página de
    redirecionamento para cada entrada de REDIRECTS, dentro de
    `config["site_dir"]`."""
    site_dir = config["site_dir"]
    for old_path, new_path in REDIRECTS.items():
        old_slug = old_path.rsplit(".", 1)[0]
        new_slug = new_path.rsplit(".", 1)[0]
        target_html = os.path.join(site_dir, old_slug, "index.html")
        destination_url = f"../{new_slug}/"
        _write_redirect_html(target_html, destination_url)
```

Duas peças merecem atenção:

- **O dicionário `REDIRECTS`** é o único lugar que precisa ser editado para adicionar um novo redirecionamento: cada chave é o caminho antigo de uma página (relativo a `docs_dir`), e cada valor é o caminho atual correspondente.
- **A função `on_post_build(config, **kwargs)`** corresponde a um evento do ciclo de vida do MkDocs, disparado **depois** que o site inteiro já foi escrito em disco, dentro da pasta `site/` (`config["site_dir"]`). Ou seja, quando esse evento dispara, todas as páginas reais (incluindo `playwright-preview.md` já convertida em `site/playwright-preview/index.html`) já existem. A função aproveita exatamente esse momento para escrever um arquivo HTML **extra**, que não corresponde a nenhuma página Markdown real, contendo apenas uma tag `<meta http-equiv="refresh" content="0; url=...">` — a técnica padrão de redirecionamento automático em páginas HTML estáticas, sem exigir um servidor com lógica de redirecionamento.

O exemplo configurado

O mapeamento ativo hoje é:

```
REDIRECTS = {
    "preview.md": "playwright-preview.md",
}
```

Isso simula uma situação comum em sites de documentação reais: a página hoje chamada "Preview com Playwright" (`playwright-preview.md`) já teve, no passado, um nome mais curto, `preview.md`. Qualquer link antigo — em um favorito salvo, em outro site, em um resultado de busca já indexado — que ainda

aponte para `.../preview/` continua funcionando: ao acessá-lo, o visitante é redirecionado automaticamente para `.../playwright-preview/`, a URL atual da página real, sem receber um erro 404.

4. Minificação (`mkdocs-minify-plugin`)

Instalação

```
uv add mkdocs-minify-plugin
```

Configuração

```
plugins:
  - minify:
      minify_html: true
```

O que ele faz

O `mkdocs-minify-plugin` processa o HTML já gerado por cada página do site e remove tudo que é irrelevante para a renderização no navegador: espaços em branco redundantes entre tags, quebras de linha de formatação, comentários HTML e outras sobras de indentação que só existem para facilitar a leitura do código-fonte. Com `minify_html: true`, essa limpeza é aplicada a todo arquivo `.html` do site gerado.

O resultado visual da página **não muda em nada** — é o mesmo HTML, semanticamente idêntico, só mais compacto como texto. O benefício é puramente de tamanho de arquivo: páginas menores para transferir, o que reduz (ainda que de forma modesta, em sites deste porte) o tempo de carregamento e o consumo de banda de quem acessa a documentação publicada.

5. Organização automática de navegação (`mkdocs-awesome-pages-plugin`)

Instalação

```
uv add mkdocs-awesome-pages-plugin
```

Configuração

```
plugins:
  - awesome-pages
```

O que ele faz, e por que não tem efeito neste projeto

O `mkdocs-awesome-pages-plugin` permite controlar a navegação do site a pasta por pasta, sem precisar listar cada página manualmente na seção `nav` do `mkdocs.yml`. Isso é feito através de arquivos chamados `.pages`, colocados dentro de cada pasta de `docs_dir`, com opções como:

- `title`: o título exibido para aquela pasta/seção na navegação.
- `nav`: a ordem e os rótulos das páginas daquela pasta (pode reordenar, renomear ou omitir arquivos).
- `collapse`: se a seção deve iniciar recolhida no menu lateral.

Para ser honesto sobre o comportamento real neste projeto: **o `awesome-pages` não tem efeito nenhum sobre a navegação deste site específico.** Isso não é um bug nem uma configuração incompleta — é o comportamento documentado oficialmente pelo próprio plugin: ele não faz nada quando o `mkdocs.yml` já define uma seção `nav` explícita. Como este projeto declara, desde a Tarefa 1.1, uma seção `nav` completa e manual (as treze entradas visíveis em `mkdocs.yml`, da página "Início" até "Outros plugins"), o `awesome-pages` permanece instalado e habilitado, mas inativo na prática: a navegação real é inteiramente determinada pelo `nav` explícito, não por arquivos `.pages`.

O plugin continua configurado aqui porque faz parte do catálogo de plugins do ecossistema coberto por esta página — e porque pode ser útil no futuro, caso o `nav` explícito seja removido. Para quem quiser usar o plugin de verdade, em um projeto **sem** `nav` explícito, um arquivo `.pages` de referência se pareceria com isto:

```
title: Minha Seção
nav:
  - Visão geral: index.md
  - ...
collapse: true
```

Colocado dentro de uma subpasta de `docs_dir`, esse arquivo faria o `awesome-pages` renomear a seção correspondente para "Minha Seção" na navegação, definir "Visão geral" (`index.md`) como primeira entrada, preencher o restante com `...` (um atalho do próprio plugin para "todas as outras páginas da pasta, em ordem alfabética") e exibir a seção inicialmente recolhida no menu lateral.

6. Exportação para PDF (`mkdocs-exporter`)

Instalação

```
uv add mkdocs-exporter
```

Configuração

A configuração real usada neste projeto, em `mkdocs.yml`:

```yaml
plugins:
  - exporter:
      formats:
        pdf:
          enabled: true
```

O que ele faz

O `mkdocs-exporter` exporta cada página do site (ou a documentação inteira) como um arquivo PDF. A parte interessante de como ele faz isso: em vez de tentar converter o Markdown ou o HTML diretamente em PDF por conta própria, ele abre a própria página já publicada em um **navegador headless controlado via Playwright** e manda esse navegador renderizar a página exatamente como um visitante humano a veria — com o CSS do tema Material aplicado, com diagramas Mermaid já desenhados, com sintaxe de código já destacada — e só então converte essa renderização final em PDF.

Isso significa que o PDF gerado não é uma reinterpretação simplificada do conteúdo: é, literalmente, uma cópia fiel e impressa da página como ela existe no site.

Vale destacar explicitamente: o `mkdocs-exporter` **não traz uma dependência nova de automação de navegador**. Ele reaproveita o mesmíssimo pacote `playwright`, já instalado neste projeto desde a página Preview com Playwright, onde é usado pelo Preview_Script (`docs_preview.py`) para abrir o site buildado localmente. O `mkdocs-exporter` apenas dá um segundo uso a uma ferramenta que já estava presente: no Preview_Script, o Playwright abre o navegador para que um humano veja a página; no `mkdocs-exporter`, o Playwright abre o navegador para capturar a página e congelá-la em PDF — a mesma tecnologia de automação, dois propósitos diferentes.

Como usar

Depois de um `mkdocs build` (ou `mkdocs serve`) com esse plugin habilitado, cada página do site passa a exibir um botão/link "Download as PDF" próximo ao topo. Clicar nesse botão baixa um arquivo PDF contendo apenas aquela página específica, já convertida — sem precisar rodar nenhum comando adicional manualmente para gerar o PDF daquela página.

2026-11-11 2026-11-11

Pacotes utilizados

Esta página reúne as dependências Python declaradas diretamente pelo projeto em `pyproject.toml`, explica o papel de cada uma e aponta para suas referências. São os pacotes escolhidos para montar e publicar este site; as dependências indiretas instaladas por eles ficam registradas no `uv.lock`.

As referências são geradas automaticamente em Referências de pacotes a cada execução do preview local e antes do build no workflow de publicação do GitHub Actions.

Dependências do projeto

Pacote	Uso neste projeto	Referência
`mkdocs`	Gera o site estático a partir dos arquivos Markdown e do `mkdocs.yml`.	Documentação do MkDocs · PyPI
`mkdocs-material`	Tema Material: layout, navegação, busca, paletas e componentes do site.	Documentação do Material for MkDocs · PyPI
`mkdocs-awesome-pages-plugin`	Oferece organização automática de páginas e navegação por arquivos `.pages`.	PyPI
`mkdocs-exporter`	Exporta páginas do site, inclusive para PDF; usa o navegador via Playwright.	PyPI
`mkdocs-git-revision-date-localized-plugin`	Exibe datas de criação e de atualização baseadas no histórico Git.	PyPI
`mkdocs-glightbox`	Adiciona ampliação de imagens ao clicar nelas.	PyPI
`mkdocs-macros-plugin`	Permite usar variáveis, macros e filtros nos arquivos Markdown.	PyPI
`mkdocs-minify-plugin`	Minifica os arquivos HTML gerados para publicação.	PyPI

aws PARTNER Advanced Tier Services
Immersion Day · Solution Provider · DevOps Services Competency · Migration Services Competency
Nonprofit Services Competency · Public Sector Solution Provider · Well-Architected Partner Program
Solution Spark Public Sector Partners · Amazon EC2 for Windows Server Delivery
dataRain FAR BEYOND THE CLOUD

Pacote	Uso neste projeto	Referência
`mkdocstrings[python]`	Gera documentação a partir do código e das docstrings Python; o extra `python` instala o handler Python.	Documentação do mkdocstrings · PyPI
`mkdocstrings-typescript`	Integra o handler TypeScript com o mkdocstrings.	PyPI
`griffe-typedoc`	Lê a saída JSON do TypeDoc para documentar APIs TypeScript. O projeto fixa uma revisão Git que contém a correção de compatibilidade com `schema_version`.	Repositório e revisão fixada
`playwright`	Automatiza o navegador Chromium no preview e na exportação de páginas.	Documentação do Playwright para Python · PyPI
`pymdown-extensions`	Fornece extensões Markdown usadas no projeto, como `superfences`, admonitions e blocos de legenda.	Documentação do PyMdown Extensions · PyPI

As versões mínimas e a origem Git de `griffe-typedoc` estão especificadas em `pyproject.toml`. O `uv.lock` registra as versões resolvidas para que a instalação do projeto seja reproduzível.

Instalar as dependências deste projeto

Na raiz do repositório, execute:

```
uv sync
```

`uv sync` lê todas as dependências de `pyproject.toml` e usa `uv.lock` para instalar as versões resolvidas, incluindo a revisão Git específica de `griffe-typedoc`. **Não é necessário adicionar os pacotes novamente** quando você está trabalhando neste repositório.

O `uv sync` instala apenas dependências Python. A geração da API TypeScript também precisa do **Node.js**, que inclui o npm, e do **TypeDoc** — um pacote JavaScript, não uma biblioteca Python. Para configurar esse pré-requisito em uma máquina local:

1. Instale a versão LTS do Node.js; o npm é instalado junto.
2. Abra um novo terminal e confirme que `node` e `npm` estão disponíveis:

```
node --version
npm --version
```

1. Se Node.js estiver instalado no local padrão do Windows, mas `node` / `npm` não forem reconhecidos, ajuste o `PATH` desta janela do PowerShell:

```
$env:Path = "C:\Program Files\nodejs;$env:APPDATA\npm;$env:Path"
node --version
npm --version
```

Para manter a correção, adicione `C:\Program Files\nodejs` e `%APPDATA%\npm` à variável `Path` do usuário no Windows e reabra o terminal. Se a instalação estiver em outro local, use o caminho real do Node.js.

1. Instale o TypeDoc globalmente e confira se o comando está acessível:

```
npm install --global typedoc
typedoc --version
```

O handler TypeScript do `mkdocstrings` executa `typedoc` diretamente. Se o build falhar com `FileNotFoundError: [WinError 2]`, confira se Node.js, npm e TypeDoc estão instalados e se os diretórios necessários estão no `PATH`. O workflow do GitHub Actions também instala Node.js e TypeDoc antes do build. Consulte a documentação do TypeDoc e o pacote no npm.

Para criar outro projeto e adicionar o mesmo conjunto de dependências, use `uv add` com as especificações abaixo:

```
uv add "mkdocs>=1.6.1" "mkdocs-awesome-pages-plugin>=2.10.1" "mkdocs-exporter>=6.2.0"
"mkdocs-git-revision-date-localized-plugin>=1.7.0" "mkdocs-glightbox>=0.5.3" "mkdocs-
macros-plugin>=1.5.0" "mkdocs-material>=9.8.1" "mkdocs-minify-plugin>=0.8.0"
"mkdocstrings-typescript>=0.1.1" "mkdocstrings[python]>=1.0.8" "playwright>=1.65.0"
"pymdown-extensions>=13.0" "griffe-typedoc @ git+https://github.com/mkdocstrings/griffe-
typedoc.git@d0afb31733b2954daed464b5015db6f0e832a21e"
```

`uv add` registra os pacotes no projeto atual e atualiza o lockfile. No comando de exemplo, `griffe-typedoc` usa a mesma revisão Git fixada neste repositório, em vez de uma versão diferente do índice de pacotes.

Instalar o Chromium para Playwright

O pacote Python `playwright` instala a biblioteca de automação, mas o navegador Chromium é um recurso separado. Para gerar previews e PDFs localmente, instale-o uma vez com:

```
uv run playwright install chromium
```

No workflow do GitHub Actions, a instalação do navegador e das dependências de sistema é feita pelo próprio workflow.

2026-11-12 2026-11-12

Referências de pacotes Python

Esta página é gerada automaticamente a partir das dependências do projeto. Para atualizar a lista manualmente, execute `uv run docs_preview.py --generate-references`.

Pacote	Requisito declarado	Referência
`griffe-typedoc`	`sem versão mínima`	Abrir referência
`mkdocs`	`>=1.6.1`	Abrir referência
`mkdocs-awesome-pages-plugin`	`>=2.10.1`	Abrir referência
`mkdocs-exporter`	`>=6.2.0`	Abrir referência
`mkdocs-git-revision-date-localized-plugin`	`>=1.7.0`	Abrir referência
`mkdocs-glightbox`	`>=0.5.3`	Abrir referência
`mkdocs-macros-plugin`	`>=1.5.0`	Abrir referência
`mkdocs-material`	`>=9.8.1`	Abrir referência
`mkdocs-minify-plugin`	`>=0.8.0`	Abrir referência
`mkdocstrings-typescript`	`>=0.1.1`	Abrir referência
`mkdocstrings[python]`	`>=1.0.8`	Abrir referência
`playwright`	`>=1.65.0`	Abrir referência
`pymdown-extensions`	`>=13.0`	Abrir referência
`pypdf`	`>=7.0.1`	Abrir referência
`pyyaml`	`>=6.0.3`	Abrir referência

Instalar os pacotes

Na raiz do repositório, instale as dependências e as versões registradas no lockfile com:

```
uv sync
```

Função de extração

A função abaixo lê o `pyproject.toml` ou o `requirements.txt` e gera esta página:

`docs_preview.generate_references_markdown(project_root=PROJECT_ROOT)`

Gera `docs/referencias.md` com referências extraídas das dependências.

Lê as dependências de `pyproject.toml` (incluindo origens Git definidas em `tool.uv.sources`) ou, se não houver uma lista de dependências, usa `requirements.txt`. Para dependências GitHub com revisão fixada, cria um link para o commit; para pacotes sem URL direta, cria uma referência à página correspondente no PyPI. A página gerada também documenta a instalação com `uv sync` e inclui esta função via `mkdocstrings`.

A função roda antes do build em `run_mkdocs_build` e no workflow de publicação. Também pode ser chamada isoladamente com `uv run docs_preview.py --generate-references`.

Parameters:

Name	Type	Description	Default
`project_root`	`Path`	Raiz do projeto que contém os arquivos de dependências.	`PROJECT_ROOT`

Returns:

Name	Type	Description
`Path`	`Path`	Caminho do arquivo Markdown gerado.

Raises:

Type	Description
`FileNotFoundError`	Se não houver um `requirements.txt` de fallback.
`ValueError`	Se as dependências estiverem vazias ou malformadas.

Source code in `docs_preview.py`

```python
def generate_references_markdown(project_root: Path = PROJECT_ROOT) -> Path:
    """Gera ``docs/referencias.md`` com referências extraídas das dependências.

    Lê as dependências de ``pyproject.toml`` (incluindo origens Git definidas
    em ``tool.uv.sources``) ou, se não houver uma lista de dependências,
    usa ``requirements.txt``. Para dependências GitHub com revisão fixada, cria
    um link para o commit; para pacotes sem URL direta, cria uma referência à
    página correspondente no PyPI. A página gerada também documenta a
    instalação com ``uv sync`` e inclui esta função via ``mkdocstrings``.

    A função roda antes do build em ``run_mkdocs_build`` e no workflow de
    publicação. Também pode ser chamada isoladamente com
    ``uv run docs_preview.py --generate-references``.

    Args:
        project_root: Raiz do projeto que contém os arquivos de dependências.

    Returns:
        Path: Caminho do arquivo Markdown gerado.

    Raises:
        FileNotFoundError: Se não houver um ``requirements.txt`` de fallback.
        ValueError: Se as dependências estiverem vazias ou malformadas.
    """
    project_root = project_root.resolve()
    dependencies = _dependency_references(project_root)
    rows = [
        "| Pacote | Requisito declarado | Referência |",
        "|---|---|---|",
    ]

    for name, specifier, reference in dependencies:
        safe_name = name.replace("|", r"\|")
        safe_specifier = (specifier or "sem versão mínima").replace("|", r"\|")
        safe_reference = reference.replace("|", r"\|")
        rows.append(
            f"| `{safe_name}` | `{safe_specifier}` | [Abrir referência]({safe_reference})
|"
        )

    content = "\n".join(
        [
            "---",
            "pdf: false",
            "---",
            "",
            "# Referências de pacotes Python",
            "",
            "Esta página é gerada automaticamente a partir das dependências do projeto.",
            "Para atualizar a lista manualmente, execute `uv run docs_preview.py --
generate-references`.",
            "",
            *rows,
            "",
            "## Instalar os pacotes",
            "",
```

```
            "Na raiz do repositório, instale as dependências e as versões registradas no
lockfile com:",
            "",
            "```bash",
            "uv sync",
            "```",
            "",
            "## Função de extração",
            "",
            "A função abaixo lê o `pyproject.toml` ou o `requirements.txt` e gera esta
página:",
            "",
            "::: docs_preview.generate_references_markdown",
            "",
        ]
    )

    output_path = project_root / "docs" / "referencias.md"
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_text(content, encoding="utf-8")
    return output_path
```

2026-11-12 2026-11-12

Git

Git é um sistema distribuído de controle de versão. Ele registra alterações em arquivos, permite criar linhas paralelas de trabalho (branches) e combinar essas alterações. GitHub é um serviço para hospedar repositórios Git e colaborar pela web; os comandos `git` funcionam localmente e não exigem o GitHub para criar commits.

1. Instalar e configurar a identidade

Instale o Git para seu sistema operacional e confirme que ele está disponível:

```
git --version
```

Configure o nome e o e-mail que serão gravados nos commits. Use o endereço que deseja associar às contribuições; se quiser ocultar seu e-mail pessoal no GitHub, use o endereço `noreply` fornecido nas configurações da conta.

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@example.com"
git config --global init.defaultBranch main
```

Confira as configurações:

```
git config --global --list
```

Esses valores identificam autoria nos commits; não autenticam no GitHub. Para operações remotas, configure HTTPS com o GitHub CLI ou uma chave SSH. Veja também o guia de GitHub CLI.

2. Criar um repositório ou clonar um existente

Para iniciar um repositório Git em uma pasta local:

```
mkdir meu-projeto
cd meu-projeto
git init -b main
```

Para obter uma cópia local de um repositório que já existe no GitHub:

```
git clone https://github.com/ORGANIZACAO/REPOSITORIO.git
cd REPOSITORIO
```

Confira o estado e os endereços remotos configurados:

```
git status
git remote -v
```

Conectar uma pasta local a um repositório remoto

Crie o repositório remoto primeiro no GitHub. Se a pasta local ainda não tiver commits, crie um arquivo inicial e registre-o:

```
echo "# Meu projeto" > README.md
git add README.md
git commit -m "docs: inicia o projeto"
```

Se a pasta já tem commits, não repita essa etapa. Configure `origin` e envie a branch principal:

```
git remote add origin https://github.com/ORGANIZACAO/REPOSITORIO.git
git remote -v
git push -u origin main
```

`origin` é apenas o nome convencional do remote. Se ele já existir, veja sua URL com `git remote -v` antes de alterá-la. Para configurar autenticação HTTPS pelo GitHub CLI:

```
gh auth login
gh auth setup-git
```

3. Ciclo básico: revisar, preparar e commitar

O Git só inclui no commit os arquivos que foram preparados com `git add`. Revise as alterações antes de registrá-las:

```
git status
git diff
git add docs/guia.md
git diff --staged
git commit -m "docs: adiciona guia do projeto"
```

Use `git add` com caminhos específicos para não incluir arquivos acidentais. Para adicionar todas as alterações rastreadas e não rastreadas, primeiro confira `git status` e então use:

```
git add -A
```

Um commit é salvo no histórico local. Para ver os commits recentes:

```
git log --oneline --graph --decorate -10
```

4. Branches, `pull`, `fetch` e `push`

Uma branch permite desenvolver uma alteração sem modificar diretamente a branch principal. Um fluxo comum é atualizar `main`, criar uma branch e trabalhar nela:

```
git switch main
git pull --ff-only
git switch -c docs/novo-capitulo
```

- `git fetch` baixa novos commits e atualiza as referências remotas, sem integrá-los à branch atual.
- `git pull` busca e integra as alterações da branch remota que está sendo acompanhada. `--ff-only` evita criar um merge commit automaticamente se houver divergência.
- `git push` envia commits locais para o remote.

Depois de commitar as alterações na branch, publique-a e configure o acompanhamento remoto:

```
git add docs/novo-capitulo.md
git commit -m "docs: adiciona novo capítulo"
git push -u origin docs/novo-capitulo
```

Após configurar a branch remota uma vez, os próximos envios podem usar:

```
git push
git pull
```

5. Integrar alterações com merge

O `merge` combina os commits de uma branch na branch atual. Depois que uma revisão de Pull Request estiver aprovada, atualize `main` e faça o merge da branch de trabalho:

```
git switch main
git pull --ff-only
git merge --no-ff docs/novo-capitulo
git push origin main
```

Em projetos colaborativos, geralmente o merge é feito no Pull Request pelo GitHub, em vez de enviar diretamente à branch principal. Isso permite revisão e verificações automáticas antes da integração.

Resolver conflitos de merge

Se duas branches alterarem as mesmas linhas, o Git marca um conflito nos arquivos. Confira os caminhos afetados:

```
git status
```

Abra cada arquivo e escolha o conteúdo correto entre os marcadores `<<<<<<<`, `=======` e `>>>>>>>`. Depois de editar e revisar o resultado, marque o conflito como resolvido e conclua o merge:

```
git add caminho/do/arquivo
git diff --staged
git commit
```

Não apague os marcadores sem decidir qual conteúdo deve permanecer. Se ainda não começou a resolver e quiser cancelar um merge em andamento:

```
git merge --abort
```

6. Ignorar arquivos e desfazer alterações com segurança

O arquivo `.gitignore` lista arquivos locais ou gerados que não devem ser adicionados ao Git. Por exemplo:

```
.venv/
__pycache__/
site/
.env
```

Confira o que será incluído antes de commitar, especialmente arquivos de configuração local, logs e credenciais:

```
git status --short
git diff
```

Para desfazer uma alteração já commitada sem reescrever o histórico compartilhado, crie um commit inverso:

```
git revert HASH-DO-COMMIT
```

Evite `git push --force` em branches compartilhadas; ele pode remover commits publicados por outros colaboradores. Para aprender a criar Pull Requests e fazer merge pelo GitHub, consulte GitHub CLI (`gh`).

7. Comandos principais

Comando	Uso
`git status`	Mostra a branch atual e as alterações.
`git diff`	Exibe alterações ainda não preparadas.
`git add`	Prepara arquivos para o próximo commit.
`git commit`	Registra as alterações preparadas no histórico local.
`git log`	Consulta o histórico de commits.
`git switch`	Muda de branch ou cria uma branch.
`git fetch`	Atualiza referências remotas sem integrar commits.
`git pull`	Busca e integra alterações remotas.
`git push`	Publica commits locais no repositório remoto.
`git merge`	Integra o histórico de uma branch à branch atual.
`git remote -v`	Lista os remotes configurados.

Para consultar as opções de qualquer comando:

```
git help <comando>
git help push
```

8. Referências oficiais

- Git: primeiros passos
- Manual do Git
- Conectar-se ao GitHub com SSH
- GitHub CLI (`gh`)

2026-11-13 2026-11-13

GitHub Actions

GitHub Actions é a plataforma de automação integrada ao GitHub. Ela executa tarefas de integração contínua e entrega/publicação contínua (CI/CD), como validar, testar, compilar e publicar um projeto quando ocorrem eventos no repositório. Para aprender a versionar os arquivos de workflow e enviá-los ao GitHub, consulte Git.

1. Como funciona

Um workflow é um processo descrito em YAML e salvo em `.github/workflows/`. Quando um evento configurado acontece, o GitHub inicia um ou mais jobs em runners. Cada job executa steps em sequência; um step pode executar um comando ou chamar uma action reutilizável.

Conceito	Significado
Workflow	Automação completa definida em um arquivo `.yml` ou `.yaml`.
Evento (`on`)	Situação que inicia a automação, como `push`, `pull_request` ou `workflow_dispatch`.
Job	Conjunto de etapas executado em um runner. Jobs diferentes podem executar em paralelo ou depender de outro job.
Runner	Máquina hospedada pelo GitHub ou máquina própria que executa um job.
Step	Um comando `run` ou uma action `uses` executada dentro do job.
Action	Componente reutilizável chamado por uma etapa, por exemplo, para clonar o repositório.

O ciclo típico é: evento → workflow → job → runner → steps → resultado e logs na aba **Actions**.

2. Onde criar o arquivo YAML

Crie um arquivo `.yml` ou `.yaml` dentro desta pasta do repositório:

```
.github/
└── workflows/
    └── validar-documentacao.yml
```

Ao enviar esse arquivo ao GitHub, o workflow aparece na aba **Actions**. O conteúdo do arquivo especifica quando executar e quais tarefas realizar.

3. Exemplo: validar este projeto MkDocs

O workflow abaixo valida a documentação em pushes para `main`, em Pull Requests direcionados a `main` e quando iniciado manualmente. Como este projeto documenta TypeScript e exporta PDFs, o job também instala Node.js/TypeDoc e Chromium para Playwright.

Salve este conteúdo como `.github/workflows/validar-documentacao.yml`:

```yaml
name: Validar documentação

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Baixar o repositório
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Configurar Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "24"

      - name: Instalar TypeDoc
        run: npm install --global typedoc

      - name: Instalar uv
        uses: astral-sh/setup-uv@v3

      - name: Instalar dependências Python
        run: uv sync

      - name: Instalar Chromium do Playwright
        run: uv run playwright install --with-deps chromium

      - name: Atualizar referências de pacotes
        run: uv run python docs_preview.py --generate-references

      - name: Validar o build do MkDocs
```

```
        run: uv run mkdocs build --strict
```

O arquivo `.github/workflows/docs.yml` deste repositório já contém o workflow de build e publicação do Pages. Não crie outro workflow de build junto deste, a menos que queira executar ambos. O exemplo acima serve como modelo de validação; para publicar o site, adapte o workflow existente ou use o fluxo de deploy conforme Integração com GitHub.

4. Como ler e adaptar o YAML

- `name` é o nome mostrado na aba **Actions**.
- `on` lista os eventos de início. `push` filtra branch; `pull_request` valida alterações propostas; `workflow_dispatch` habilita a execução manual.
- `permissions` concede ao token automático somente o acesso necessário. Para um build que apenas lê código, `contents: read` é suficiente.
- `jobs` define um ou mais jobs. Cada job escolhe um runner em `runs-on`.
- `steps` lista ações e comandos executados em sequência. `uses` chama uma action; `run` executa comandos no shell do runner.
- `fetch-depth: 0` baixa o histórico Git completo, útil para plugins que leem datas de commits.
- `--strict` faz o MkDocs falhar quando encontra avisos, como links internos inválidos.

Um evento agendado, por exemplo, usa uma expressão cron em UTC:

```yaml
on:
  schedule:
    - cron: "0 9 * * 1"
```

O exemplo executa às 09:00 UTC toda segunda-feira. Consulte a documentação oficial para a sintaxe cron e os eventos disponíveis.

5. Publicar documentação no GitHub Pages

Para publicar o site gerado pelo MkDocs, o workflow precisa empacotar a pasta de saída e executar um job de deploy que dependa do build. O workflow real deste projeto já inclui estas etapas e as permissões correspondentes. A configuração usual é:

```yaml
permissions:
  contents: read
  pages: write
  id-token: write
```

No job de build, envie `site/` como artefato do Pages:

```yaml
- name: Enviar artefato do Pages
  uses: actions/upload-pages-artifact@v3
  with:
    path: site
```

Depois do build, um job de deploy dependente publica o artefato:

```yaml
deploy:
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${{ steps.deployment.outputs.page_url }}
  steps:
    - name: Publicar no GitHub Pages
      id: deployment
      uses: actions/deploy-pages@v4
```

No repositório, selecione **Settings** → **Pages** → **Build and deployment** → **Source** → **GitHub Actions**. A fonte e as permissões estão detalhadas na página Integração com GitHub.

6. Segredos, permissões e repositórios privados

Não grave tokens, senhas ou chaves no YAML. Cadastre valores sensíveis em **Settings** → **Secrets and variables** → **Actions** e acesse-os como secrets no workflow:

```yaml
env:
  API_TOKEN: ${{ secrets.API_TOKEN }}
```

Para clonar o próprio repositório privado, `actions/checkout` usa por padrão o `GITHUB_TOKEN` do workflow. Se um job precisar ler outro repositório privado, use um token ou GitHub App com acesso mínimo, salvo como secret e passado apenas à etapa necessária.

```yaml
- name: Baixar um repositório privado adicional
  uses: actions/checkout@v4
  with:
    repository: ORGANIZACAO/REPOSITORIO-PRIVADO
    token: ${{ secrets.REPO_READ_TOKEN }}
    path: dependencias/privadas
```

Quem tem permissão de escrita no repositório pode alterar workflows e potencialmente usá-los para acessar secrets. Revise alterações de workflows, limite permissões do `GITHUB_TOKEN` e não disponibilize secrets a código não confiável de Pull Requests externos.

7. Acompanhar, iniciar e depurar execuções

Pela interface, abra a aba **Actions**, escolha o workflow e clique em uma execução para ver cada job, step e log. Se a execução falhar, comece pelo primeiro step vermelho e confira o comando e a mensagem de erro.

Pelo GitHub CLI:

```
gh auth status
gh workflow list
gh run list --workflow docs.yml
gh run view ID-DA-EXECUCAO --log
gh run watch ID-DA-EXECUCAO
```

Para iniciar manualmente um workflow que declara `workflow_dispatch`:

```
gh workflow run docs.yml --ref main
```

Para reexecutar uma execução anterior:

```
gh run rerun ID-DA-EXECUCAO
```

`gh run watch` acompanha a execução até ela terminar; `gh run view --log` mostra os logs. Uma execução agendada ou manual pode consumir recursos de Actions dependendo do plano, runner e tipo de repositório. Veja os limites e a cobrança na seção Enterprise de Integração com GitHub.

8. Referências oficiais

- Entender GitHub Actions
- Sintaxe dos workflows
- Eventos que iniciam workflows
- Usar secrets em GitHub Actions
- Visualizar o histórico de execuções

2026-11-13 2026-11-13

Integração com GitHub

1. O que significa "integrar" a documentação com o repositório

Até aqui, o Documentation_Site foi tratado como um artefato isolado: um site estático gerado a partir dos arquivos em `docs/`. O MkDocs Material, porém, sabe se conectar ao repositório Git/GitHub onde esses arquivos vivem, e usar essa conexão para acrescentar, automaticamente, três coisas em cada página do site:

1. Um **link (ou ícone) para o repositório** no topo do site, apontando para a página do projeto no GitHub.
2. **Botões de ação no topo de cada página**: um para abrir o arquivo Markdown daquela página direto no GitHub, pronto para edição (e eventual Pull Request), e outro para apenas visualizar o Markdown bruto no repositório.
3. A **data da última modificação** daquela página (e, opcionalmente, a data de criação), lida do histórico de commits do arquivo no Git.

Nenhuma dessas três coisas depende de hospedar o site no GitHub Pages especificamente — elas dependem apenas de o `mkdocs.yml` apontar para onde o repositório (ou pelo menos seu histórico Git local) está. As três próximas seções explicam, respectivamente, como cada uma é configurada neste projeto.

2. `repo_url`, `repo_name` e `edit_uri`

O `mkdocs.yml` deste projeto define:

```
repo_url: https://github.com/dataRainBR/mkdocs-documentation
repo_name: dataRainBR/mkdocs-documentation
edit_uri: edit/main/docs/
```

- **`repo_url`**: a URL do repositório. É esse valor que o Material usa para exibir o ícone do GitHub (ou GitLab/Bitbucket, dependendo do domínio) no cabeçalho do site, funcionando como um link direto para a página do projeto.
- **`repo_name`**: o texto exibido junto desse link no cabeçalho. Aqui usa o formato organização/repositório, mas poderia ser qualquer rótulo, como apenas o nome do projeto.
- **`edit_uri`**: o caminho relativo, anexado a `repo_url`, usado para montar o link "editar esta página" de cada arquivo. O valor `edit/main/docs/` instrui o MkDocs a montar URLs no formato `repo_url/edit/main/docs/<caminho-do-arquivo>.md` — ou seja, "editar", na branch `main`, dentro da pasta `docs/`. É exatamente essa a URL que o GitHub usa para abrir seu editor de arquivos online.

Ao usar este modelo em outro repositório, substitua a organização, o repositório e a branch pelos valores reais:

Confirme o nome da branch padrão e onde os arquivos Markdown estão localizados. Se a branch se chamar `master`, por exemplo, altere `edit_uri` para usar `edit/master/docs/`.

3. `theme.features`: `content.action.edit` e `content.action.view`

O `mkdocs.yml` também habilita, na seção `theme.features`:

```
theme:
  features:
    - content.action.edit
    - content.action.view
```

Essas duas features controlam os botões/ícones exibidos no topo de **cada página** do site:

- **`content.action.edit`**: adiciona um ícone de lápis que leva direto para a URL de edição daquele arquivo Markdown específico, montada a partir de `repo_url` + `edit_uri` + o caminho do arquivo dentro de `docs/`. Clicar nele abre o editor de arquivos do GitHub já na página certa, pronta para receber alterações e, a partir delas, abrir um Pull Request.
- **`content.action.view`**: adiciona um segundo ícone, de "ver código-fonte", que leva para a visualização do Markdown bruto daquela página no repositório (sem abrir o modo de edição) — útil para quem só quer confirmar o texto original antes de copiá-lo ou citá-lo.

Ambos os botões dependem inteiramente de `repo_url` e `edit_uri` estarem corretos: é esse par de valores que o Material usa para montar a URL exata de cada botão, para cada página, automaticamente — nenhuma configuração adicional por página é necessária.

4. `git-revision-date-localized`: data de última modificação

O `mkdocs.yml` configura o plugin `git-revision-date-localized` assim:

```
plugins:
  - git-revision-date-localized:
      fallback_to_build_date: true
      enable_creation_date: true
      type: timeago
```

Esse plugin lê o histórico `git log` de cada arquivo em `docs/` e, a partir dele, insere na página:

- a **data da última modificação** do arquivo (sempre), obtida do commit mais recente que o alterou;
- a **data de criação** do arquivo (graças a `enable_creation_date: true`), obtida do commit mais antigo que o introduziu;

- ambas formatadas de forma relativa — "há 2 dias", "há 3 meses" — em vez de uma data absoluta, graças a `type: timeago`.

O plugin depende do histórico Git dos arquivos. Para conferir se o clone local tem histórico:

```
git log --oneline -5
```

Se o checkout não incluir commits — por exemplo, um clone raso ou uma pasta ainda sem commits —, o plugin não terá datas históricas para exibir. A opção `fallback_to_build_date: true` permite usar a data do build como alternativa. No workflow deste projeto, `actions/checkout` usa `fetch-depth: 0` para disponibilizar o histórico completo.

5. GitHub Actions: o workflow real de CI/CD

Existe um workflow real em `.github/workflows/docs.yml`, e os valores de `repo_url`, `repo_name` e `edit_uri` no `mkdocs.yml` apontam para `dataRainBR/mkdocs-documentation`, na branch `main`. Esta seção apresenta o conteúdo desse workflow e explica, passo a passo, o que cada etapa faz.

5.1 O arquivo `.github/workflows/docs.yml`

```yaml
name: Build e publica a documentação

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Clonar o repositório
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Configurar Node.js (necessário para o TypeDoc)
        uses: actions/setup-node@v4
        with:
          node-version: "20"
```

```
      - name: Instalar TypeDoc
        run: npm install --global typedoc

      - name: Instalar o uv
        uses: astral-sh/setup-uv@v3

      - name: Instalar dependências Python
        run: uv sync

      - name: Instalar o Chromium do Playwright
        run: uv run playwright install --with-deps chromium

      - name: Gerar referências dos pacotes
        run: uv run python docs_preview.py --generate-references

      - name: Buildar a documentação
        run: uv run mkdocs build --strict

      - name: Publicar artefato do GitHub Pages
        uses: actions/upload-pages-artifact@v3
        with:
          path: site

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - name: Publicar no GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

O workflow dispara automaticamente a cada `push` na branch `main`, e também pode ser disparado manualmente a qualquer momento através do `workflow_dispatch`, direto na aba "Actions" do GitHub.

5.2 Passo a passo do job `build`

1. **Clonar o repositório** (`actions/checkout@v4` com `fetch-depth: 0`) — baixa o histórico completo de commits, em vez do padrão `fetch-depth: 1` (apenas o commit mais recente). Assim, o `git-revision-date-localized` consegue calcular as datas reais de modificação e criação das páginas.
2. **Configurar Node.js** (`actions/setup-node@v4`) **e instalar TypeDoc** (`npm install --global typedoc`) — a diretiva `calcular-media` (handler `typescript` do `mkdocstrings`, usada na página Exemplos de código) depende do TypeDoc. O runner do GitHub Actions também precisa ter o Node.js disponível para executar essa ferramenta.
3. **Instalar o `uv`** (`astral-sh/setup-uv@v3`) **e instalar as dependências Python** (`uv sync`) — preparam o ambiente Python do projeto a partir do `pyproject.toml` / `uv.lock`, da mesma forma que um desenvolvedor faria localmente ao rodar `uv sync` na própria máquina.

4. **Instalar o Chromium do Playwright** (`uv run playwright install --with-deps chromium`) — instala o navegador Chromium (e suas dependências de sistema) necessário para o plugin `exporter` do `mkdocs.yml`, que abre cada página renderizada em um navegador headless para gerar o PDF correspondente durante o build. Sem essa etapa, o build falharia ao tentar abrir um navegador que ainda não está instalado no runner.
5. **Gerar referências dos pacotes** (`uv run python docs_preview.py --generate-references`) — cria ou atualiza `docs/referencias.md` a partir do `pyproject.toml` (ou de `requirements.txt` como alternativa), antes que o MkDocs transforme as páginas em HTML.
6. **Buildar a documentação** (`uv run mkdocs build --strict`) — gera o site estático completo na pasta `site/`, já incluindo a página de referências atualizada. A flag `--strict` faz o build falhar (em vez de só avisar) caso exista qualquer referência quebrada, como um item de `nav` apontando para um arquivo inexistente — garantindo que um site incompleto nunca chegue a ser publicado.
7. **Publicar artefato do GitHub Pages** (`actions/upload-pages-artifact@v3`, apontando para `site`) — empacota o conteúdo da pasta `site/`, já gerada pela etapa anterior, como um artefato de build do GitHub Actions, pronto para ser consumido pelo job `deploy` a seguir.

5.3 O job `deploy`

O job `deploy` depende do `build` através de `needs: build`, ou seja, só começa a executar se todas as etapas do `build` tiverem sido concluídas com sucesso. Ele usa o ambiente especial `github-pages` do GitHub — que expõe a URL publicada do site em `steps.deployment.outputs.page_url` — e publica o artefato gerado pelo job `build` através de `actions/deploy-pages@v4`.

Essa é, atualmente, a forma oficialmente recomendada pelo GitHub para publicar um site no GitHub Pages a partir de um workflow, substituindo o método mais antigo de fazer commit do site estático gerado em uma branch dedicada (normalmente chamada `gh-pages`). Com `actions/deploy-pages`, não existe uma branch extra para manter sincronizada: o artefato é publicado diretamente a partir do build.

5.4 As `permissions` do workflow

```yaml
permissions:
  contents: read
  pages: write
  id-token: write
```

Essas três permissões controlam o que o token automático do workflow (`GITHUB_TOKEN`, gerado pelo próprio GitHub a cada execução) está autorizado a fazer:

- **`contents: read`**: permite ao workflow ler o código do repositório — necessário para a etapa de checkout.
- **`pages: write`**: autoriza o workflow a publicar conteúdo no GitHub Pages.

- `id-token: write`: permite que o job `deploy` obtenha um token OIDC de curta duração, usado internamente por `actions/deploy-pages` para autenticar a publicação.

Com essas três permissões declaradas, o workflow publica no GitHub Pages usando apenas o `GITHUB_TOKEN` automático — sem que você precise criar, configurar ou armazenar nenhum secret ou token de acesso pessoal manualmente no repositório.

5.5 Configuração necessária no GitHub

As etapas acima cobrem tudo o que acontece *dentro* do workflow, mas existe um passo manual que precisa ser feito uma única vez, fora dele, direto no site do GitHub:

No repositório `dataRainBR/mkdocs-documentation`, acesse **Settings** → **Pages** → **Build and deployment** → **Source** e selecione a opção **"GitHub Actions"** — em vez da opção padrão **"Deploy from a branch"**.

Sem essa configuração manual, o GitHub Pages não tem como aceitar o artefato publicado por `actions/deploy-pages`, mesmo que o workflow esteja configurado corretamente: a opção padrão ("Deploy from a branch") espera que o conteúdo do site seja enviado por commit a uma branch específica, não por um artefato de build.

5.6 Acessar a documentação publicada

Depois de selecionar **GitHub Actions** como fonte do Pages, envie as alterações para a branch `main`. O evento `push` configurado em `.github/workflows/docs.yml` inicia automaticamente o workflow de build e publicação:

```
git add .
git commit -m "docs: atualiza documentação"
git push origin main
```

Antes de enviar, confira `git status` e revise as alterações para garantir que apenas os arquivos desejados serão publicados. Se o primeiro push ocorreu **antes** de configurar o Pages, a alteração da fonte, por si só, não inicia uma nova execução: faça um novo push para `main` ou dispare o workflow manualmente, conforme descrito abaixo.

Pela interface do GitHub

1. Abra o repositório no GitHub e selecione a aba **Actions**.
2. Na lista de workflows, escolha **Build e publica a documentação**.
3. Abra a execução mais recente da branch `main`. Verifique se os jobs `build` e `deploy` terminaram com sucesso (indicados como concluídos).
4. Quando a publicação terminar, abra **Settings** → **Pages**. Na seção do GitHub Pages, use o endereço do site exibido para acessar a documentação.

5. Se o endereço ainda não aparecer, confirme em **Settings** → **Pages** → **Build and deployment** se a fonte está como **GitHub Actions**, e confira os detalhes e logs da execução na aba **Actions**.

Também é possível abrir a execução e acompanhar o job `deploy`; o ambiente `github-pages` pode exibir o endereço publicado como link da implantação.

Pelo terminal com `gh`

Autentique-se no repositório e confira as execuções recentes:

```
gh auth status
gh workflow list
gh run list --workflow docs.yml --branch main
```

Copie o ID da execução mais recente e acompanhe até concluir:

```
gh run watch ID-DA-EXECUCAO
gh run view ID-DA-EXECUCAO
```

Para abrir os detalhes da execução no navegador:

```
gh run view ID-DA-EXECUCAO --web
```

Depois de uma execução bem-sucedida, consulte pelo `gh` o endereço configurado no GitHub Pages:

```
gh api repos/dataRainBR/mkdocs-documentation/pages --jq .html_url
```

O comando imprime a URL do site; abra-a no navegador. Se a consulta retornar erro ou ainda não houver URL, confirme a configuração em **Settings** → **Pages** e verifique se o job `deploy` concluiu com sucesso.

Se precisar iniciar uma execução sem fazer um novo push, o workflow também pode ser iniciado manualmente porque declara `workflow_dispatch`:

```
gh workflow run docs.yml --ref main
gh run list --workflow docs.yml --branch main
```

O primeiro comando agenda a execução; o segundo permite localizar o ID para acompanhá-la com `gh run watch`. A publicação só estará disponível quando o build e o deploy terminarem com sucesso.

6. MkDocs com GitHub Enterprise e repositórios privados

GitHub Enterprise tem duas opções de implantação, e é importante distingui-las:

- **GitHub Enterprise Cloud (GHEC)** é o serviço hospedado pelo GitHub em `github.com` ou no domínio de residência de dados da empresa. Ele permite publicar um GitHub Pages privado para uma organização Enterprise Cloud.

- **GitHub Enterprise Server (GHES)** é instalado e administrado pela própria empresa. O endereço da instância, o acesso de rede, as políticas de Actions, os runners e a disponibilidade de Pages dependem da configuração e da versão mantidas pelos administradores.

6.1 Apontar MkDocs para o GitHub Enterprise

No GitHub Enterprise Cloud padrão, os URLs do repositório usam `github.com`. No Enterprise Server, copie a URL HTTPS do repositório na instância da empresa. Use o mesmo host em `repo_url`; o `edit_uri` deve apontar para a branch e a pasta que contêm os Markdown:

```
repo_url: https://github.empresa.example/ORGANIZACAO/REPOSITORIO
repo_name: ORGANIZACAO/REPOSITORIO
edit_uri: edit/main/docs/
```

Configure também `site_url` com o endereço que aparece nas configurações do GitHub Pages. Sites privados podem usar um subdomínio diferente do endereço público:

```
site_url: https://SUBDOMINIO-PRIVADO.pages.github.io/
```

Confirme o formato do host e do caminho com a opção de clonar do repositório Enterprise e com a página **Settings** → **Pages** da instância. Em GHES, o administrador pode usar caminhos ou domínios próprios.

6.2 Construir e publicar a partir de um repositório privado

O workflow de MkDocs pode permanecer em `.github/workflows/docs.yml`. Para publicar com o workflow do GitHub Pages, ele precisa das permissões abaixo:

```
permissions:
  contents: read
  pages: write
  id-token: write
```

`actions/checkout` usa por padrão o `GITHUB_TOKEN` para ler o próprio repositório privado. Não é necessário criar um token pessoal apenas para clonar o código do workflow. Se o build depender de outro repositório privado, use um GitHub App ou um token de acesso fino com somente a permissão de leitura necessária, guarde-o como secret e não o imprima nos logs:

```
- name: Clonar dependência privada
  uses: actions/checkout@v4
  with:
    repository: ORGANIZACAO/DEPENDENCIA-PRIVADA
    token: ${{ secrets.REPO_READ_TOKEN }}
    path: dependencias/privadas
```

Além do YAML, confirme que Actions está habilitado para a organização e que as políticas Enterprise permitem as actions utilizadas, incluindo `actions/checkout`, `actions/upload-pages-artifact` e `actions/deploy-pages`. Em GHES, o administrador também precisa disponibilizar runners e acesso de rede aos repositórios e dependências do build.

Para publicar o site:

1. Abra **Settings** → **Pages** → **Build and deployment** no repositório.
2. Selecione **GitHub Actions** como fonte de publicação.
3. Quando o plano e a organização permitirem, configure a visibilidade do site como **Private** em **Settings** → **Pages**.
4. Envie uma alteração ou inicie o workflow manualmente e confira o job `deploy` na aba **Actions**.

Atenção à visibilidade: um repositório privado não torna automaticamente o site Pages privado. Um site publicado como público pode ser acessado por qualquer pessoa, mesmo que o código-fonte esteja em um repositório privado. A publicação privada do Pages está disponível no **GitHub Enterprise Cloud**; nesse modo, o site fica acessível somente a pessoas com acesso de leitura ao repositório. Não publique dados confidenciais no site, nos artefatos ou nos logs do workflow.

No GitHub Enterprise Server, não presuma que as opções de publicação privada do Enterprise Cloud estejam disponíveis. Confirme com o administrador a política de Pages, a URL interna e o modelo de autenticação aceito pela instalação.

6.3 Custos e limites

O GitHub Enterprise é um plano pago. A cobrança exata pode depender do contrato, da quantidade de usuários e dos produtos habilitados; consulte a página de preços e a área de faturamento do seu Enterprise. A documentação oficial de cobrança do Actions registra as cotas atuais de compute e armazenamento:

Item do GitHub Enterprise Cloud	**Referência oficial consultada em novembro de 2026**
Assinatura Enterprise	A página pública de preços lista atualmente **US$ 21 por usuário/mês** para Enterprise. Contratos empresariais podem ter condições diferentes; confira a página de preços do GitHub e o contrato da organização.
Compute incluído do Actions	A documentação de cobrança lista US$ 300 de uso de compute incluído por ciclo, aproximadamente 50.000 minutos em runner Linux padrão. A página de preços também apresenta 50.000 minutos mensais para Enterprise.

Item do GitHub Enterprise Cloud	Referência oficial consultada em novembro de 2026
Armazenamento de artefatos	A documentação de cobrança lista 50 GB compartilhados com o armazenamento do GitHub Packages.
Cache do Actions	10 GB por repositório incluídos; uso acima da franquia pode ser cobrado conforme a configuração e o uso.
Pages privado	Requer uma organização no GitHub Enterprise Cloud; o custo da assinatura Enterprise se aplica.

O uso de runners hospedados pelo GitHub em repositórios privados ou internos pode consumir o compute incluído; após a franquia, o uso adicional é cobrado se houver faturamento habilitado. Os custos são atribuídos ao proprietário do repositório. A documentação atual também informa que runners self-hosted não geram cobrança de uso do Actions, mas a infraestrutura, operação e manutenção dessas máquinas continuam sendo responsabilidade da empresa. O uso padrão de runners para GitHub Pages é indicado como gratuito pela documentação de cobrança.

No GitHub Enterprise Server, licenciamento, infraestrutura e cobrança dependem do contrato e da instalação; a tabela do GitHub Enterprise Cloud não deve ser aplicada automaticamente ao GHES.

Consulte sempre os valores vigentes antes de estimar o orçamento:

- Preços do GitHub
- Cobrança do GitHub Actions e cotas por plano
- Calculadora de preços do Actions
- Ver uso e faturamento dos produtos
- GitHub Enterprise Cloud
- Publicar um site GitHub Pages privado
- Configurar uma fonte de publicação do Pages

Para instruções de colaboração e versionamento, veja Git. Para configurar e acompanhar automações, veja GitHub Actions.

2026-11-13 2026-11-11

GitHub CLI (`gh`)

O **GitHub CLI**, chamado pelo comando `gh`, permite trabalhar com repositórios, issues, pull requests e GitHub Actions pelo terminal. Ele complementa o Git: comandos como `git pull` e `git push` continuam sendo comandos Git, enquanto `gh` facilita operações que normalmente seriam feitas pela interface web do GitHub. Para um guia dedicado ao versionamento local, consulte Git; para workflows e arquivos YAML, consulte GitHub Actions.

Neste capítulo, os comandos são apresentados no PowerShell e também funcionam, em geral, em terminais macOS e Linux. Não cole tokens ou senhas em comandos compartilhados, arquivos do projeto ou capturas de tela.

1. Criar uma conta no GitHub

Se você ainda não tem uma conta:

1. Acesse github.com/signup.
2. Informe seu e-mail, crie uma senha e escolha um nome de usuário.
3. Conclua a verificação solicitada e confirme o endereço de e-mail.
4. Ative a autenticação de dois fatores nas configurações de segurança da conta.

Criar uma conta é uma etapa feita no site; o `gh` não cria contas. Se o trabalho for em uma organização, peça acesso ao repositório antes de tentar enviar alterações.

2. Instalar o GitHub CLI

O GitHub CLI precisa estar instalado separadamente do Git. Se ainda não tiver Git, instale-o também: o `gh` usa o Git para operações com os arquivos e o histórico local.

Windows

No PowerShell, com o gerenciador `winget` disponível:

```
winget install --id GitHub.cli
```

Feche e reabra o terminal após a instalação e confira:

```
gh --version
git --version
```

macOS

Com o Homebrew:

```
brew install gh
```

Linux e outros métodos

As instruções de instalação variam conforme a distribuição Linux e o gerenciador de pacotes. Consulte o guia oficial: Instalar GitHub CLI.

3. Autenticar e configurar o Git

No terminal, inicie o login:

```
gh auth login
```

Responda às perguntas interativas. Para uma conta comum, escolha `GitHub.com`, selecione HTTPS ou SSH e autentique pelo navegador quando solicitado. O fluxo pelo navegador evita que você precise copiar uma senha para o terminal.

Verifique o estado da autenticação:

```
gh auth status
```

Se escolheu HTTPS e quer que o Git use as credenciais gerenciadas pelo `gh`, execute:

```
gh auth setup-git
```

Configure o nome e o e-mail que serão gravados nos commits Git (use o e-mail que deseja associar aos commits):

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@example.com"
```

Essas configurações identificam os commits; elas não autenticam você no GitHub. A autenticação é feita pelo `gh auth login` (ou por uma chave SSH configurada).

4. Criar ou obter um repositório

Clonar um repositório existente

```
gh repo clone ORGANIZACAO/REPOSITORIO
```

```
cd REPOSITORIO
```

Por exemplo:

```
gh repo clone dataRainBR/mkdocs-documentation
cd mkdocs-documentation
```

O comando baixa o repositório e configura o remote `origin`. Confira os remotes com:

```
git remote -v
```

Criar um repositório

Para criar um repositório no GitHub e também conectar a pasta atual:

```
gh repo create NOME-DO-REPOSITORIO --public --source=. --remote=origin
```

Use `--private` em vez de `--public` para um repositório privado. Revise cuidadosamente os arquivos antes de publicar um repositório público.

Para criar o repositório e já enviar os commits locais:

```
gh repo create NOME-DO-REPOSITORIO --private --source=. --remote=origin --push
```

`--push` envia o histórico existente para o GitHub. Não o use sem conferir se a pasta e o conteúdo são os corretos.

5. Fluxo básico: atualizar, alterar e enviar

Um fluxo comum de colaboração é criar uma branch para cada alteração. Evite trabalhar diretamente na branch principal do projeto.

```
git switch main
git pull
git switch -c docs/novo-capitulo
```

- `git switch main` muda para a branch principal (ela pode se chamar `master` ou ter outro nome).
- `git pull` baixa e integra as alterações recentes da branch rastreada.
- `git switch -c ...` cria uma branch nova e muda para ela.

Depois de editar os arquivos, revise e registre as alterações:

```
git status
git diff
```

```
git add docs/
git commit -m "docs: adiciona capítulo sobre GitHub CLI"
```

`git add` prepara arquivos para o commit; `git commit` salva um ponto no histórico local. Ajuste o caminho após `git add` aos arquivos que você realmente deseja incluir.

Envie a branch ao GitHub:

```
git push -u origin docs/novo-capitulo
```

`-u` configura o acompanhamento da branch remota; nos próximos envios dessa mesma branch, normalmente basta `git push`. Para trazer depois novas alterações da branch atual, use `git pull`.

> **Git e `gh` têm papéis diferentes:** `git pull`, `git push`, `git add`, `git commit` e `git switch` trabalham com o repositório e seus arquivos. O `gh` autentica com o GitHub e opera recursos como repositórios, issues, pull requests e Actions.

6. Pull requests e merge

Depois do `git push`, abra um pull request (PR) para propor a integração das mudanças na branch principal:

```
gh pr create --base main --title "Adiciona capítulo sobre GitHub CLI" --body "Documenta instalação, autenticação, Git e GitHub Actions."
```

Se preferir responder às perguntas no terminal, use `gh pr create` sem argumentos. O PR pode ser revisado pela interface web ou pelo terminal.

Comandos úteis:

```
gh pr list
gh pr status
gh pr view
gh pr view 42
gh pr diff 42
gh pr checkout 42
```

Substitua `42` pelo número do PR. `gh pr checkout` baixa e muda para a branch do PR selecionado.

Após a revisão e a aprovação das verificações exigidas, faça o merge:

```
gh pr merge 42 --squash --delete-branch
```

`--squash` combina os commits do PR em um único commit; alternativas comuns são `--merge` e `--rebase`. A exclusão da branch local/remota é opcional. O repositório pode exigir aprovação, verificações

bem-sucedidas ou uma fila de merge; nesse caso, respeite as regras configuradas. Para solicitar merge automático quando os requisitos forem satisfeitos:

```
gh pr merge 42 --auto --squash
```

Após o merge, atualize sua branch principal:

```
git switch main
git pull
```

7. Funcionalidades principais do `gh`

O `gh` tem comandos para as principais tarefas do GitHub. Alguns exemplos:

```
gh repo view
gh issue list
gh issue create
gh pr list
gh release list
gh workflow list
gh run list
```

Para consultar a ajuda de um grupo de comandos ou de uma operação:

```
gh help
gh help pr
gh help workflow run
```

Também é possível indicar outro repositório com `--repo ORGANIZACAO/REPOSITORIO` (ou `-R ORGANIZACAO/REPOSITORIO`) em comandos que aceitam essa opção.

8. GitHub Actions pelo terminal

GitHub Actions executa automações definidas em arquivos YAML dentro de `.github/workflows/`. Um workflow pode iniciar, por exemplo, ao enviar commits (`push`), abrir ou atualizar um pull request (`pull_request`) ou ser iniciado manualmente (`workflow_dispatch`).

Consultar workflows e execuções

```
gh workflow list
gh workflow view docs.yml
gh run list
```

Para ver uma execução específica, incluindo seus passos e logs:

```
gh run view ID-DA-EXECUCAO
gh run view ID-DA-EXECUCAO --log
```

Para acompanhar uma execução em andamento até terminar:

```
gh run watch ID-DA-EXECUCAO
```

Iniciar e reexecutar workflows

Para iniciar manualmente um workflow que declare `workflow_dispatch`:

```
gh workflow run docs.yml --ref main
```

O comando agenda a execução; não significa que ela já foi concluída. Consulte `gh run list` para obter o ID e use `gh run watch` para acompanhar o resultado.

Para reexecutar uma execução anterior:

```
gh run rerun ID-DA-EXECUCAO
```

Se o workflow não aparecer, confira se o arquivo existe em `.github/workflows/`, se está habilitado e se a branch selecionada contém a versão correta dele. Para a execução manual, o YAML precisa incluir o evento `workflow_dispatch`.

Exemplo de workflow de documentação

Este exemplo valida a documentação deste projeto em cada push ou pull request. Ele instala as dependências Python e as ferramentas TypeScript e Chromium usadas pelo `mkdocs.yml`:

```yaml
name: Validar documentação

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Baixar o repositório
        uses: actions/checkout@v4

      - name: Configurar Node.js
        uses: actions/setup-node@v4
        with:
```

```
          node-version: "20"

      - name: Instalar TypeDoc
        run: npm install --global typedoc

      - name: Instalar uv
        uses: astral-sh/setup-uv@v3

      - name: Instalar dependências Python
        run: uv sync

      - name: Instalar Chromium do Playwright
        run: uv run playwright install --with-deps chromium

      - name: Validar o build
        run: uv run mkdocs build --strict
```

O workflow de build e publicação deste projeto já está em `.github/workflows/docs.yml`; não crie outro arquivo com o mesmo nome. Use o exemplo para entender as etapas ou adaptar um workflow equivalente. O push ou a abertura/atualização de um PR dispara a execução; também é possível iniciá-la com `gh workflow run docs.yml --ref main`.

Este exemplo só faz o build e validação; ele não publica o site. Para publicar no GitHub Pages, configure o Pages e as permissões necessárias conforme Integração com GitHub. O workflow de publicação deste projeto já está em `.github/workflows/docs.yml`.

Segredos e permissões

- Guarde credenciais e chaves em **Settings** → **Secrets and variables** → **Actions**; nunca as escreva diretamente no YAML, no código ou em logs.
- Em workflows, dê ao `GITHUB_TOKEN` somente as permissões necessárias ao trabalho. A permissão de escrita não deve ser concedida sem necessidade.
- O `gh auth login` é para a sua sessão local. Nos workflows, prefira o `GITHUB_TOKEN` fornecido pelo GitHub, em vez de copiar para o workflow o token pessoal da sua conta.
- Ao usar workflows de terceiros ou ações externas, confira a origem, o código e as permissões antes de executá-los.

9. Referências oficiais

- GitHub CLI: instalação
- Manual do GitHub CLI
- Autenticação com GitHub CLI
- Workflows do GitHub Actions pelo `gh`
- Execuções do GitHub Actions pelo `gh`

- GitHub Actions: entender workflows

2026-11-13 2026-11-11